Botho

A Privacy-Preserving Cryptocurrency with Hybrid Post-Quantum Security and Fair Economics

Version 1.0

The Botho Project
https://botho.io

November 2, 2026

Abstract

We present Botho, a privacy-preserving cryptocurrency that combines efficient ring signatures with post-quantum stealth addresses, achieving strong privacy guarantees with practical performance. Unlike existing privacy coins that either lack quantum resistance or impose impractical overhead, Bothoemploys a *hybrid architecture*: post-quantum cryptography (ML-KEM-768) protects permanent on-chain data (recipient identities), while efficient classical cryptography (CLSAG ring signatures) provides ephemeral sender anonymity.

Bothointroduces a hybrid consensus mechanism combining proof-of-work block proposal with Stellar Consensus Protocol (SCP) finalization, achieving deterministic finality in approximately 5 seconds while maintaining permissionless participation. The protocol includes *progressive fees* based on coin provenance (cluster tags), a cluster-scaled demurrage term, and a lottery: 80% of transaction fees, plus a height-scheduled share of emission, are paid to uniformly drawn outputs, and 20% of fees are burned. These mechanisms are designed to counter wealth concentration, but a measurable reduction in inequality is *not demonstrated* at production parameters: a code-faithful simulation measures the mechanism's effect on the Gini coefficient at -0.028 to $+0.003$ against a 0.05 design bar, lottery ticket-splitting is currently profitable, and a redesign is under study (Section 10.6.2).

Key parameters: ring size 20, transaction size $\sim$4 KB, 5-second finality, perpetual 2% tail emission with dynamic inflation dampening. We provide formal security proofs for transaction unlinkability, double-spend prevention, and fork freedom under standard cryptographic assumptions.

Keywords: cryptocurrency, privacy, ring signatures, post-quantum cryptography, consensus protocol, monetary policy

Contents

1 Introduction **11**
- 1.1 The Privacy Problem . . . 11
- 1.2 The Fairness Problem . . . 11
- 1.3 Design Philosophy . . . 11
- 1.4 Contributions . . . 12
- 1.5 Paper Organization . . . 12

2 Related Work **13**
- 2.1 Privacy Cryptocurrencies . . . 13
  - 2.1.1 CryptoNote and Monero . . . 13

2.1.2 MobileCoin . . . 14
2.1.3 Zcash . . . 14
2.1.4 Zcash Orchard . . . 15
2.1.5 Firo (Lelantus Spark) . . . 15
2.1.6 Secret Network . . . 15
2.1.7 Aztec . . . 16
2.1.8 MimbleWimble . . . 16
2.2 Post-Quantum Approaches . . . 17
2.2.1 Quantum Threat Model . . . 17
2.2.2 Existing Post-Quantum Cryptocurrencies . . . 17
2.2.3 Post-Quantum Ring Signature Research . . . 17
2.2.4 Hybrid Approaches . . . 18
2.3 Consensus Mechanisms . . . 18
2.3.1 Nakamoto Consensus . . . 18
2.3.2 Classical BFT . . . 18
2.3.3 Stellar Consensus Protocol . . . 19
2.4 Economic Mechanisms . . . 19
2.4.1 Fixed Supply . . . 19
2.4.2 Tail Emission . . . 19
2.4.3 Demurrage . . . 19
2.4.4 Novel: Progressive Fees . . . 19

3 Preliminaries **20**
3.1 Notation . . . 20
3.2 Cryptographic Assumptions . . . 20
3.3 Building Blocks . . . 20
3.3.1 Elliptic Curve Group . . . 20
3.3.2 Pedersen Commitments . . . 21
3.3.3 Bulletproofs . . . 21
3.3.4 ML-KEM-768 . . . 21
3.4 Security Definitions . . . 22

4 Cryptographic Protocol **22**
4.1 Key Hierarchy . . . 22
4.1.1 Master Seed Generation . . . 22
4.1.2 Account Key Derivation . . . 22
4.1.3 Subaddress Derivation . . . 23
4.2 Post-Quantum Stealth Addresses . . . 24
4.2.1 Protocol Description . . . 24
4.2.2 Security Analysis . . . 25
4.3 Ring Signatures (CLSAG) . . . 27
4.3.1 Ring Construction . . . 27
4.3.2 Signature Generation . . . 27
4.3.3 Verification . . . 27
4.3.4 Security Properties . . . 28
4.4 Confidential Transactions . . . 28
4.4.1 Amount Commitment . . . 28
4.4.2 Value Conservation . . . 29
4.4.3 Range Proofs . . . 29
4.5 Minting Attribution . . . 29
4.6 Hybrid Architecture Rationale . . . 30

5 Transaction Format **31**
5.1 Transaction Types . . . 31
5.2 Private Transaction Structure . . . 31
5.2.1 Transaction Components . . . 31
5.2.2 Input Structure . . . 31
5.2.3 Output Structure . . . 32
5.3 Minting Transaction Structure . . . 32
5.4 Cluster Tags and Progressive Fees . . . 33
5.4.1 Tag Vector Representation . . . 34
5.4.2 Cluster Creation at Minting . . . 34
5.4.3 Tag Inheritance and Blending . . . 34
5.4.4 Tag Decay (Wallet Policy, as Built) . . . 34
5.4.5 Cluster Wealth Accumulation . . . 35
5.4.6 Cluster Factor Calculation . . . 35
5.4.7 Ring Signature Tag Propagation . . . 35
5.4.8 Sybil Resistance Analysis . . . 36
5.5 Validation Rules . . . 38
5.6 Transaction Size Analysis . . . 39
5.7 Decoy Selection . . . 39
5.8 Transaction Malleability . . . 40

6 Consensus Mechanism **40**
6.1 Design Rationale . . . 40
6.1.1 Why Not Pure PoW? . . . 40
6.1.2 Why Not Pure BFT? . . . 40
6.1.3 Hybrid Approach . . . 40
6.2 Stellar Consensus Protocol . . . 41
6.2.1 Quorum Slices . . . 41
6.2.2 Quorum Intersection . . . 42
6.2.3 Tiered Quorum Structure . . . 42
6.3 Consensus Phases . . . 42
6.3.1 Phase 1: Block Proposal (PoW) . . . 43
6.3.2 Phase 2: Nomination . . . 43
6.3.3 Phase 3: Ballot Protocol . . . 44
6.3.4 Phase 4: Externalize . . . 44
6.4 Block Structure . . . 44
6.5 Difficulty Adjustment . . . 45
6.6 Fork Resolution . . . 45
6.7 Timing Analysis . . . 46
6.8 Liveness Guarantees . . . 47
6.9 Security Properties . . . 47
6.9.1 Byzantine Fault Tolerance . . . 47
6.9.2 Nothing-at-Stake Resistance . . . 47
6.9.3 Long-Range Attack Resistance . . . 48
6.10 Comparison with Alternatives . . . 48
6.11 Mining Pool Considerations . . . 48
6.11.1 Pool Protocol Compatibility . . . 48
6.11.2 Block Reward Attribution . . . 48
6.11.3 SCP Participation . . . 49
6.11.4 Pool Centralization Risks . . . 49
6.11.5 Solo Mining Viability . . . 49

6.11.6 Decentralized Pool Alternatives . . . 50

7 Monetary Policy **50**
7.1 Design Goals . . . 50
7.2 Emission Schedule . . . 50
7.2.1 Initial Emission . . . 50
7.2.2 Tail Emission . . . 51
7.2.3 Supply Projection . . . 52
7.3 Dynamic Block Timing . . . 52
7.3.1 Mechanism . . . 52
7.3.2 Economic Implications . . . 53
7.3.3 Emission Bounds . . . 53
7.4 Fee Economics . . . 54
7.4.1 Base Fee . . . 54
7.4.2 Progressive Fee Multiplier . . . 54
7.4.3 Demurrage Class-Transition Charge . . . 54
7.4.4 Lottery-Based Fee Redistribution . . . 55
7.4.5 Lottery Mechanism . . . 56
7.4.6 Effective Inflation . . . 57
7.4.7 Fee Flow Analysis . . . 57
7.5 Minter Incentives . . . 58
7.5.1 Block Reward Composition . . . 58
7.5.2 Profitability Analysis . . . 58
7.5.3 Decentralization Incentives . . . 58
7.6 Long-Term Sustainability . . . 59
7.6.1 Security Budget . . . 59
7.6.2 Comparison with Fixed Supply . . . 59
7.6.3 Wealth Tax Equivalence . . . 59
7.7 Monetary Constants . . . 59

8 Network Protocol **59**
8.1 Network Architecture . . . 59
8.1.1 Node Types . . . 59
8.1.2 Transport Layer . . . 59
8.2 Peer Discovery . . . 60
8.2.1 Bootstrap Nodes . . . 60
8.2.2 Kademlia DHT . . . 60
8.2.3 Peer Limits . . . 61
8.3 Message Protocol . . . 61
8.3.1 Message Types . . . 61
8.3.2 Message Serialization . . . 61
8.3.3 Wire Format Specification . . . 61
8.4 Protocol State Machine . . . 63
8.4.1 Connection States . . . 63
8.4.2 Message Sequences . . . 63
8.4.3 Error Handling . . . 64
8.5 Block Propagation . . . 65
8.5.1 Compact Block Relay . . . 65
8.5.2 Block Validation . . . 65
8.6 Transaction Propagation . . . 65
8.6.1 Dandelion++ . . . 65
8.6.2 Transaction Validation . . . 66

8.7 Synchronization . . . 66
8.7.1 Initial Block Download . . . 66
8.7.2 Block Locator . . . 66
8.7.3 Pruning . . . 66
8.8 Denial-of-Service Protection . . . 67
8.8.1 Rate Limiting . . . 67
8.8.2 Peer Scoring . . . 67
8.8.3 Resource Bounds . . . 67
8.9 Privacy Considerations . . . 67
8.9.1 Traffic Analysis Resistance . . . 67
8.9.2 Tor/I2P Support . . . 67
8.10 Light Client Security . . . 67
8.10.1 Verification Capabilities . . . 68
8.10.2 Trust Model . . . 68
8.10.3 Output Scanning . . . 68
8.10.4 Transaction Submission . . . 69
8.10.5 Privacy Implications . . . 69
8.10.6 Comparison to Full Node Security . . . 69
8.10.7 Mobile Wallet Recommendations . . . 70
8.11 Network Constants . . . 70

9 Security Analysis **70**
9.1 Threat Model . . . 70
9.1.1 Adversary Capabilities . . . 70
9.1.2 Security Goals . . . 70
9.2 Privacy Analysis . . . 71
9.2.1 Recipient Unlinkability . . . 71
9.2.2 Sender Anonymity . . . 71
9.2.3 Amount Confidentiality . . . 71
9.2.4 Anonymity Set Analysis . . . 71
9.3 Formal Privacy Framework . . . 71
9.3.1 Privacy Definitions . . . 72
9.3.2 BothoPrivacy Properties . . . 72
9.3.3 Privacy Budget Analysis . . . 73
9.3.4 Metadata Leakage Quantification . . . 73
9.3.5 Information-Theoretic Bounds . . . 74
9.4 Consensus Security . . . 74
9.4.1 Double-Spend Resistance . . . 74
9.4.2 Byzantine Fault Tolerance . . . 75
9.4.3 51% Attack Resistance . . . 75
9.5 Cryptographic Security . . . 76
9.5.1 Hash Function Security . . . 76
9.5.2 Signature Security . . . 76
9.5.3 Key Encapsulation Security . . . 76
9.6 Attack Resistance . . . 76
9.6.1 Timing Attacks . . . 76
9.6.2 Transaction Graph Analysis . . . 76
9.6.3 Sybil Attacks . . . 77
9.6.4 Denial of Service . . . 77
9.6.5 Eclipse Attacks . . . 77
9.6.6 Selfish Mining . . . 77

9.7 Formal Security Properties . . . 78
9.7.1 Safety . . . 78
9.7.2 Liveness . . . 78
9.8 Security Comparison . . . 78
9.9 Detailed Attack Scenarios . . . 78
9.9.1 Scenario: Timing-Based Deanonymization . . . 78
9.9.2 Scenario: Chain Analysis Attack . . . 79
9.9.3 Scenario: Flood-and-Loot Attack . . . 79
9.9.4 Scenario: Lottery Grinding Attack . . . 80
9.9.5 Scenario: Eclipse Attack on Light Client . . . 81
9.9.6 Scenario: Progressive Fee Evasion . . . 81
9.10 Known Limitations . . . 82
9.10.1 Classical Ring Signature Vulnerability . . . 82
9.10.2 Metadata Leakage . . . 82
9.10.3 Implementation Risks . . . 82

10 Economic Analysis **82**
10.1 Incentive Alignment . . . 83
10.1.1 Miner Incentives . . . 83
10.1.2 User Incentives . . . 83
10.1.3 Validator Incentives . . . 83
10.2 Progressive Fee Analysis . . . 83
10.2.1 Fee Distribution . . . 83
10.2.2 Redistribution Flows . . . 84
10.2.3 Anti-Hoarding Dynamics . . . 84
10.3 Game-Theoretic Analysis . . . 85
10.3.1 Miner Strategy . . . 85
10.3.2 Validator Strategy . . . 85
10.3.3 User Strategy . . . 86
10.4 Long-Term Sustainability . . . 86
10.4.1 Security Budget . . . 86
10.4.2 Network Effect . . . 86
10.4.3 Fee Market Stability . . . 86
10.5 Comparison with Alternatives . . . 86
10.5.1 Bitcoin Model . . . 86
10.5.2 Monero Model . . . 86
10.6 Quantitative Economic Modeling . . . 86
10.6.1 Adversarial Redistribution Simulation . . . 87
10.6.2 Status of the Redistribution Claim . . . 88
10.6.3 Formal Nash Equilibrium Analysis . . . 89
10.6.4 Economic Attack Cost Analysis . . . 90
10.7 Economic Risks . . . 91
10.7.1 Low Adoption . . . 91
10.7.2 Mining Centralization . . . 91
10.7.3 Validator Cartel . . . 91
10.8 Economic Constants Summary . . . 91

11 The BTH ↔ wBTH Bridge **91**
11.1 Motivation and Scope . . . 92
11.2 Custody and Trust Model . . . 92
11.2.1 Threshold authorization . . . 93
11.2.2 Per-chain signature schemes . . . 93

11.2.3 Attestation envelopes and exactly-once semantics . . . 93
11.2.4 The three-Safe structure on Ethereum . . . 93
11.3 The Peg . . . 94
11.3.1 Factor-1-only wrapping (ADR 0003) . . . 94
11.3.2 The demurrage-settlement on-ramp (target-state) . . . 94
11.3.3 Post-wrap demurrage escape (accepted) . . . 94
11.4 Privacy at the Boundary . . . 95
11.5 Import Cluster Tagging . . . 95
11.5.1 Epoch-keyed import cluster . . . 95
11.5.2 Why the epoch key is load-bearing (Sybil-resistance) . . . 96
11.5.3 Shared fate and decay by circulation . . . 96
11.6 Security . . . 97
11.7 Implementation Status . . . 98

12 Implementation **100**
12.1 Reference Implementation . . . 100
12.1.1 Architecture Overview . . . 100
12.1.2 Dependencies . . . 100
12.1.3 Build Requirements . . . 101
12.2 Performance Characteristics . . . 101
12.2.1 Transaction Validation . . . 101
12.2.2 Transaction Creation . . . 101
12.2.3 Throughput . . . 101
12.2.4 Resource Usage . . . 101
12.3 Wallet Implementations . . . 101
12.3.1 Command-Line Wallet . . . 101
12.3.2 Desktop Wallet . . . 102
12.3.3 Exchange Scanner . . . 102
12.4 Security Measures . . . 102
12.4.1 Memory Protection . . . 102
12.4.2 Constant-Time Operations . . . 103
12.4.3 Input Validation . . . 103
12.5 Testing . . . 103
12.5.1 Test Coverage . . . 103
12.5.2 Test Networks . . . 103
12.6 Scalability Analysis . . . 103
12.6.1 Blockchain Growth . . . 103
12.6.2 UTXO Set Growth . . . 104
12.6.3 Key Image Database . . . 104
12.6.4 Cluster Tag Database . . . 104
12.6.5 Throughput Limits . . . 104
12.6.6 Comparison to Existing Systems . . . 105
12.6.7 Layer-2 Scaling . . . 105
12.6.8 Pruning Effectiveness . . . 105
12.6.9 Node Hardware Requirements . . . 106
12.7 Future Development . . . 106
12.7.1 Planned Improvements . . . 106
12.7.2 Post-Quantum Ring Signatures . . . 106
12.8 User Experience Design . . . 106
12.8.1 Address Format . . . 106
12.8.2 Transaction Confirmation UX . . . 107

12.8.3 Wallet Backup and Recovery . . . 107
12.8.4 Subaddress Management . . . 108
12.8.5 Fee Estimation . . . 108
12.8.6 Error Handling . . . 108
12.8.7 Privacy Indicators . . . 108
12.9 Testnet Deployment . . . 109
12.9.1 Testnet Parameters . . . 109
12.9.2 Faucet . . . 109
12.9.3 Genesis Block . . . 109
12.9.4 Testnet Reset Policy . . . 110
12.10 Deployment . . . 110
12.11 Code Availability . . . 110

13 Governance and Protocol Upgrades **110**
13.1 Design Principles . . . 111
13.1.1 Conservative by Default . . . 111
13.1.2 Transparency and Predictability . . . 111
13.2 Protocol Versioning . . . 111
13.2.1 Version Scheme . . . 111
13.3 Upgrade Mechanisms . . . 111
13.3.1 Soft Forks . . . 111
13.3.2 Hard Forks . . . 112
13.4 Emergency Procedures . . . 112
13.5 Decentralization and Fork Rights . . . 113
13.5.1 Fork Rights . . . 113

14 Conclusion **113**
14.1 Summary . . . 113
14.1.1 Key Contributions . . . 113
14.1.2 Design Philosophy . . . 113
14.2 Limitations and Future Work . . . 114
14.2.1 Current Limitations . . . 114
14.2.2 Future Research Directions . . . 114
14.2.3 Interoperability Architecture . . . 114
14.3 Broader Impact . . . 116
14.3.1 Privacy as Human Right . . . 116
14.3.2 Economic Implications . . . 116
14.4 Closing Remarks . . . 117

A Notation Reference **121**
A.1 Mathematical Notation . . . 122
A.2 Protocol Identifiers . . . 123
A.3 Network Constants . . . 123
A.4 Cryptographic Parameters . . . 123
A.5 Transaction Sizes . . . 123
A.6 Monetary Parameters . . . 123

B Parameter Justification **124**
B.1 Cryptographic Parameters . . . 124
B.1.1 Ring Size: 20 . . . 124
B.1.2 ML-KEM Security Level: 768 . . . 124
B.2 Consensus Parameters . . . 125

B.2.1 Block Time: 3–40 Seconds (Dynamic) . . . 125
B.2.2 Quorum Thresholds: 3-of-4 Infrastructure, 2-of-3 Community . . . 125
B.2.3 Difficulty Adjustment: Per-Block, Time-Driven . . . 126
B.3 Economic Parameters . . . 126
B.3.1 Initial Block Reward: 50 BTH . . . 126
B.3.2 Halving Period: 6,307,200 Blocks (∼1 Year) . . . 126
B.3.3 Tail Emission: 2.5% Gross Annual Rate (Supply-Dependent) . . . 127
B.3.4 Fee Split: 80% Lottery / 20% Burn . . . 128
B.3.5 Progressive Fee Multiplier: 1× to 6× . . . 128
B.3.6 Tag Decay: 0.95 per Transfer (Wallet Policy) . . . 128
B.4 Network Parameters . . . 129
B.4.1 Maximum Transaction Size: 100 KB . . . 129
B.4.2 Maximum Block Size: 2 MB . . . 129
B.4.3 Connection Limits: 8 Outbound, 117 Inbound . . . 129
B.5 Summary Table . . . 129

C Regulatory Considerations **129**
C.1 Privacy and Compliance Tension . . . 130
C.1.1 Design Philosophy . . . 130
C.2 View Key Disclosure . . . 130
C.2.1 View Key Capabilities . . . 130
C.2.2 Audit Scenarios . . . 131
C.2.3 View Key Limitations . . . 131
C.3 Selective Transparency Options . . . 131
C.3.1 Payment Proofs . . . 131
C.3.2 Income Proofs . . . 131
C.3.3 Balance Proofs . . . 132
C.4 Exchange Integration . . . 132
C.4.1 Know Your Customer (KYC) . . . 132
C.4.2 Travel Rule Compliance . . . 132
C.4.3 Suspicious Activity Reporting . . . 132
C.5 Comparison to Other Privacy Approaches . . . 133
C.6 Jurisdictional Considerations . . . 133
C.7 Technical Measures for Service Providers . . . 133
C.7.1 Deposit Monitoring . . . 133
C.7.2 Withdrawal Controls . . . 133
C.8 Limitations and Honest Assessment . . . 134
C.8.1 What Botho Cannot Provide . . . 134
C.8.2 Regulatory Risk . . . 134
C.9 Conclusion . . . 134

D Formal Specifications **134**
D.1 State Definitions . . . 134
D.1.1 Blockchain State . . . 134
D.1.2 Consensus State . . . 135
D.2 State Transition Functions . . . 135
D.2.1 Block Application . . . 135
D.2.2 Transaction Application . . . 136
D.2.3 Fee Processing . . . 137
D.3 Validity Predicates . . . 138
D.3.1 Transaction Validity . . . 138
D.3.2 Block Validity . . . 139

D.4 SCP State Machine . . . 140
D.4.1 Message Types . . . 140
D.4.2 State Transitions . . . 140
D.5 Invariants . . . 141

E Security Audit Guide **142**
E.1 Security Claims . . . 142
E.1.1 Cryptographic Security . . . 142
E.1.2 Consensus Security . . . 142
E.1.3 Economic Security . . . 142
E.2 Threat Model . . . 143
E.3 Audit Scope . . . 143
E.3.1 Critical Components . . . 143
E.3.2 Test Vectors . . . 144
E.4 Known Limitations . . . 144
E.5 Reporting . . . 144

1 Introduction

"Motho ke motho ka batho"—A person is a person through other people.
—Sesotho proverb

1.1 The Privacy Problem

Financial surveillance has become pervasive in the digital age. Traditional banking systems create detailed records of every transaction, enabling comprehensive monitoring of individuals' economic activities. The emergence of Bitcoin [43] and subsequent cryptocurrencies initially promised an alternative—yet most blockchain systems create permanent, public records that enable even more thorough surveillance than traditional finance.

The transparency of Bitcoin's blockchain allows any observer to trace the flow of funds, link addresses to identities, and reconstruct complete financial histories [39]. Chain analysis companies have built entire industries around deanonymizing cryptocurrency users. This surveillance capability extends not only to law enforcement but to any party with access to blockchain data—including authoritarian governments, stalkers, and criminals seeking targets.

Privacy is not merely a preference; it is a prerequisite for human dignity. Financial privacy protects victims of domestic abuse, enables charitable giving without social pressure, preserves commercial confidentiality, and shields individuals from discrimination based on their economic activities. The Universal Declaration of Human Rights [64] recognizes privacy as a fundamental right.

Compounding this concern is the emerging quantum computing threat. Current cryptographic systems, including those protecting cryptocurrency addresses, rely on mathematical problems believed to be computationally hard for classical computers. Quantum computers, when sufficiently powerful, will render many of these protections obsolete [27, 61]. The "harvest now, decrypt later" threat model suggests that adversaries may already be collecting encrypted data for future decryption—including blockchain transactions that will remain on-chain indefinitely.

1.2 The Fairness Problem

Beyond privacy, existing cryptocurrencies exhibit troubling economic dynamics. Early adopter advantage creates extreme wealth concentration: those who acquired Bitcoin in 2010 for pennies now hold assets worth tens of thousands of dollars per coin. This dynamic transfers wealth from late adopters to early adopters without corresponding value creation.

Traditional fee markets create perverse incentives. When fees flow directly to block producers, pressure builds toward mining centralization—larger operations achieve lower marginal costs through economies of scale. Fee volatility during network congestion disproportionately harms small transactions and users in developing economies.

Fixed supply caps, while appealing for scarcity, create deflationary spirals that discourage spending and economic activity. The security budget problem looms: as Bitcoin's block rewards diminish toward zero, network security will depend entirely on transaction fees, creating uncertainty about long-term viability.

1.3 Design Philosophy

The name **Botho** (pronounced BOH-toh) comes from the Sesotho and Setswana languages of Southern Africa, meaning *humanity*, *humaneness*, or *ubuntu*. It is a national principle of Botswana and a core philosophy across many African cultures.

The opening proverb—*"Motho ke motho ka batho"*—translates to "a person is a person through other people." It expresses the idea that our humanity is defined by our relationships and responsibilities to one another, not by individual accumulation.

In currency design, this philosophy rejects the "number go up" mentality. Instead, Bothoasks: *how can money serve community rather than concentrate power?*

This philosophy manifests in concrete design decisions:

- **Privacy as baseline**: All transactions are private by default, not as a premium feature. Privacy protects the dignity of all participants.
- **Pragmatic security**: Rather than maximizing any single property, we make deliberate tradeoffs based on actual threat models. Permanent data receives permanent protection; ephemeral data can use efficient classical cryptography.
- **Fair economics**: Progressive fees, cluster-scaled demurrage and a fee-and-emission lottery are aimed at wealth concentration; that they measurably reduce it is not yet demonstrated (Section 10.6.2). Perpetual tail emission ensures sustainable security. Fee burning aligns incentives with network health.
- **Community consensus**: The Stellar Consensus Protocol [38] enables decisions by community agreement rather than hashpower dominance.

1.4 Contributions

This paper presents Botho, a privacy-preserving cryptocurrency with the following novel contributions:

1. **Hybrid post-quantum architecture**: We introduce a principled framework for applying post-quantum cryptography selectively based on data lifetime. Recipient identities, which persist on-chain indefinitely, are protected by ML-KEM-768 [6, 45]. Sender privacy, which has ephemeral value, uses efficient CLSAGring signatures [25]. This achieves meaningful quantum resistance while keeping transactions practical (∼4 KB vs. ∼65 KB for full post-quantum).
2. **PoW + SCP consensus**: We combine proof-of-work block proposal with Stellar Consensus Protocol finalization. This achieves permissionless participation (anyone can mine) with fast deterministic finality (∼5 seconds) and Byzantine fault tolerance.
3. **Progressive fee mechanism**: We introduce cluster-based fees that increase with wealth concentration. Unlike identity-based approaches, this preserves privacy while resisting Sybil attacks—fees are based on coin *ancestry*, not current ownership. The fee schedule is progressive; a measurable reduction in inequality from the combined mechanism is not demonstrated at production parameters (Section 10.6.2).
4. **Dynamic block timing**: Block intervals adapt to network load (3–40 seconds, 5-second steady-state reference), creating natural inflation dampening. Low network utility produces fewer blocks and less emission; high utility produces more blocks and rewards participants.

1.5 Paper Organization

The remainder of this paper is organized as follows:

- **Section 2** reviews related work in privacy cryptocurrencies, post-quantum approaches, consensus mechanisms, and economic designs.
- **Section 3** establishes notation and reviews cryptographic building blocks.

- **Section 4** details the cryptographic protocol including stealth addresses, ring signatures, and confidential transactions.
- **Section 5** specifies transaction formats, validation rules, and the progressive fee mechanism.
- **Section 6** describes the hybrid PoW + SCP consensus mechanism.
- **Section 7** presents the monetary policy including emission schedule, dynamic timing, and fee economics.
- **Section 8** outlines the peer-to-peer network protocol.
- **Section 9** provides security analysis including threat models, privacy guarantees, and attack resistance.
- **Section 10** analyzes economic incentives and long-term sustainability.
- **Section 12** describes the reference implementation and performance characteristics.
- **Section 14** concludes with a summary and discussion of future work.

2 Related Work

2.1 Privacy Cryptocurrencies

2.1.1 CryptoNote and Monero

The CryptoNote protocol [65] introduced two foundational privacy techniques: *stealth addresses* for recipient privacy and *ring signatures* [58] for sender privacy. Monero [41], the most widely adopted CryptoNote implementation, has evolved significantly since its 2014 launch.

Monero's current protocol uses:

- **CLSAG ring signatures** [25]: Concise Linkable Spontaneous Anonymous Group signatures, providing approximately 45% size reduction over the earlier MLSAG scheme [48, 49].
- **Pedersen commitments** [52]: Amount hiding with additive homomorphism for value conservation verification.
- **Bulletproofs** [8]: Logarithmic-size range proofs ensuring committed amounts are non-negative.
- **ECDH stealth addresses**: One-time keys derived via elliptic curve Diffie-Hellman.

Monero achieves strong privacy but faces several limitations that Bothoaddresses:

1. **No quantum resistance**: All cryptographic primitives are vulnerable to quantum attack.
2. **Probabilistic finality**: Nakamoto consensus requires multiple confirmations; reorgs remain possible.
3. **No anti-hoarding mechanism**: No economic incentive against wealth concentration.

2.1.2 MobileCoin

MobileCoin [40] is the closest existing system to Botho's design: it pairs CryptoNote-derived privacy with federated Byzantine consensus, the same two ingredients Bothocombines. Its transaction layer is a fork of Monero's cryptography—stealth addresses, ring signatures (RingCT/CLSAG), Pedersen commitments, and Bulletproofs—so privacy is mandatory rather than opt-in. Unlike Monero, however, MobileCoin replaces proof-of-work Nakamoto consensus with the Stellar Consensus Protocol [38], achieving fast, deterministic finality—the same consensus family Bothoadopts (discussed in the consensus review below).

MobileCoin's distinguishing choice is its reliance on trusted hardware:

- **Intel SGX secure enclaves**: Consensus and ledger-serving nodes process transaction contents inside SGX enclaves, so the input rings and other correlating data are never exposed in cleartext to node operators. Confidentiality is enforced at runtime by the hardware, with remote attestation used to verify that peers run the expected enclave code—an approach it shares with Secret Network's TEE model (discussed below).
- **Fog service and oblivious RAM**: A dedicated service lets light (mobile) clients discover their outputs without scanning the whole chain, using oblivious-RAM techniques to avoid leaking which records a client reads.

MobileCoin differs from Bothoin three respects that motivate our design:

- **No hardware-trust dependency**: Bothoenforces privacy at the protocol level and does not require SGX enclaves or remote attestation. This removes MobileCoin's trust in Intel's attestation service and its exposure to the recurring stream of SGX side-channel vulnerabilities, at the cost of MobileCoin's enclave-enforced in-memory confidentiality.
- **No quantum resistance**: Like Monero, MobileCoin's primitives are entirely classical; Bothoadds ML-KEM-768 stealth-address encapsulation for quantum-durable recipient privacy.
- **No anti-hoarding mechanism**: MobileCoin has a fixed supply and no demurrage or progressive-fee structure to discourage wealth concentration.

2.1.3 Zcash

Zcash [30] pioneered the use of zero-knowledge proofs (zk-SNARKs) [4] for cryptocurrency privacy. Unlike ring signatures, which hide the sender among a set of decoys, zk-SNARKs prove transaction validity without revealing any transaction details.

Zcash's privacy model differs fundamentally from Botho:

- **Opt-in privacy**: Most Zcash transactions are transparent; shielded transactions require explicit user action.
- **Trusted setup history**: The original Sprout ceremony required trust in participant integrity. Later upgrades (Sapling, Orchard) reduced but did not eliminate setup requirements.
- **Computational cost**: Proof generation requires significant computation, limiting usability on constrained devices.

Bothochooses mandatory privacy with ring signatures—simpler, faster, and requiring no trusted setup—accepting the tradeoff of probabilistic rather than cryptographic anonymity.

2.1.4 Zcash Orchard

Zcash's Orchard protocol [31] represents the latest evolution of zk-SNARK-based privacy, deployed in 2022. Key innovations include:

- **Halo 2 proof system**: Eliminates trusted setup via recursive proof composition. Proofs verify in constant time regardless of circuit complexity.
- **Unified addresses**: Single address format supporting transparent, Sapling, and Orchard funds, simplifying user experience.
- **Pallas/Vesta curves**: Purpose-built curves enabling efficient recursive proofs without pairing-based cryptography.
- **Action-based model**: Transactions consist of "actions" that can spend and create notes simultaneously, improving privacy analysis.

Despite these advances, Orchard maintains characteristics Bothoavoids:

- **Optional shielding**: Transparent transactions remain possible and common, fragmenting the anonymity set.
- **Heavy computation**: Proof generation requires 2–5 seconds on modern hardware, challenging for mobile devices.
- **Classical cryptography**: Pallas/Vesta curves remain vulnerable to Shor's algorithm.

2.1.5 Firo (Lelantus Spark)

Firo (formerly Zcoin) introduced Lelantus Spark [33] in 2023, combining Spark addresses with Lelantus-style one-out-of-many proofs:

- **Spark addresses**: Diversified addresses with incoming view key separation, similar to Monero's subaddresses but with additional flexibility.
- **One-out-of-many proofs**: Alternative to ring signatures providing similar sender anonymity with different tradeoffs.
- **No trusted setup**: Like ring signatures, Lelantus requires no parameter ceremony.

Lelantus Spark offers comparable privacy to ring signatures with different performance characteristics. Botho's choice of CLSAG reflects:

- Extensive security analysis and real-world deployment (Monero)
- Smaller proof sizes for typical transactions
- Better understood cryptographic assumptions

2.1.6 Secret Network

Secret Network [60] takes a fundamentally different approach: privacy through trusted execution environments (TEEs):

- **Intel SGX enclaves**: Computation occurs in hardware-isolated regions; even node operators cannot observe transaction details.

- **Secret contracts**: Programmable privacy with Turing-complete smart contracts—a capability pure cryptographic approaches cannot efficiently achieve.
- **Fast execution**: No proof generation overhead; privacy adds minimal latency.

The TEE approach involves different security assumptions:

- **Hardware trust**: Security depends on Intel's implementation and absence of hardware vulnerabilities.
- **Side-channel attacks**: SGX has faced multiple side-channel attacks (Spectre, Meltdown variants, etc.) [23].
- **Centralized manufacturing**: Hardware availability depends on Intel's production and cooperation.

Botho's purely cryptographic approach provides:

- No hardware trust assumptions
- Mathematically verifiable security properties
- Resilience against physical side-channel attacks

2.1.7 Aztec

Aztec [2] brings privacy to Ethereum through a zk-rollup Layer 2:

- **Private L2**: Transactions are private within Aztec; only validity proofs are posted to Ethereum.
- **Composability**: Private smart contracts can interact with Ethereum DeFi through controlled interfaces.
- **PLONK proofs**: Uses efficient universal zkSNARKs with updateable trusted setup.

Aztec's L2 approach offers interesting tradeoffs:

- **Inherits L1 security**: Settlement finality comes from Ethereum's consensus.
- **Limited decentralization**: Sequencer sets remain somewhat centralized during early development.
- **Bridging friction**: Moving funds between L1 and L2 introduces latency and potential vulnerabilities.

Bothooperates as an independent L1, avoiding bridge security concerns while sacrificing Ethereum composability.

2.1.8 MimbleWimble

MimbleWimble [55] achieves privacy through transaction cut-through and kernel aggregation. Implementations include Grin [26] and Beam [3].

Key limitations addressed by Botho:

- **Interactive transactions**: MimbleWimble requires sender and recipient to exchange data, complicating user experience.
- **Linkability attacks**: Research has demonstrated significant transaction graph leakage [20].

2.2 Post-Quantum Approaches

2.2.1 Quantum Threat Model

Shor's algorithm [61] enables quantum computers to solve the discrete logarithm and integer factorization problems in polynomial time, breaking ECDSA, ECDH, and related schemes. Grover's algorithm [27] provides quadratic speedup for symmetric key search, effectively halving security levels.

The "harvest now, decrypt later" threat is particularly relevant for blockchain systems: adversaries may record encrypted/signed data today for decryption when quantum computers become available. Since blockchain data is permanent and public, this threat applies to all historical transactions.

2.2.2 Existing Post-Quantum Cryptocurrencies

QRL (Quantum Resistant Ledger) [57] uses XMSS hash-based signatures. While quantum-resistant, XMSS is stateful—requiring careful key management to avoid catastrophic signature reuse.

IOTA has explored Winternitz one-time signatures, facing similar statefulness challenges.

Research proposals for post-quantum ring signatures [16, 63] exist but impose significant size overhead (∼50× classical signatures).

2.2.3 Post-Quantum Ring Signature Research

The search for practical post-quantum ring signatures remains an active research area. Current approaches include:

Lattice-based constructions [15]:

- Based on Ring-LWE or Module-LWE problems
- Signature sizes: 15–50 KB for ring size 16
- Verification: ∼10–50 ms
- Primary challenge: Size scales poorly with ring size

Hash-based ring signatures:

- Rely only on hash function security (conservative assumption)
- Even larger signatures than lattice approaches
- Better understood security reductions

Isogeny-based constructions:

- Based on supersingular isogeny problems
- More compact than lattice approaches
- Slower computation; SIDH attacks [10] have reduced confidence in isogeny hardness assumptions

Comparison to Botho's CLSAG:

Scheme	Ring 20	Ring 40	Quantum Safe
CLSAG	704 B	1,344 B	No
Lattice Ring (typical)	35 KB	70 KB	Yes
Hash-based Ring	100+ KB	200+ KB	Yes

Timeline assessment: Practical post-quantum ring signatures achieving sub-10 KB sizes for ring size 20 remain 5–10 years away based on current research trajectories. Botho's hybrid architecture is designed to accommodate migration when such constructions mature, while providing immediate quantum resistance for permanent on-chain data (recipient identities).

Migration path: Botho's modular cryptographic design allows future protocol upgrades to replace CLSAG with post-quantum alternatives. The signature abstraction layer (Section 12) isolates ring signature specifics from transaction processing, enabling migration via soft fork when practical PQ ring signatures emerge.

2.2.4 Hybrid Approaches

NIST's post-quantum cryptography standardization [47] produced ML-KEM (formerly Kyber) [6, 45] for key encapsulation and ML-DSA (formerly Dilithium) [14, 46] for signatures. These lattice-based schemes provide strong post-quantum security with reasonable performance.

Botho's hybrid approach applies these standards strategically:

- **ML-KEM-768** for stealth addresses: Recipient identity is permanent on-chain data requiring permanent protection.
- **CLSAG** for ring signatures: Sender privacy is ephemeral; classical cryptography provides efficiency.

No post-quantum signature is used: minting attribution is bound by the proof-of-work preimage over the minter's public keys, a hash-based commitment that is itself quantum-resistant (Section 4.5).

This selective application achieves meaningful quantum resistance while maintaining practical transaction sizes.

2.3 Consensus Mechanisms

2.3.1 Nakamoto Consensus

Bitcoin's Nakamoto consensus [43] achieves agreement through proof-of-work and longest-chain selection. Its key properties:

- **Permissionless**: Anyone can participate by providing hashpower.
- **Probabilistic finality**: Transactions become exponentially harder to reverse with additional confirmations, but reorgs remain theoretically possible.
- **Slow**: Bitcoin's 10-minute blocks and 6-confirmation standard yield ∼60-minute finality.

2.3.2 Classical BFT

Byzantine Fault Tolerant protocols like PBFT [9] achieve deterministic finality with $3f+1$ nodes tolerating f Byzantine failures [36]. However, classical BFT requires:

- Known, fixed participant set
- $O(n^2)$ message complexity
- Synchrony assumptions for liveness

These requirements conflict with permissionless cryptocurrency design.

2.3.3 Stellar Consensus Protocol

SCP [38] introduces *federated Byzantine agreement*, achieving BFT-like properties with open membership:

- **Quorum slices**: Each node declares which nodes it trusts.
- **Quorum intersection**: Safety requires overlapping quorums.
- **Open membership**: New nodes can join by declaring trust relationships.
- **Fast finality**: Consensus typically completes in seconds.

Bothocombines PoW block proposal (for permissionless participation and fair distribution) with SCP finalization (for fast deterministic finality).

2.4 Economic Mechanisms

2.4.1 Fixed Supply

Bitcoin's 21 million coin cap creates absolute scarcity but raises long-term security concerns. As block rewards diminish, network security depends increasingly on transaction fees. If fee revenue proves insufficient, security may degrade.

2.4.2 Tail Emission

Monero's perpetual tail emission (∼0.6 XMR per block indefinitely [41]) ensures permanent miner incentive. This trades absolute scarcity for security sustainability—a tradeoff Bothoembraces.

2.4.3 Demurrage

Demurrage currencies [24] (Freigeld, Chiemgauer) impose holding costs through time-based value decay. Classical demurrage taxes *all* holdings uniformly, which harms ordinary users and complicates the user experience. Bothoinstead applies demurrage *selectively*: the stock-level fee-floor term scales with a coin's cluster factor (Section 5.4), so idle high-concentration wealth carries a holding cost while ordinary factor-1 balances are exempt.

2.4.4 Novel: Progressive Fees

Bothointroduces *cluster-based progressive fees*—a novel mechanism that:

- Increases transaction costs for concentrated wealth
- Tracks coin *ancestry* rather than current ownership
- Resists Sybil attacks (splitting coins doesn't reduce fees)
- Preserves privacy (no identity required)

Combined with fee burning and the cluster-scaled demurrage term above, this is designed to create economic pressure toward circulation while sparing ordinary holders the uniform holding cost of classical demurrage. Whether the combination measurably reduces inequality is not demonstrated at production parameters (Section 10.6.2).

3 Preliminaries

3.1 Notation

We establish the following notation used throughout this paper:

Symbol	Description
λ	Security parameter
$\mathbb{G}$	Elliptic curve group (Ristretto255)
G	Generator of $\mathbb{G}$
H	Secondary generator for Pedersen commitments
q	Order of $\mathbb{G}$ (prime)
$\mathbb{Z}_q$	Integers modulo q
R_q	Polynomial ring $\mathbb{Z}_q[X]/(X^n+1)$ for lattice operations
a, b	View and spend private keys (scalars)
A, B	View and spend public keys (points)
(C, D)	Subaddress public key pair
P	One-time public key
x	One-time private key
I	Key image
$H_s(\cdot)$	Hash function to scalar
$H_p(\cdot)$	Hash function to curve point
$\mathcal{H}(\cdot)$	General cryptographic hash
$\{P_i\}_{i=0}^{n-1}$	Ring of public keys
π	Secret index of real key in ring
σ	Signature
C_v	Pedersen commitment to value v
$\xleftarrow{\$}$	Uniform random sampling
$\|$	Concatenation
$\mathsf{negl}(\lambda)$	Negligible function in λ

3.2 Cryptographic Assumptions

The security of Bothorelies on the following computational assumptions:

Definition 3.1 (Discrete Logarithm Problem (DLP))**.** Given $G \in \mathbb{G}$ and $Y = xG$ for random $x \xleftarrow{\$} \mathbb{Z}_q$, compute x.

Definition 3.2 (Decisional Diffie-Hellman (DDH))**.** Distinguish (G, aG, bG, abG) from (G, aG, bG, cG) for random $a, b, c \xleftarrow{\$} \mathbb{Z}_q$.

Definition 3.3 (Module Learning With Errors (MLWE))**.** Given $(\mathbf{A}, \mathbf{As}+\mathbf{e})$ where $\mathbf{A} \xleftarrow{\$} R_q^{k\times l}$, $\mathbf{s} \xleftarrow{\$} R_q^l$, and $\mathbf{e}$ is a small error vector, distinguish from $(\mathbf{A}, \mathbf{u})$ where $\mathbf{u} \xleftarrow{\$} R_q^k$.

Definition 3.4 (Module Short Integer Solution (MSIS))**.** Given $\mathbf{A} \xleftarrow{\$} R_q^{k\times l}$, find $\mathbf{x} \in R_q^l$ with $\mathbf{Ax} = \mathbf{0}$ and $\|\mathbf{x}\| \le \beta$ for bound β.

3.3 Building Blocks

3.3.1 Elliptic Curve Group

Bothouses the Ristretto255 group [13], a prime-order group constructed from Curve25519 [5]. Ristretto eliminates cofactor-related complexities, providing a clean abstraction with:

- Prime order $q = 2^{252} + 27742317777372353535851937790883648493$
- Efficient constant-time operations
- Resistance to small-subgroup attacks

3.3.2 Pedersen Commitments

A Pedersen commitment [52] to value v with blinding factor r is:

$$C = vH + rG \tag{1}$$

where G and H are independent generators (the discrete log of H with respect to G is unknown).

Theorem 3.5 (Pedersen Commitment Properties)**.** *Pedersen commitments satisfy:*

1. ***Hiding*** *(information-theoretic): C reveals nothing about v without r.*
2. ***Binding*** *(computational): Finding $(v', r') \neq (v, r)$ with the same commitment requires solving DLP.*
3. ***Homomorphic:*** *$C_1 + C_2 = (v_1 + v_2)H + (r_1 + r_2)G$.*

The homomorphic property enables verification that transaction inputs equal outputs plus fee, without revealing individual amounts.

3.3.3 Bulletproofs

Bulletproofs [8] provide zero-knowledge range proofs with logarithmic size. For a commitment $C = vH + rG$, a Bulletproof proves:

$$v \in [0, 2^{64}) \tag{2}$$

This prevents negative amounts (which would allow coin creation) while keeping amounts confidential.

Aggregation: Multiple range proofs can be aggregated, with size growing logarithmically in the number of proofs. A transaction with m outputs has proof size $O(\log m)$.

3.3.4 ML-KEM-768

ML-KEM (Module-Lattice-based Key-Encapsulation Mechanism) [6, 45], formerly known as Kyber, is a NIST-standardized post-quantum key encapsulation mechanism.

- $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{ML\text{-}KEM.KeyGen}()$: Generate keypair
- $(c, K) \leftarrow \mathsf{ML\text{-}KEM.Encap}(\mathsf{pk})$: Encapsulate to shared secret
- $K \leftarrow \mathsf{ML\text{-}KEM.Decap}(\mathsf{sk}, c)$: Decapsulate to recover secret

ML-KEM-768 provides approximately 192-bit security against quantum attacks, with:

- Public key size: 1,184 bytes
- Ciphertext size: 1,088 bytes
- Shared secret size: 32 bytes

The protocol uses no post-quantum signature scheme: spend authorization is classical (Section 4.3) and minting attribution is bound by the proof-of-work preimage itself (Section 4.5), which is hash-based and therefore quantum-resistant without a signature. Should a post-quantum authorization path be introduced in a future protocol version, ML-DSA (Module-Lattice-based Digital Signature Algorithm) [14, 46] at security level 3 (ML-DSA-65) is the designated signature family.

3.4 Security Definitions

Definition 3.6 (Unforgeability (EUF-CMA))**.** A signature scheme is *existentially unforgeable under chosen message attack* if no PPT adversary, given access to a signing oracle, can produce a valid signature on a message not queried to the oracle, except with negligible probability.

Definition 3.7 (Anonymity (Ring Signatures))**.** A ring signature scheme provides *anonymity* if no PPT adversary can identify the actual signer among ring members with probability significantly better than $1/n$, where n is the ring size.

Definition 3.8 (Linkability)**.** A ring signature scheme is *linkable* if there exists an efficient algorithm that, given two signatures from the same secret key, outputs 1; and given signatures from different keys, outputs 0 (with high probability).

Definition 3.9 (IND-CCA2 Security)**.** A key encapsulation mechanism is *IND-CCA2 secure* if no PPT adversary with access to a decapsulation oracle can distinguish real from random shared secrets, except with negligible advantage.

4 Cryptographic Protocol

4.1 Key Hierarchy

Bothoderives all cryptographic keys from a single BIP39 [51] mnemonic using SLIP-10 [59] hierarchical derivation.

4.1.1 Master Seed Generation

A 24-word mnemonic encodes 256 bits of entropy. The master seed is derived via:

$$\text{seed} = \text{PBKDF2-SHA512}(\text{mnemonic}, \text{“mnemonic”} \| \text{passphrase}, 2048) \tag{3}$$

This produces a 512-bit seed from which all keys are derived.

4.1.2 Account Key Derivation

Using SLIP-10 hardened derivation with path $m/44'/866'/\text{account}'$:

$$\text{view_key_material} = \text{SLIP10}(\text{seed}, m/44'/866'/0'/0') \tag{4}$$
$$\text{spend_key_material} = \text{SLIP10}(\text{seed}, m/44'/866'/0'/1') \tag{5}$$

These materials are then processed through HKDF-SHA512 [34] to produce the final Ristretto255 scalars:

$$a = \text{HKDF}(\text{view_key_material}, \text{“botho-ristretto255-view”}) \mod q \tag{6}$$
$$b = \text{HKDF}(\text{spend_key_material}, \text{“botho-ristretto255-spend”}) \mod q \tag{7}$$

The corresponding public keys are:

$$A = aG, \quad B = bG \tag{8}$$

The public address is the tuple (A, B).

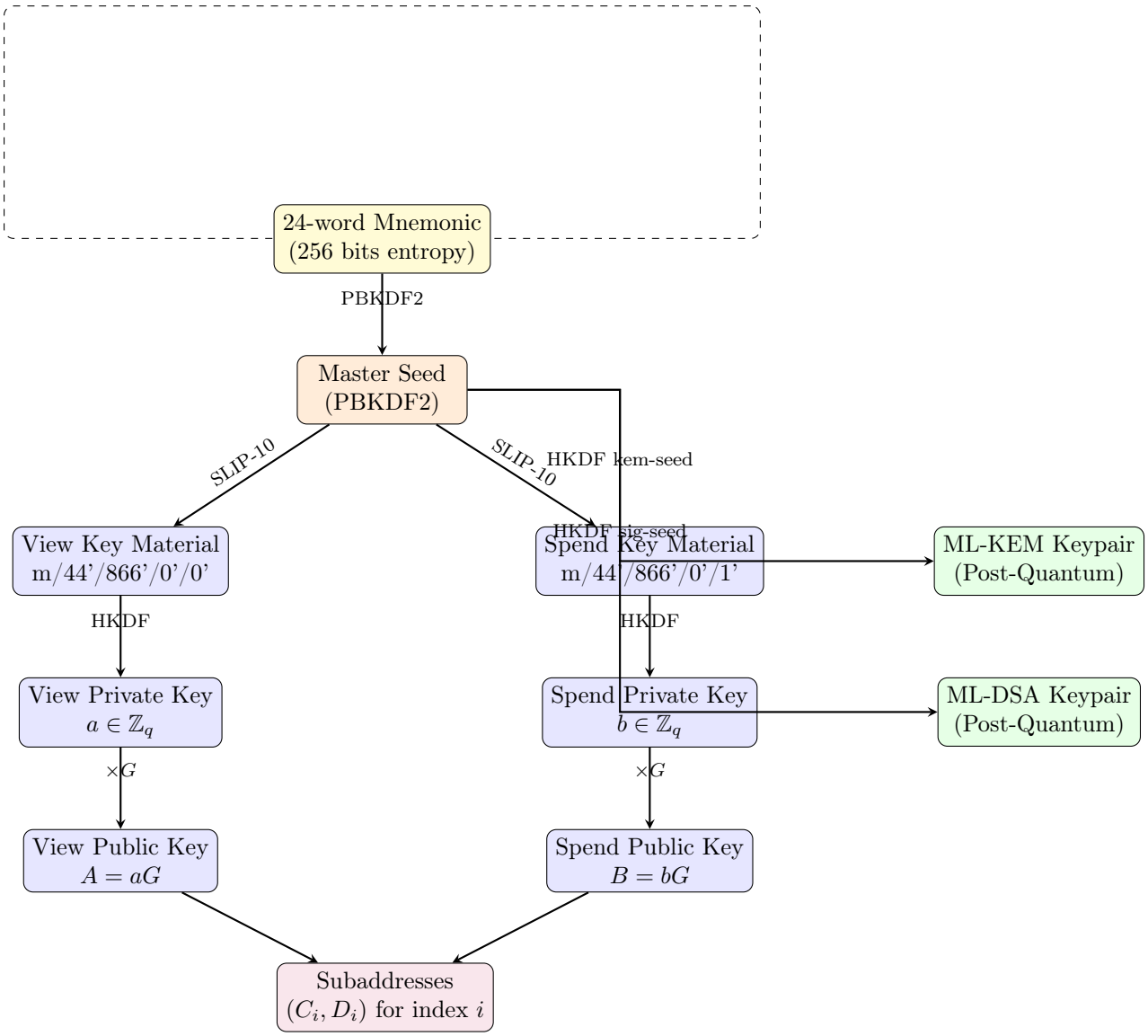


Figure 1: Key derivation hierarchy. A single BIP39 mnemonic derives all keys via SLIP-10 paths and HKDF domain separation. Classical Ristretto255 keys support ring signatures, while post-quantum ML-KEM/ML-DSA keys protect recipient privacy and minting operations.

4.1.3 Subaddress Derivation

Subaddresses allow generation of unlimited unlinkable addresses from a single master key pair. Bothouses the *multiplicative* subaddress construction (as in MobileCoin), which differs from Monero's additive scheme: the view subaddress key is the product $a \cdot D_i$ rather than a shifted point $A + \delta_i G$. For subaddress index i, a domain-separated scalar is derived with Blake2b-512:

$$\delta_i = H_s(\text{“bth_subaddress”} \| a \| i), \quad H_s(\cdot) = \text{Blake2b-512}(\cdot) \bmod q \tag{9}$$

The subaddress private key pair (c_i, d_i) — view and spend respectively — is then:

$$d_i = \delta_i + b \mod q \tag{10}$$

$$c_i = a \cdot d_i = a\,(\delta_i + b) \mod q \tag{11}$$

with corresponding public keys:

$$D_i = d_i G = \delta_i G + B, \qquad C_i = c_i G = a \cdot D_i \tag{12}$$

Here D_i is the subaddress spend public key and C_i is the subaddress view public key; the published subaddress is the pair (C_i, D_i). Note that the view key is obtained multiplicatively as $a \cdot D_i$, not by the additive offset $A + \delta_i G$: this is the defining difference from the Monero construction and is what the reference implementation computes.

Theorem 4.1 (Subaddress Unlinkability)**.** *Without knowledge of the view key a, subaddresses (C_i, D_i) and (C_j, D_j) for $i \neq j$ are computationally indistinguishable from independent random points.*

4.2 Post-Quantum Stealth Addresses

Traditional CryptoNote stealth addresses use ECDH for key exchange [11, 65], building on the dual-key stealth-address construction (DKSAP). Bothokeeps that classical ECDH step and *additionally* binds an ML-KEM-768 encapsulation into the same one-time key—a *hybrid* construction. Each output's one-time key is derived from BOTH the classical Diffie–Hellman shared secret and the ML-KEM shared secret, so an attacker must break *both* elliptic-curve ECDH *and* lattice ML-KEM to recover or link an output. This achieves post-quantum recipient privacy without discarding the classical security the network already relies on.

Intuition. Recipient privacy in a classical CryptoNote coin rests entirely on the hardness of the ECDH problem: the link between an output and its owner is an elliptic-curve Diffie–Hellman shared secret. That secret is not disclosed on chain, but everything needed to *recompute* it under a quantum attacker is—the sender's ephemeral public key sits in the transaction permanently. An adversary who records the chain today and later runs Shor's algorithm recovers the shared secret and deanonymizes every historical recipient. This is the *harvest-now, decrypt-later* exposure, and because recipient identity is permanent on-chain data (Table 1) it cannot be mitigated by waiting or rotating keys after the fact.

Folding an ML-KEM-768 encapsulation into the one-time key—alongside, not in place of, the classical ECDH secret—closes this exposure at its source. The sender encapsulates a fresh shared secret against the recipient's KEM public key and mixes it with the classical Diffie–Hellman secret when deriving the output's one-time key; recovering that key requires breaking a lattice problem believed hard even for quantum adversaries, so the recipient-unlinkability reduction of the Security Analysis below survives quantumly. Because the derivation is hybrid, the output also remains at least as private as a classical CryptoNote coin against any adversary who has *not* broken ECDH: a flaw in the lattice assumption alone cannot deanonymize the recipient. Crucially, the KEM hardens only the *key-exchange* step—the part whose secrecy must last forever—while sender anonymity continues to rely on the compact classical ring signature of Section 4.3. Making the entire spend authorization post-quantum would instead demand lattice ring signatures, whose $\sim$50$\times$ size overhead we quantify in Section 4.6 under "Why not full post-quantum?"; the hybrid split buys quantum-durable recipient privacy for the cost of a single 1,088-byte ciphertext per output rather than that blowup.

4.2.1 Protocol Description

The recipient's ML-KEM keypair is *not* derived from the Ristretto address point. Instead it is generated independently from the same BIP39 seed as the rest of the account-key hierarchy: the 512-bit seed is fed through HKDF-SHA256 with domain separator `"botho-pq-v1"` and info label `"kem-seed"` to produce a 32-byte ML-KEM seed, from which the ML-KEM-768 keypair $(\mathsf{pk}_{\text{kem}}, \mathsf{sk}_{\text{kem}})$ is deterministically generated (a sibling `"sig-seed"` label likewise seeds the ML-DSA-65 signing keypair). Consequently the recipient's address publishes pk_{kem} directly alongside

the Ristretto spend key B; there is no hash-of-point step, which would in any case not yield a valid ML-KEM key.

The one-time key binds *two* shared secrets: the classical ECDH secret $dh = rC = aR$ (as in classical CryptoNote, where r is the sender's ephemeral scalar, $R = rG$ the published transaction public key, C the recipient's view key, and a its secret) *and* the ML-KEM shared secret K. The stealth scalar is the domain-separated hash of both:

$$s = H_s\big(dh \,\|\, K \,\|\, \text{output_index}\big).$$

Sender (creating output for recipient):

1. Obtain the recipient's view key C and ML-KEM public key pk_{kem} from their address
2. Sample ephemeral scalar r; publish transaction public key $R = rG$
3. Compute classical DH secret: $dh = rC$
4. Encapsulate: $(c, K) \leftarrow \mathsf{ML\text{-}KEM.Encap}(\mathsf{pk}_{\text{kem}})$
5. Compute hybrid scalar: $s = H_s(dh \| K \| \text{output_index})$
6. Compute one-time public key: $P = sG + B$
7. Include ciphertext c (1,088 bytes) in transaction output

Recipient (scanning for received outputs):

1. Regenerate the ML-KEM secret key sk_{kem} from the BIP39 seed (HKDF label `"kem-seed"`, as above)
2. For each output with transaction public key R and ciphertext c:
   (a) Recompute classical DH secret: $dh = aR$ $(= rC)$
   (b) Decapsulate: $K \leftarrow \mathsf{ML\text{-}KEM.Decap}(\mathsf{sk}_{\text{kem}}, c)$
   (c) Compute hybrid scalar: $s' = H_s(dh \| K \| \text{output_index})$
   (d) Compute expected key: $P' = s'G + B$
   (e) If $P' = P$ (output's public key), this output belongs to us
   (f) Compute private key: $x = s' + b \mod q$

Verification: The one-time private key x satisfies $xG = (s + b)G = sG + B = P$, and because $aR = a(rG) = r(aG) = rC$ the sender and recipient derive the identical hybrid scalar $s = s'$.

4.2.2 Security Analysis

Theorem 4.2 (Recipient Unlinkability)**.** *If* either *the ML-KEM-768 encapsulation is IND-CCA2 secure* or *the elliptic-curve Diffie–Hellman problem is hard, an adversary cannot link outputs to recipients with probability better than negligible, given only the blockchain. In particular, a quantum adversary—who is assumed to break ECDH—is still defeated by the ML-KEM branch, while an adversary who breaks only the lattice assumption is still defeated by the classical branch.*

Proof. Because the stealth scalar $s = H_s(dh \| K \| \text{output_index})$ is a domain-separated hash of *both* shared secrets, modelling H_s as a random oracle makes s (hence the one-time key P) pseudorandom whenever *either* dh or K is unknown to the adversary. It therefore suffices to exhibit a reduction for each branch; we give the post-quantum (ML-KEM) branch, as the classical ECDH branch is the standard CryptoNote unlinkability argument [65] applied to the dh input.

We prove the ML-KEM branch by reduction to ML-KEM IND-CCA2 security. Suppose adversary $\mathcal{A}$ can distinguish outputs belonging to recipient R with non-negligible advantage ϵ, given dh (i.e. even after ECDH is broken). We construct adversary $\mathcal{A}'$ that breaks ML-KEM IND-CCA2 with advantage $\epsilon/2$.

Setup: $\mathcal{A}'$ receives ML-KEM public key pk^* from the IND-CCA2 challenger. $\mathcal{A}'$ sets this as the target recipient's KEM public key.

Query phase: $\mathcal{A}$ may request outputs for various recipients. For non-target recipients, $\mathcal{A}'$ generates honestly. For the target recipient, $\mathcal{A}'$ uses the decapsulation oracle.

Challenge: $\mathcal{A}$ submits two recipients R_0, R_1 (one being the target). $\mathcal{A}'$ queries the IND-CCA2 challenger with $(\mathsf{pk}^*, \mathsf{pk}_{\text{other}})$ and receives ciphertext c^* encapsulating either K_0 or K_1 (random bit b).

$\mathcal{A}'$ computes $s^* = H_s(dh \| K_b \| \text{index})$ and $P^* = s^*G + B_b$, returning output (c^*, P^*) to $\mathcal{A}$.

Analysis: If $\mathcal{A}$ correctly identifies the recipient with probability $1/2 + \epsilon$, then $\mathcal{A}'$ correctly guesses b with the same probability, contradicting IND-CCA2 security.

The reduction is tight: $\text{Adv}^{\text{IND-CCA2}}_{\mathcal{A}'} \geq \epsilon/2$. □

Corollary 4.3 (Quantum Security)**.** *The above reduction holds against quantum adversaries since ML-KEM-768 provides IND-CCA2 security in the quantum random oracle model (QROM).*

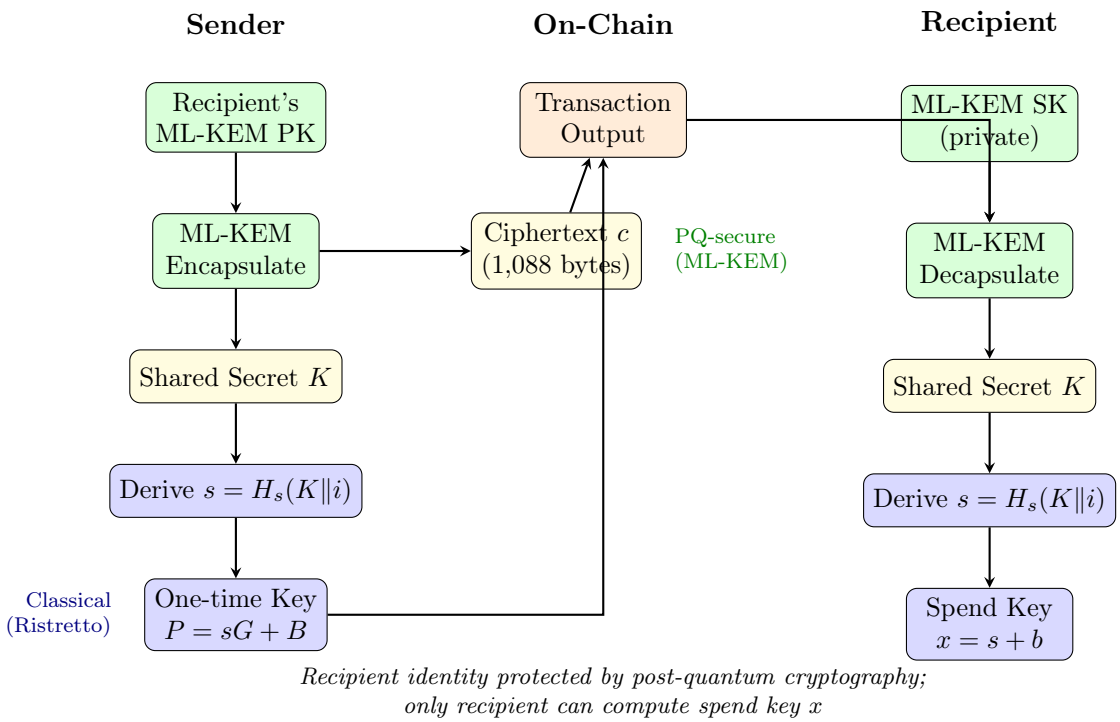


Figure 2: Post-quantum stealth address protocol. The sender encapsulates to the recipient's ML-KEM public key, deriving a shared secret used to create a unique one-time public key P. The on-chain ciphertext c is quantum-resistant—only the recipient with the ML-KEM secret key can decapsulate and derive the corresponding private key x to spend the output.

4.3 Ring Signatures (CLSAG)

CLSAG(Concise Linkable Spontaneous Anonymous Group) [25] provides efficient linkable ring signatures [22] for sender privacy.

4.3.1 Ring Construction

For each input being spent, the sender:

1. Selects $n-1$ decoy outputs from the blockchain (we use $n = 20$)
2. Forms a ring $\mathcal{R} = \{(P_0, C_0), \ldots, (P_{n-1}, C_{n-1})\}$ where each (P_i, C_i) is a one-time public key and commitment
3. The real input is at secret index π

4.3.2 Signature Generation

Given:

- Ring $\mathcal{R}$ with real index π
- Private key x_π and commitment blinding factor z_π
- Message m (transaction hash)

Key Image: $I = x_\pi \cdot H_p(P_\pi)$
The key image is deterministic given the private key and serves as a unique tag preventing double-spending.
Aggregation Coefficients:

$$\mu_P = \mathcal{H}(\text{“CLSAG_agg_0”} \| \mathcal{R} \| I \| D) \tag{13}$$
$$\mu_C = \mathcal{H}(\text{“CLSAG_agg_1”} \| \mathcal{R} \| I \| D) \tag{14}$$

where D is an auxiliary point for commitment verification.
Signature: The signature $\sigma = (c_0, s_0, \ldots, s_{n-1}, I, D)$ is computed via a Fiat-Shamir transform [21] of an interactive protocol, forming a “ring” of challenges that closes only if the prover knows one of the private keys.

4.3.3 Verification

Given signature σ and ring $\mathcal{R}$:

1. Recompute aggregation coefficients μ_P, μ_C
2. For $i = 0, \ldots, n-1$:

$$W_i = \mu_P P_i + \mu_C (C_i - C_{\text{out}}) \tag{15}$$
$$L_i = s_i G + c_i W_i \tag{16}$$
$$R_i = s_i H_p(P_i) + c_i (\mu_P I + \mu_C D) \tag{17}$$
$$c_{i+1} = \mathcal{H}(\text{“CLSAG_round”} \| \mathcal{R} \| L_i \| R_i \| m) \tag{18}$$

3. Accept if $c_n = c_0$ (ring closure)

4.3.4 Security Properties

Definition 4.4 (Unforgeability Game)**.** The unforgeability experiment $\text{Exp}_{\mathcal{A}}^{\text{UNF}}$ proceeds:

1. Challenger generates key pairs (x_i, P_i) for $i \in [n]$
2. $\mathcal{A}$ receives $\{P_i\}$ and access to signing oracle
3. $\mathcal{A}$ outputs $(m^*, \sigma^*, \mathcal{R}^*)$
4. $\mathcal{A}$ wins if $\mathsf{Verify}(\sigma^*, m^*, \mathcal{R}^*) = 1$ and m^* was never queried to signing oracle with ring $\mathcal{R}^*$

Definition 4.5 (Anonymity Game)**.** The anonymity experiment $\text{Exp}_{\mathcal{A}}^{\text{ANON}}$ proceeds:

1. Challenger generates key pairs, gives public keys to $\mathcal{A}$
2. $\mathcal{A}$ selects ring $\mathcal{R}$, message m, two indices i_0, i_1
3. Challenger flips bit b, signs with key x_{i_b}
4. $\mathcal{A}$ outputs guess b'
5. $\mathcal{A}$'s advantage: $|\Pr[b' = b] - 1/2|$

Theorem 4.6 (CLSAG Security)**.** *Under the DLP assumption in the random oracle model, CLSAG satisfies:*

1. ***Unforgeability****: For all PPT $\mathcal{A}$,* $\Pr[Exp_{\mathcal{A}}^{UNF} = 1] \leq \mathsf{negl}(\lambda)$.
2. ***Anonymity****: For all PPT $\mathcal{A}$,* $Adv_{\mathcal{A}}^{ANON} \leq \mathsf{negl}(\lambda)$.
3. ***Linkability****: For all PPT $\mathcal{A}$,* $\Pr[\textit{same key, different images}] \leq \mathsf{negl}(\lambda)$ *and* $\Pr[\textit{different keys, same image}]$ $\mathsf{negl}(\lambda)$.

Proof (Unforgeability). By reduction to DLP. Given DLP instance $(G, Y = xG)$, embed Y as one ring member's public key. A forking lemma argument shows that a successful forger can be rewound to extract the discrete log, contradicting DLP hardness. Full proof in [25]. □

Proof (Anonymity). The signature is a Fiat-Shamir transform of a Σ-protocol. In the random oracle model, the simulated transcript is indistinguishable from real, regardless of which key was used. The ring structure ensures all positions are computationally indistinguishable. □

Proof (Linkability). Key image $I = x \cdot H_p(P)$ is deterministic given (x, P). For the same key, the same image is always produced. For different keys $x \neq x'$, collision requires $x \cdot H_p(P) = x' \cdot H_p(P')$, which occurs with probability $1/q$ (negligible) when H_p is modeled as a random oracle. □

4.4 Confidential Transactions

4.4.1 Amount Commitment

Each output commits to its amount v with blinding factor r:

$$C = vH + rG \tag{19}$$

The blinding factor r is derived deterministically from the shared secret to enable recipient recovery.

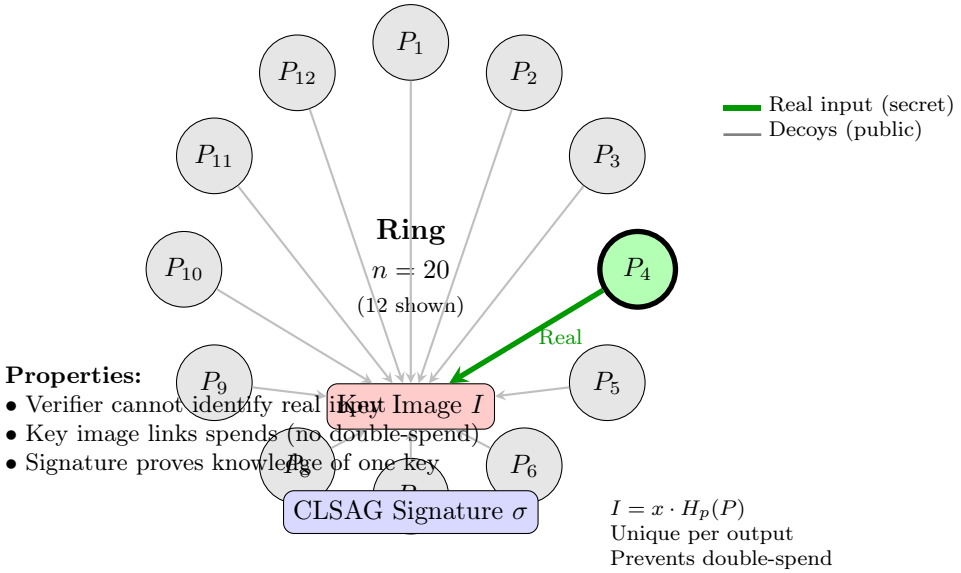


Figure 3: Ring signature structure. The real input (green) is hidden among 19 decoy outputs selected from the blockchain. The signature proves the signer knows one of the private keys without revealing which. The key image I is deterministically derived from the real input, enabling double-spend detection while preserving anonymity.

4.4.2 Value Conservation

For a transaction with inputs $\{C_{\text{in}}^{(i)}\}$ and outputs $\{C_{\text{out}}^{(j)}\}$, value conservation requires:

$$\sum_i C_{\text{in}}^{(i)} = \sum_j C_{\text{out}}^{(j)} + fH \tag{20}$$

where f is the transaction fee (public). This holds if and only if:

$$\sum_i v_{\text{in}}^{(i)} = \sum_j v_{\text{out}}^{(j)} + f \tag{21}$$

Validators check commitment arithmetic without learning individual values.

4.4.3 Range Proofs

Each output includes a Bulletproof demonstrating:

$$v_{\text{out}}^{(j)} \in [0, 2^{64}) \tag{22}$$

This prevents:

- Negative amounts (which would create coins from nothing)
- Overflow attacks (amounts wrapping around)

4.5 Minting Attribution

Block rewards (minting transactions) carry no signature. The minter's identity is public, and attribution is bound by the proof of work itself: the RandomX preimage commits to the minter's public address keys,

$$h_{\text{PoW}} = \text{RandomX}\big(\text{nonce} \| \text{prev_hash} \| \mathsf{pk}_{\text{view}} \| \mathsf{pk}_{\text{spend}}\big) \tag{23}$$

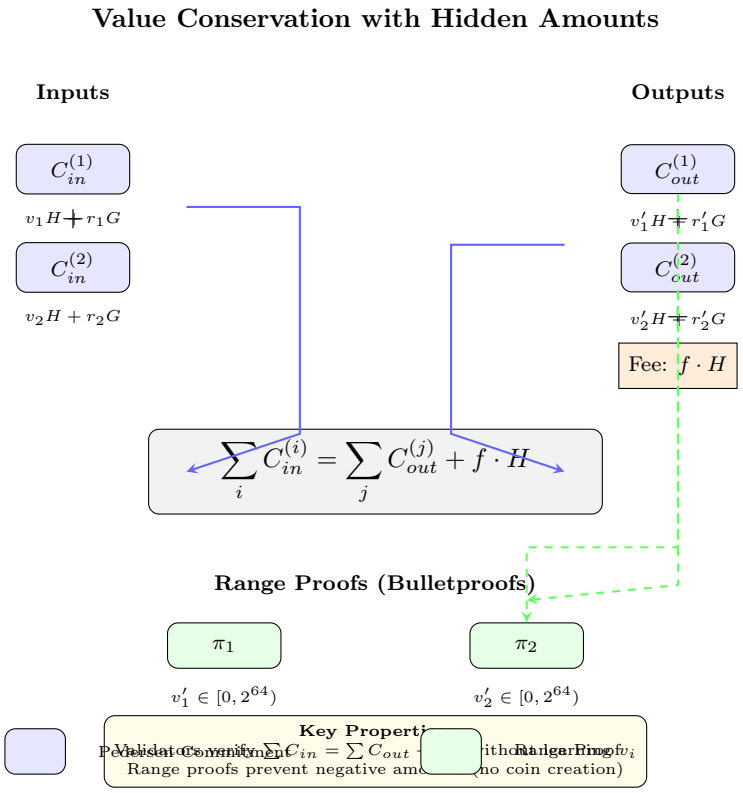


Figure 4: Confidential transaction structure. Input and output values are hidden in Pedersen commitments ($C = vH + rG$). Value conservation is verified via commitment arithmetic: the sum of input commitments must equal output commitments plus the public fee. Bulletproof range proofs ensure all output amounts are non-negative, preventing coin creation through overflow attacks.

and the minting transaction is valid only if h_{PoW} meets the difficulty target. Reattributing a block's reward would change the minting transaction's hash — and with it the cluster origin identifier and the stealth reward output derived from it (Section 5.4) — and would require redoing the proof of work against a chain whose blocks are already externalized by consensus (Section 6). Because the binding is hash-based, it is subject only to Grover-type quantum speedups and remains sound against a cryptographically relevant quantum computer: minting attribution needs no post-quantum signature, and adding one (ML-DSA-65's 3,309-byte signature plus 1,952-byte public key on every block) would cost roughly 33 GB of chain growth per year at the reference block time for no additional security in this model.

4.6 Hybrid Architecture Rationale

Table 1: Cryptographic choices by data lifetime

Data	Lifetime	Algorithm	Rationale
Recipient identity	Permanent	ML-KEM-768	On-chain forever; must be PQ
Sender identity	Ephemeral	CLSAG	Value degrades; efficiency wins
Amounts	Permanent	Pedersen	Information-theoretic hiding
Minting attribution	Permanent	PoW hash binding	Hash-based; quantum-resistant

Why not full post-quantum?
Post-quantum ring signatures (e.g., lattice-based constructions) impose approximately 50×

size overhead. A CLSAG signature is ∼700 bytes per input; a comparable lattice ring signature would be ∼35 KB. With multiple inputs, transactions would exceed 100 KB, making desktop nodes impractical.

Why is ephemeral sender privacy acceptable?

The value of sender deanonymization degrades over time. Learning who sent a transaction in 2025 from a 2045 perspective has minimal economic relevance—the goods have been delivered, contracts fulfilled, and economic context forgotten. In contrast, recipient identity remains valuable ("who owns this address?") indefinitely.

This asymmetry justifies asymmetric protection: permanent data gets permanent (post-quantum) protection; ephemeral data gets efficient (classical) protection.

5 Transaction Format

5.1 Transaction Types

Bothosupports two transaction types:

Table 2: Transaction type comparison

Type	Inputs	Outputs	Signature	Size	Use Case
Minting	0	1	PoW binding (none)	∼0.2 KB	Block rewards
Private	1–16	1–16	CLSAG	∼4 KB	All transfers

5.2 Private Transaction Structure

A private transaction transfers value while hiding sender, recipient, and amount.

5.2.1 Transaction Components

```
PrivateTransaction
    prefix: TransactionPrefix,
    ring_signatures: Vec<CLSAGSignature>,
    bulletproofs: AggregatedRangeProof,


TransactionPrefix
    version: u8,
    inputs: Vec<TxInput>,
    outputs: Vec<TxOutput>,
    fee: u64,
    extra: Vec<u8>,
```

5.2.2 Input Structure

Each input references a previously unspent output without revealing which:

```
TxInput
    ring: [TxOutRef; RING_SIZE],  // RING_SIZE = 20
    key_image: KeyImage,           // 32 bytes
    cluster_tag: ClusterTag,       // 32 bytes


TxOutRef
```

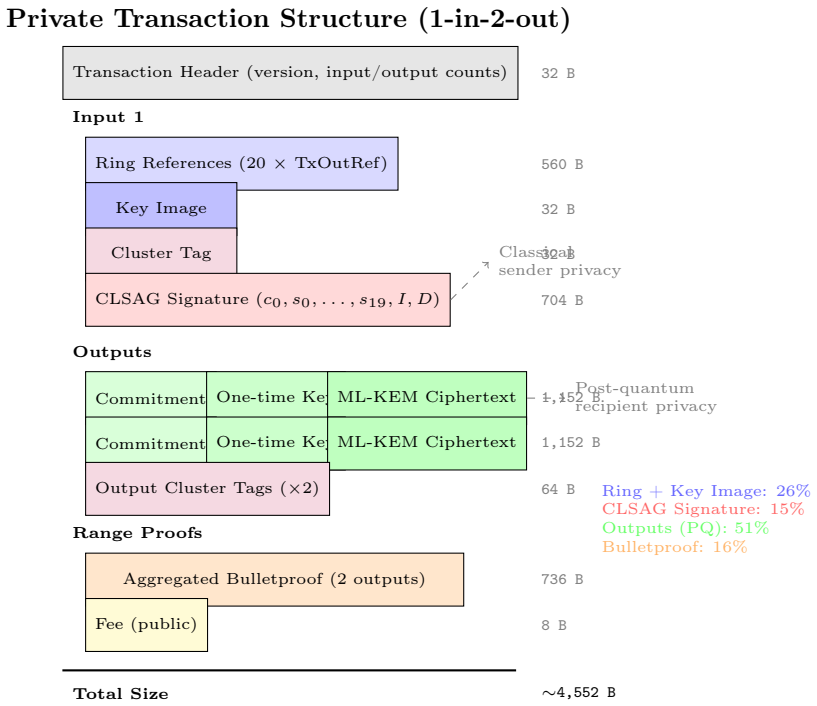


Figure 5: Anatomy of a private transaction (1 input, 2 outputs). The ML-KEM ciphertext dominates output size (1,088 bytes each) but provides post-quantum recipient privacy. Ring signatures use classical CLSAG for efficiency. Total size is approximately 4.5 KB, compared to ~50 KB with post-quantum ring signatures.

```
    block_height: u64,
    tx_index: u16,
    output_index: u8,
```

The `ring` contains references to 20 possible source outputs. The `key_image` uniquely identifies the spent output (for double-spend prevention) without revealing which ring member it corresponds to.

5.2.3 Output Structure

Each output contains the encrypted amount and one-time keys:

```
TxOutput
    commitment: CompressedPoint,      // Pedersen commitment (32 bytes)
    one_time_key: CompressedPoint,    // One-time public key (32 bytes)
    encrypted_amount: [u8; 32],       // AES-encrypted amount
    ml_kem_ciphertext: [u8; 1088],    // ML-KEM ciphertext
    cluster_tag: ClusterTag,          // Derived from input tags
```

5.3 Minting Transaction Structure

Minting transactions create new coins as block rewards:

```
MintingTx
    block_height: u64,
    reward: u64,                  // Public amount (picocredits)
```

```
minter_view_key: [u8; 32],   // Bound by the PoW preimage
minter_spend_key: [u8; 32],  // Bound by the PoW preimage
target_key: [u8; 32],        // Stealth one-time reward key
public_key: [u8; 32],        // Ephemeral key for minter scanning
prev_block_hash: [u8; 32],
difficulty: u64,
nonce: u64,                  // PoW solution
timestamp: u64,
```

Unlike private transactions:

- No inputs (new coins created)
- Public reward amount (for supply auditability)
- Known sender (the minter's address keys are in the clear and bound by the PoW preimage; no signature is carried — Section 4.5)
- The reward recipient is still hidden via a stealth output — even a minter who wins repeatedly produces unlinkable reward outputs

5.4 Cluster Tags and Progressive Fees

Cluster tags enable Sybil-resistant progressive fees by tracking coin *provenance* rather than current ownership. The key insight is that splitting coins does not change where they came from.

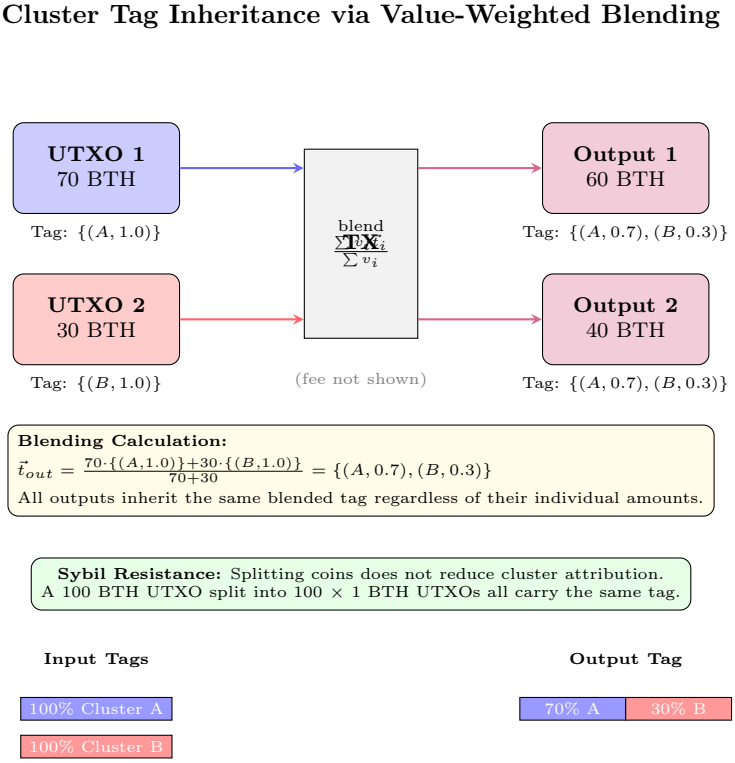


Figure 6: Cluster tag inheritance through value-weighted blending. When a transaction combines inputs from different clusters, the output tag vector is the value-weighted average of input tags. This preserves provenance information and ensures that splitting coins cannot reduce the cluster factor used for progressive fee calculation.

5.4.1 Tag Vector Representation

Each UTXO carries a *tag vector* representing the weighted distribution of its ancestry across all clusters:

$$\vec{t} = \{(c_1, w_1), (c_2, w_2), \ldots, (c_k, w_k)\} \quad \text{where} \sum_i w_i = 1 \tag{24}$$

Here c_i is a cluster identifier and w_i is the fraction of the UTXO's value attributable to that cluster.

5.4.2 Cluster Creation at Minting

Each minting transaction creates a new cluster. The minting transaction's hash — which commits to the minter's public keys, the block height, and the PoW solution — serves as the cluster identifier (truncated to 64 bits):

$$c_{\text{new}} = \text{Trunc}_{64}\big(\mathcal{H}(\text{minting tx})\big) \tag{25}$$

Minting outputs carry a tag vector with 100% attribution to the new cluster: $\vec{t}_{\text{mint}} = \{(c_{\text{new}}, 1.0)\}$.

5.4.3 Tag Inheritance and Blending

When spending multiple inputs, output tags are computed via value-weighted blending:

$$\vec{t}_{\text{out}} = \frac{\sum_i v_i \cdot \vec{t}_i}{\sum_i v_i} \tag{26}$$

where v_i is the value of input i and $\vec{t}_i$ is its tag vector.

Example: Combining a 70 BTH UTXO (100% cluster A) with a 30 BTH UTXO (100% cluster B) produces output tags: $\{(A, 0.7), (B, 0.3)\}$.

5.4.4 Tag Decay (Wallet Policy, as Built)

Tags are meant to fade as coins circulate, so that genuine commerce moves a coin toward the background factor. As built, decay is a *wallet* rule, not a consensus rule:

- **Stock wallets** (the node wallet and the CLI wallet) apply a flat 5% decay on every transfer when they compute output tags, via value-weighted blending of the inputs followed by

  $$\vec{t}'_{\text{out}} = 0.95 \cdot \vec{t}_{\text{out}} \tag{27}$$

  (`DEFAULT_CLUSTER_DECAY_RATE` = 50,000 ppm, `botho/src/wallet.rs`, `botho-wallet/src/transaction.rs`). There is no age gate: a weight halves after $\ln 0.5 / \ln 0.95 \approx 13.5$ transfers, however quickly they are made.

- **Consensus** applies no decay (`CONSENSUS_TAG_DECAY_RATE` = 0, `botho/src/ledger/store.rs`). Block validation only rejects *inflation*: for each cluster, the outputs may not claim more tag mass than the per-ring maximum of the inputs (`check_cluster_tag_inheritance`, `store.rs`). Smaller or empty output tags are accepted, so a modified wallet can drop a coin's tags in one transaction. Its only consensus cost is the class-downgrade charge of Section 7.4.3. This is the as-built behaviour, pending #1687.

Earlier drafts of this section specified an *age-gated* decay (decay only when the spent UTXO is at least 720 blocks old, bounding a wash trader to one decay event per 720 blocks). That rule exists as a library model (`cluster-tax/src/age_decay.rs`) but has no production caller, and the event rates quoted for it (24/day at 5 s blocks, 12/day at 10 s blocks) describe a rule the network does not run.

5.4.5 Cluster Wealth Accumulation

Although each minting event creates a fresh, per-block cluster id (Equation, Section 5.4), a cluster's *wealth* is not one block reward: it is the cumulative tagged value that lineage controls across the entire UTXO set. Formally, the wealth of cluster c is

$$W_c = \sum_{U \in \mathcal{U}} v(U)\, w_c(U), \tag{28}$$

the value-weighted sum of every UTXO's attribution to c. As coins minted in that lineage circulate and blend, W_c accumulates through tag inheritance and can grow to hundreds of thousands of BTH — far above a single 50 BTH reward. The ledger tracks W_c as a cumulative `u128` per cluster.

5.4.6 Cluster Factor Calculation

A UTXO's scalar cluster factor is obtained by (i) collapsing its tag vector to an *effective wealth*, the tag-weighted sum of the constituent clusters' wealths,

$$W_{\text{eff}}(U) = \sum_{c} w_c(U)\, W_c, \tag{29}$$

and (ii) mapping that wealth through a *log-domain* sigmoid that rises across orders of magnitude of wealth:

$$\phi(U) = 1 + 5\,\sigma\!\left(\frac{\log_2 W_{\text{eff}}(U) - \log_2 W_{\text{mid}}}{\omega}\right), \tag{30}$$

where σ is the logistic function, $W_{\text{mid}} = 100{,}000$ BTH is the midpoint (a cluster at the midpoint sits at exactly 3.5×), and $\omega = 2.0$ octaves per sigmoid unit sets the width. The factor is clamped to $[1\times, 6\times]$. Diffuse, low-wealth attribution dilutes W_{eff} toward zero, so genuinely small holders sit at 1×.

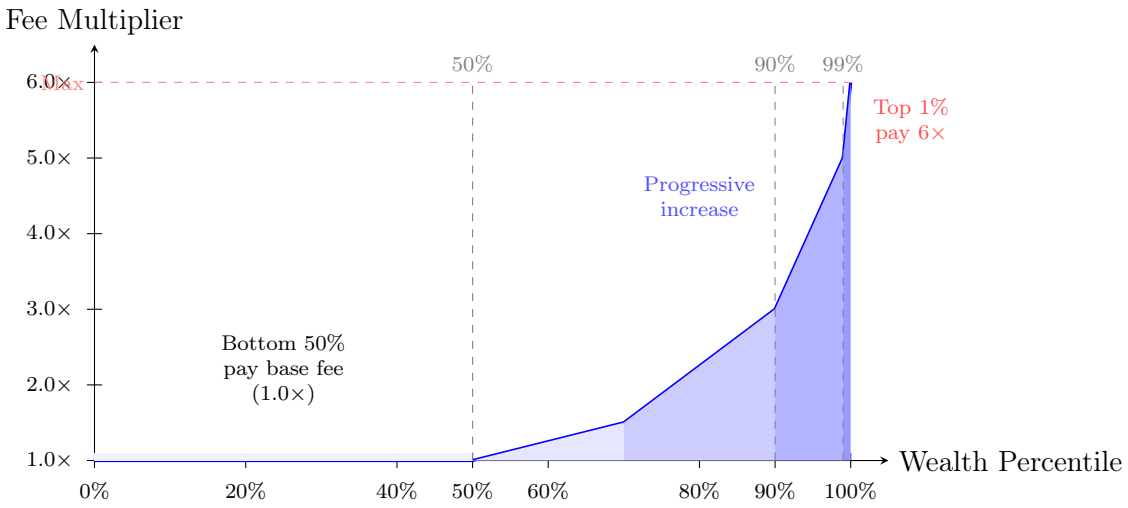
Table 3: Progressive fee multipliers (log-domain curve, $W_{\text{mid}} = 100$k BTH)

Effective Cluster Wealth	Fee Multiplier
≤1,000 BTH	≈1.0×
∼100,000 BTH (midpoint)	3.5×
≳1,000,000 BTH	≈5.2× – 6.0×

5.4.7 Ring Signature Tag Propagation

Ring signatures hide which input is real among n ring members, but every member's tag vector is public. As built, the consensus fee floor (`consensus_fee_floor`, `botho/src/ledger/store/validation.rs`) uses tags in three places:

1. **Size fee factor**: the cluster factor of the size-based fee is computed from the spender's *output* tags, weighted by output value against the committed per-cluster wealth (`effective_cluster_wealth_from_outputs`, `validation.rs`).



Fee calculation: fee = base_fee × size × f_c(percentile)
where f_c is the cluster factor function shown above.

Figure 7: Progressive fee multiplier based on cluster wealth percentile. Users in the bottom 50% of wealth distribution pay the base fee (1×). Fees increase progressively for wealthier clusters, reaching maximum 6× for the top 1%. This is designed as Sybil-resistant economic pressure against wealth concentration while remaining minimal for typical users; a measurable reduction in inequality is not demonstrated (Section 10.6.2).

2. **Ring floor for demurrage**: the demurrage factor is floored at the *value-weighted centroid* of the ring members' effective cluster wealth, mapped through the factor curve (`ring_centroid_floored_factor`, `validation.rs`; `ring_centroid_implied_factor`, `cluster-tax/src/demurrage.rs`). It is a mean, not the maximum factor among ring members. Because the spender chooses the decoys, high-value low-factor decoys can pull the floor down (as built, pending #1686).

3. **Inflation guard**: output tags may not claim more per-cluster mass than the per-ring maximum of the inputs (Section 5.4.4).

Output tags are written by the wallet from the *real* inputs and are published in plaintext; no zero-knowledge proof binds them to the real input. An observer can therefore match output tags against ring members' tags, which weakens ring privacy (as built, pending #1756). Committed tag vectors with ZK inheritance proofs are a design target, not implemented.

5.4.8 Sybil Resistance Analysis

The cluster tag system resists common attacks:

- **Splitting coins**: Does not reduce fees—child outputs inherit the parent's cluster attribution unchanged.
- **Creating multiple addresses**: All outputs from the same cluster share the same factor.
- **Wash trading**: Not rate-limited by consensus. Stock wallets decay tags 5% per transfer, and consensus accepts dropped tags, so the binding cost of shedding a high factor is the class-downgrade charge (Section 7.4.3), not time (as built, pending #1687; that charge is zero in epoch 0, pending #1685).

- **Mixing through exchanges**: Reduces cluster concentration over time, but requires actual economic activity (legitimate use case).

Theorem 5.1 (Cluster Tag Sybil Resistance)**.** *For any splitting strategy that divides n BTH into k UTXOs, the total fees paid are at least as high as paying from a single UTXO.*

Proof. We prove by structural induction on transaction graphs that no splitting strategy reduces total fees.

Definitions. Let:

- $\phi : \text{UTXO} \to [1, 6]$ denote the cluster factor function
- $v : \text{UTXO} \to \mathbb{R}^+$ denote the value function
- $\vec{t} : \text{UTXO} \to \Delta^{|\mathcal{C}|}$ denote the tag vector (probability simplex over clusters)
- b denote the base fee rate per byte
- s denote the transaction size in bytes

The fee for spending UTXO U is $\text{fee}(U) = b \cdot s \cdot \phi(U)$.

Base case: A single UTXO U with value $v(U) = n$ and cluster factor $\phi(U) = f$ incurs fee $F_{\text{base}} = b \cdot s \cdot f$ to spend completely.

Inductive case: Consider splitting U into k outputs $U_1, \ldots, U_k$ via a transaction T. We analyze the fee implications.

Step 1: Tag inheritance. By the tag blending rule (Equation 5.3), when U is the sole input:

$$\vec{t}(U_i) = \frac{v(U) \cdot \vec{t}(U)}{v(U)} = \vec{t}(U) \quad \forall i \in [k]$$

Each output inherits exactly the parent's tag vector.

Step 2: Factor preservation. Since ϕ depends only on the tag vector and global cluster wealth:

$$\phi(U_i) = \phi(\vec{t}(U_i)) = \phi(\vec{t}(U)) = \phi(U) = f \quad \forall i$$

Splitting does not change cluster factors.

Step 3: Fee accounting. The splitting transaction T incurs fee:

$$F_T = b \cdot s_T \cdot f$$

where s_T is the size of the splitting transaction.

Subsequently spending all k outputs incurs total fees:

$$F_{\text{spend}} = \sum_{i=1}^{k} b \cdot s_i \cdot \phi(U_i) = b \cdot f \cdot \sum_{i=1}^{k} s_i$$

Step 4: Total comparison. Total fees under splitting strategy:

$$F_{\text{split}} = F_T + F_{\text{spend}} = b \cdot f \cdot \left(s_T + \sum_{i=1}^{k} s_i \right)$$

For a single transaction spending U and producing the same final outputs:

$$F_{\text{direct}} = b \cdot f \cdot s_{\text{direct}}$$

Since $s_T + \sum_i s_i \geq s_{\text{direct}}$ (intermediate transactions add overhead), we have $F_{\text{split}} \geq F_{\text{direct}}$.

Induction on transaction depth. For multi-level splitting (creating outputs that are further split), apply the argument recursively. Each level adds transaction overhead without reducing cluster factors. By strong induction:

$$F_{\text{depth-}d} \geq F_{\text{depth-}(d-1)} \geq \cdots \geq F_{\text{direct}}$$

Mixing attack analysis. If an attacker combines high-factor UTXOs with low-factor UTXOs:

$$\vec{t}_{\text{mixed}} = \frac{\sum_i v_i \cdot \vec{t}_i}{\sum_i v_i}$$

The resulting factor is a weighted average, not a minimum. The attacker must acquire low-factor UTXOs through legitimate economic activity (purchasing from others), which itself incurs fees and does not create value—it merely transfers it from other participants.

Conclusion. No Sybil strategy (splitting coins, creating addresses, or self-transfers) reduces total fees below the direct transaction cost. Fee reduction requires genuine economic mixing with other clusters. □

Corollary 5.2 (Wash Trading Ineffectiveness)**.** *Self-transfers through n intermediate addresses incur strictly greater fees than direct transfer, with overhead $\Omega(n \cdot s_{tx} \cdot b \cdot f)$.*

Premises (as built). Step 1 assumes outputs carry the inherited tags. Consensus enforces only that they claim no *more* cluster mass than the inputs (Section 5.4.4), so the theorem and corollary hold for wallets that write inherited tags; a modified wallet may write empty tags and pay only the class-downgrade charge (pending #1687, #1686, #1685). The size-fee model $b \cdot s \cdot \phi$ is the consensus floor; at today's constants the flat relay minimum of 10^8 pico-BTH per transaction exceeds it for ordinary transactions, so the factor term rarely binds in practice (pending #1688, #1684).

5.5 Validation Rules

A private transaction is valid if and only if:

1. **Structure**: 1–16 inputs, 1–16 outputs, valid encoding
2. **Key Image Uniqueness**: All key images are:
   - Distinct within this transaction
   - Not present in the key image database (no double-spend)
3. **Ring Validity**: For each input ring:
   - All referenced outputs exist and are unspent
   - Ring members are sorted (canonical ordering)
   - Real output is among the ring members
4. **Signature Validity**: All CLSAG signatures verify
5. **Value Conservation**:
$$\sum_i C_{\text{in}}^{(i)} = \sum_j C_{\text{out}}^{(j)} + \text{fee} \cdot H \tag{31}$$
6. **Range Proofs**: Bulletproofs verify for all output commitments (amounts in $[0, 2^{64})$)
7. **Fee**: $\text{fee} \geq \min_\text{fee}(\text{size}, \text{cluster}_\text{factor}, n_{\text{out}}) + \text{demurrage}$, the congestion-free consensus floor (`verify_consensus_fee_floor`, `botho/src/ledger/store/validation.rs`)

8. **Size**: Total serialized size $\leq$ 100 KB

Relay policy versus consensus (as built). The 20-member minimum ring size, the 10^6 pico-BTH dust floor and the 10^8 pico-BTH minimum fee (`MIN_TX_FEE`) are checked by `is_valid_structure` (`transaction/clsag/src/lib.rs`), whose only production caller is mempool admission (`botho/src/mempool.rs`). Block validation does not call it, so a block producer can include its own transactions that a default node would not relay. These limits are relay policy today, pending #1684.

5.6 Transaction Size Analysis

Table 4: Private transaction size breakdown (1-in-2-out)

Component	Size (bytes)
Transaction header	32
Input (ring references, key image)	680
Output $\times$ 2 (commitment, key, ciphertext)	2,304
CLSAG signature	704
Bulletproof (2 outputs, aggregated)	736
Cluster tags	96
Total	$\sim$4,552

For comparison, a post-quantum ring signature would add approximately 35 KB per input, making transactions 10$\times$ larger.

5.7 Decoy Selection

Decoy selection is wallet policy; consensus checks only that ring members exist and are unspent. As built:

1. **Age distribution**: Every picker (node, CLI, browser, Snap and claim-link signers) draws one random-offset age window per ring, 10% of the real input's age wide ($\pm$5%, `AGE_SIMILARITY_SPREAD_BPS`), positioned so the real input's age is uniform inside it, and samples decoys uniformly within it (`draw_age_similarity_window`, `botho/src/decoy_selection.rs`; `botho-wallet/src/ring_builder.rs`; `web/packages/wasm-signer/src/decoys.ts`; #1692, #1755). Given the window the ring members are exchangeable, so the real input's age rank is uniform. The window also bounds the demurrage over-charge of the max-age ring clock: the oldest member is at most 1.10$\times$ the real age, inside the 1.15 tolerance, on every spend.

2. **Cluster similarity**: Mempool admission rejects a ring whose value-weighted tag centroid has cosine similarity below 0.7 with the output tags (`RING_TAG_SIMILARITY_THRESHOLD`, `botho/src/mempool.rs`); empty tags score 1.0. This is relay policy, not a consensus rule. No production picker selects decoys by tag, and output tags are published in plaintext (pending #1756).

3. **Randomization**: Subject to the above constraints, selection is randomized with a CSPRNG: one draw places the window, and the decoys are a uniform sample of the candidates inside it.

Measured identification rates for each production picker are in `cluster-tax/src/simulation/PRIVACY_ANALYSIS.md`, which the privacy simulation rebuild (#1700) regenerates.

5.8 Transaction Malleability

Bothotransactions resist malleability:

- **Signature covers full prefix**: Modifying any field invalidates signatures
- **Canonical encoding**: Only one valid serialization exists
- **Key image binding**: Key images are deterministic from private keys

The transaction hash (txid) is computed over the canonical serialization of the complete transaction.

6 Consensus Mechanism

Bothoemploys a hybrid consensus mechanism combining proof-of-work (PoW) block proposal with Stellar Consensus Protocol (SCP) finalization. This achieves permissionless participation with fast deterministic finality.

6.1 Design Rationale

6.1.1 Why Not Pure PoW?

Nakamoto consensus provides permissionless participation but suffers from:

- **Slow finality**: Probabilistic finality requires multiple confirmations (typically 10–60 minutes).
- **Reorg vulnerability**: Transactions can be reversed by chain reorganizations, even after confirmation.
- **Energy waste**: Hashpower expended on orphaned blocks provides no value.

6.1.2 Why Not Pure BFT?

Classical BFT protocols provide deterministic finality but require:

- **Known participants**: Fixed validator sets conflict with permissionless design.
- **Quadratic messaging**: $O(n^2)$ communication limits scalability.
- **Synchrony assumptions**: Liveness depends on network timing bounds.

6.1.3 Hybrid Approach

Bothocombines the strengths of both:

Property	PoW	SCP
Permissionless participation	✓	—
Fair distribution	✓	—
Deterministic finality	—	✓
Fast confirmation	—	✓
Byzantine fault tolerance	—	✓

Intuition: two jobs, two mechanisms. The table above is not a menu from which Bothopicks the better column; it is a division of labor. Consensus has to answer two separate questions, and no single mechanism answers both well. The first is *who gets to issue new coins and propose the next block*—an open-entry question, where the right answer is a permissionless, Sybil-resistant lottery, which is exactly what proof-of-work provides. The second is *which proposed block is final*—an agreement question, where the right answer is a Byzantine-agreement protocol that externalizes one block irreversibly, which is exactly what SCP provides. Bothoassigns each question to the mechanism suited to it and keeps the two deliberately decoupled: a miner's hashpower buys the right to *propose* and to earn issuance, but carries no weight in the SCP *vote* that finalizes the block. Mining weight and consensus voting weight are orthogonal.

This decoupling is what distinguishes Bothofrom both of its neighbors. MobileCoin adopts the same SCP finality but has no mining at all—its supply is premined and fixed, so it never needs a permissionless issuance lottery (Section 2). Proof-of-work coins such as Bitcoin have the issuance lottery but no SCP layer, so they inherit only probabilistic finality and remain reorg-vulnerable. By running both, Bothoobtains permissionless, fairly distributed issuance *and* fast deterministic finality simultaneously—the combination quantified in the comparison of Table 6. The cost is operating two subsystems rather than one; the benefit is that neither has to be compromised to approximate the other's guarantee.

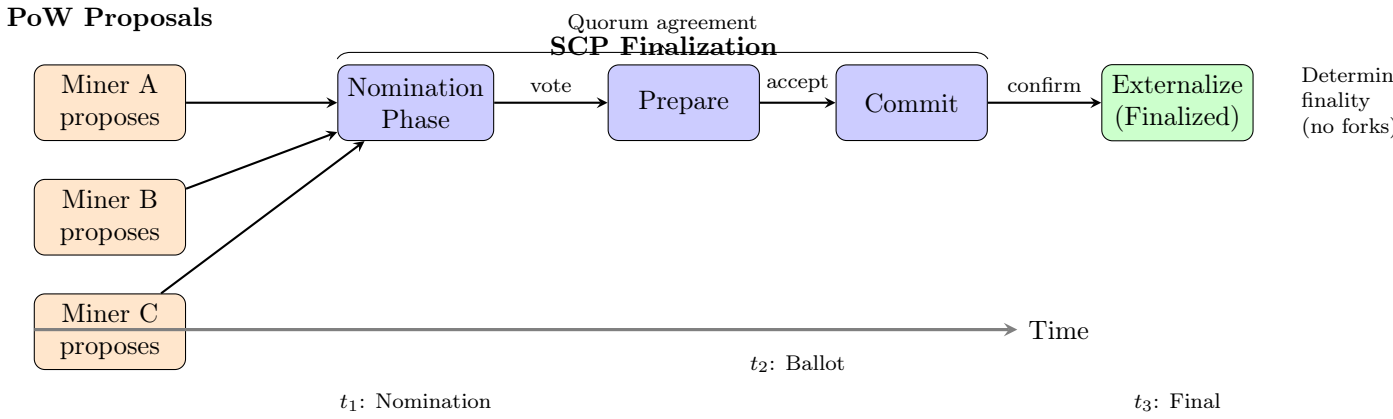


Figure 8: Hybrid PoW + SCP consensus flow. Multiple miners propose blocks via proof-of-work (permissionless participation), then SCP achieves deterministic finality through quorum agreement. Unlike pure PoW, finalized blocks cannot be reverted—providing instant settlement guarantees.

6.2 Stellar Consensus Protocol

SCP [38] achieves Byzantine agreement with open membership through *federated Byzantine agreement* (FBA).

6.2.1 Quorum Slices

Each node v declares a *quorum slice* $Q(v)$—the set of nodes v trusts for consensus. Unlike classical BFT where all nodes agree on membership, SCP allows heterogeneous trust:

Definition 6.1 (Quorum Slice)**.** A quorum slice for node v is a set $Q(v) \subseteq V$ such that $v \in Q(v)$ and v will accept a statement if all nodes in $Q(v)$ accept it.

Definition 6.2 (Quorum)**.** A set $U \subseteq V$ is a quorum if for every node $v \in U$, there exists a quorum slice $Q(v) \subseteq U$.

Informally, a quorum is a set of nodes sufficient for agreement—each member has "enough" trusted nodes within the set to be convinced.

6.2.2 Quorum Intersection

Safety requires that any two quorums overlap:

Definition 6.3 (Quorum Intersection)**.** A system has quorum intersection if for all quorums U_1, U_2: $U_1 \cap U_2 \neq \emptyset$.

Theorem 6.4 (SCP Safety)**.** *If the network has quorum intersection and the Byzantine threshold is not exceeded—so that the intersection of any two quorums contains at least one honest node—then SCP provides safety: no two honest nodes externalize different values for the same slot. Safety does not require the absence of Byzantine nodes, only that no two quorums can be satisfied entirely by disjoint, adversary-controlled sets.*

Safety and liveness are distinct properties with distinct assumptions. Safety, as stated above, holds under quorum intersection among honest nodes even under asynchrony. Liveness—that consensus eventually makes progress—is a separate guarantee requiring an intact, available honest quorum and network synchrony; it is treated in Section 6.8. When these liveness assumptions fail (e.g., a network partition), Bothopreserves safety by halting rather than forking.

6.2.3 Tiered Quorum Structure

Bothouses a tiered quorum structure balancing decentralization with robustness:

```
QuorumSlice
    // Tier 1: Infrastructure nodes (high-uptime, well-connected)
    threshold: 3,
    validators: [
        "node1.botho.org",
        "node2.botho.org",
        "node3.botho.org",
        "node4.botho.org",
    ],
    // Tier 2: Community validators
    inner_sets: [

            threshold: 2,
            validators: ["community1", "community2", "community3"],
        ,
    ],
```

This requires agreement from 3 of 4 infrastructure nodes AND 2 of 3 community validators, ensuring both stability and decentralization.

6.3 Consensus Phases

The consensus process proceeds through four phases for each block slot:

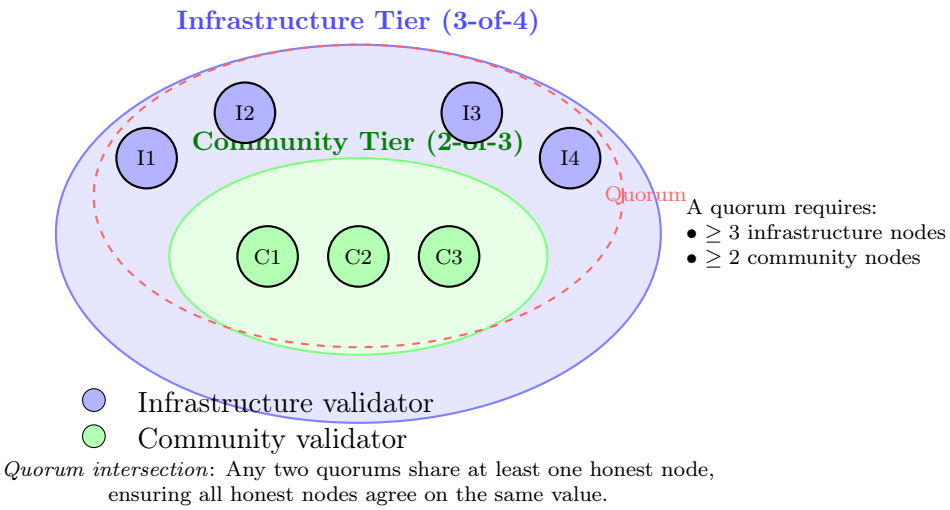


Figure 9: Tiered quorum slice structure in Botho. Each node's quorum slice requires agreement from 3-of-4 infrastructure validators AND 2-of-3 community validators. This design balances network stability (infrastructure tier) with decentralization (community tier). Quorum intersection is guaranteed when Byzantine nodes are below threshold.

6.3.1 Phase 1: Block Proposal (PoW)

Miners compete to propose blocks via proof-of-work:

1. Miner constructs candidate block with transactions from mempool
2. Miner searches for a nonce whose proof-of-work hash h_{PoW} (the RandomX preimage over the minter's address keys defined in Section 4.5) meets the difficulty target, i.e. $h_{\text{PoW}} <$ target
3. First valid block is broadcast to the network
4. Multiple proposals may arrive; SCP selects among them

Why PoW for proposal?

- **Permissionless**: Anyone can propose blocks
- **Fair distribution**: Block rewards are distributed by computational contribution
- **Sybil resistance**: Creating proposals requires real resources

6.3.2 Phase 2: Nomination

Nodes nominate candidate values (block hashes) for the slot:

1. Node receives valid block proposals
2. Node nominates the first valid proposal received (tie-breaking by lowest hash)
3. Nodes accept nominations from their quorum slices
4. Nomination converges to a single candidate when a quorum agrees

```
NominationMessage
    slot_index: u64,
    voted: Vec<BlockHash>,      // Values this node votes for
    accepted: Vec<BlockHash>,   // Values this node has accepted
```

6.3.3 Phase 3: Ballot Protocol

Once nomination produces a candidate, nodes run the ballot protocol to commit to a specific value:

1. **Prepare**: Nodes vote to prepare a ballot (n, v) where n is a counter and v is the candidate value
2. **Commit**: Once prepared, nodes vote to commit the ballot
3. **Abort**: If a ballot cannot progress, nodes abort and try a higher ballot number

The ballot protocol ensures:

- No two different values can be committed for the same slot
- Progress is made despite Byzantine nodes (up to threshold)
- Aborted ballots do not block future ballots

6.3.4 Phase 4: Externalize

When a ballot is committed, nodes externalize the value:

```
ExternalizeMessage
    slot_index: u64,
    commit: Ballot,              // The committed ballot
    quorum_set_hash: Hash,       // Proves quorum agreement
```

Externalization represents deterministic finality—the value cannot be changed without violating quorum intersection.

6.4 Block Structure

```
Block
    header: BlockHeader,
    minting_tx: MintingTransaction,
    transactions: Vec<PrivateTransaction>,
    scp_proof: SCPProof,


BlockHeader
    version: u32,
    prev_block_hash: Hash,
    tx_root: Hash,
    timestamp: u64,
    height: u64,
    difficulty: u64,
    nonce: u64,                  // PoW solution
    minter_view_key: [u8; 32],   // Bound by the PoW preimage
    minter_spend_key: [u8; 32],  // Bound by the PoW preimage


SCPProof
    slot_index: u64,
    externalize_messages: Vec<ExternalizeMessage>,
    // Sufficient messages to prove quorum agreement
```

6.5 Difficulty Adjustment

Bothouses a responsive difficulty adjustment algorithm:

$$\text{difficulty}_{n+1} = \text{difficulty}_n \times \frac{T_{\text{target}}}{T_{\text{actual}}} \tag{32}$$

where:

- T_{target} is the target block time (dynamically adjusted, see Section 7)
- T_{actual} is the actual time for the last adjustment window (144 blocks)

Adjustments are bounded to $[0.5, 2.0]\times$ per window to prevent oscillation.

6.6 Fork Resolution

Unlike pure PoW where the longest chain wins, Botho's SCP finalization makes forks impossible for externalized blocks:

Theorem 6.5 (Fork Freedom)**.** *If the network has quorum intersection and the Byzantine threshold is not exceeded, no two honest nodes can externalize different blocks at the same height.*

Proof. We prove by contradiction, following the structure of Mazières' SCP safety proof [38].

Definitions. Let:

- V denote the set of all nodes
- $Q : V \to 2^{2^V}$ denote the quorum slice function
- $\mathcal{Q} \subseteq 2^V$ denote the set of all quorums
- $\text{ext}_v(s)$ denote node v's externalized value for slot s
- $\mathcal{B} \subset V$ denote Byzantine nodes, $|\mathcal{B}| \leq f$

Quorum intersection property. By assumption:

$$\forall Q_1, Q_2 \in \mathcal{Q} : (Q_1 \setminus \mathcal{B}) \cap (Q_2 \setminus \mathcal{B}) \neq \emptyset$$

That is, any two quorums share at least one honest node.

Ballot protocol invariant. The SCP ballot protocol maintains the following invariant for each slot s:

> *Commit Exclusivity*: If honest node v commits ballot (n, x) for slot s, then no honest node commits ballot (n', x') for the same slot with $x' \neq x$ and $n' \leq n$.

This invariant is established through the prepare phase: a node only commits (n, x) after confirming that no lower ballot with a different value can be committed [38].

Contradiction argument. Suppose, for contradiction, that honest nodes A and B externalize different values for slot s (corresponding to block height h):

$$\text{ext}_A(s) = b_A \neq b_B = \text{ext}_B(s)$$

Externalization requires commitment. Let:

- A commit ballot (n_A, b_A) with quorum $Q_A \in \mathcal{Q}$
- B commit ballot (n_B, b_B) with quorum $Q_B \in \mathcal{Q}$

Without loss of generality, assume $n_A \leq n_B$.

By quorum intersection, there exists honest node $C \in (Q_A \setminus \mathcal{B}) \cap (Q_B \setminus \mathcal{B})$.

Node C participated in both quorums. For C to be in Q_A, C must have voted to commit (n_A, b_A). For C to be in Q_B, C must have voted to commit (n_B, b_B).

Case 1: $n_A = n_B$. Node C voted to commit (n_A, b_A) and (n_A, b_B) with $b_A \neq b_B$. This violates the ballot protocol rule that honest nodes vote for at most one value per ballot number. $\Rightarrow$ Contradiction.

Case 2: $n_A < n_B$. For C to vote commit on (n_B, b_B), C must first confirm that no ballot $\leq n_B$ with value $\neq b_B$ is committed. But C voted to commit (n_A, b_A) with $n_A < n_B$ and $b_A \neq b_B$. This violates the prepare-before-commit requirement. $\Rightarrow$ Contradiction.

Both cases yield contradictions. Therefore, our assumption is false, and no two honest nodes can externalize different values for the same slot.

Mapping to block heights. Each consensus slot corresponds to exactly one block height. Externalized value = finalized block hash. Fork freedom at the consensus layer implies fork freedom in the blockchain.

Relationship to PoW. Multiple PoW blocks may be proposed for the same slot. SCP nomination deterministically selects one candidate before the ballot protocol begins. The proof above shows the selected candidate is agreed upon by all honest nodes. □

Corollary 6.6 (Finality Irreversibility)**.** *Once a block is externalized at height h, no reorganization can replace it without violating quorum intersection or corrupting $> f$ nodes.*

Proof. Reversing externalization at height h requires externalizing a different block b' for the same slot. By Theorem 6.5, this is impossible under the stated assumptions. An attacker must either:

1. Corrupt enough nodes to break quorum intersection, or
2. Corrupt $> f$ nodes to violate the Byzantine threshold

Both require compromising the network's trust assumptions, not merely accumulating hashpower (unlike pure PoW). □

Handling pre-finalization forks: Multiple PoW proposals may arrive before SCP converges. The nomination phase selects a unique winner based on:

1. First valid proposal received (per node)
2. Tie-breaking by lowest block hash
3. Quorum agreement determines final selection

6.7 Timing Analysis

Table 5: Consensus timing breakdown

Phase	**Time**
Block proposal (PoW)	Variable (3–40s target, 5s reference)
Nomination	∼1s
Ballot prepare	∼1s
Ballot commit	∼1s
Externalize	<1s
Total finality	Block time + ∼3–4s

In practice, finality occurs within 5 seconds of block proposal, compared to 10–60 minutes for pure PoW systems.

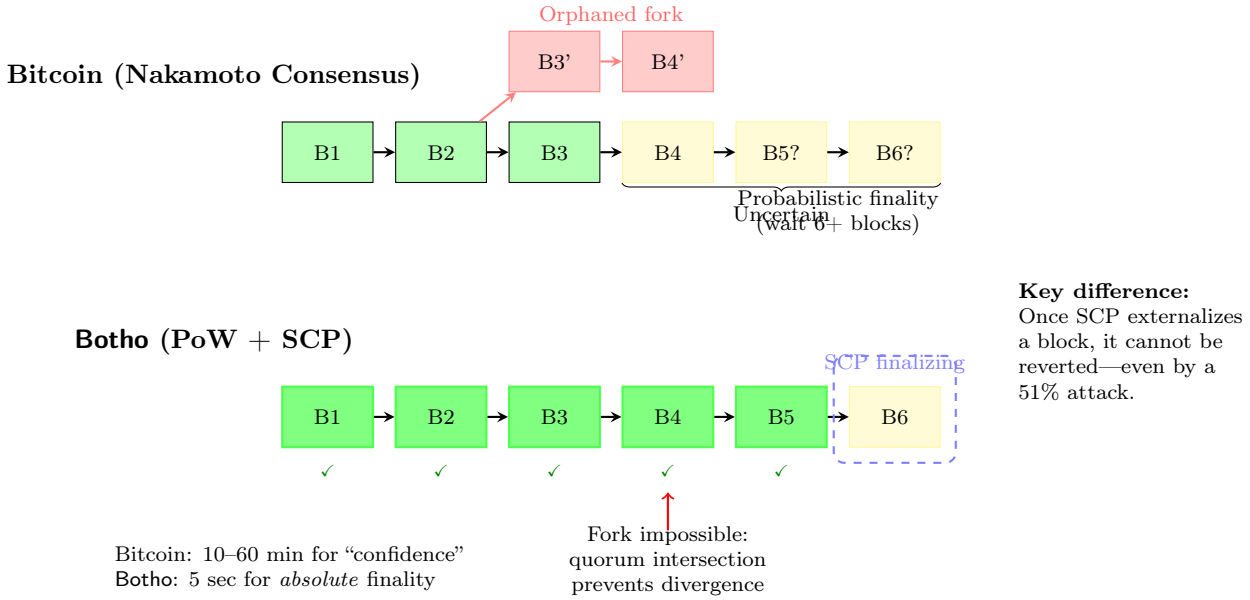


Figure 10: Fork prevention comparison. Bitcoin (top) achieves only probabilistic finality—transactions can be reversed by reorganizations, requiring multiple confirmations for confidence. Botho(bottom) achieves deterministic finality via SCP: once a block is externalized, quorum intersection mathematically guarantees no alternative history can exist. This enables instant settlement with no risk of reversal.

6.8 Liveness Guarantees

Theorem 6.7 (SCP Liveness)**.** *If the network is eventually synchronous and at most f nodes are Byzantine (where each quorum can tolerate f failures), then SCP eventually makes progress.*

Graceful degradation: If quorum intersection fails (e.g., due to network partition), safety is preserved—nodes simply halt rather than fork. This is a conscious design choice: safety over liveness.

6.9 Security Properties

6.9.1 Byzantine Fault Tolerance

The system tolerates Byzantine behavior from nodes outside any quorum's blocking threshold. For the default tier structure:

- Infrastructure tier: 1 of 4 can be Byzantine
- Community tier: 1 of 3 can be Byzantine

6.9.2 Nothing-at-Stake Resistance

Unlike pure proof-of-stake systems, PoW proposal ensures:

- Proposing multiple blocks requires multiple PoW solutions
- Resources are burned regardless of which block is selected
- No advantage to "voting for everything"

6.9.3 Long-Range Attack Resistance

SCP finality prevents long-range attacks:

- Once externalized, blocks cannot be reverted
- Rewriting history requires corrupting quorum intersection
- No "weak subjectivity" bootstrap problem

6.10 Comparison with Alternatives

We compare against Nakamoto longest-chain consensus [43] and BFT-style deterministic finality as exemplified by Tendermint [35].

Table 6: Consensus mechanism comparison

Property	Botho	Bitcoin	Tendermint
Finality	Deterministic	Probabilistic	Deterministic
Finality time	∼5–10s	∼60 min	∼6s
Permissionless	Yes	Yes	No
Fork possible	No	Yes	No
Byzantine tolerance	Quorum-based	50% hashpower	1/3 validators
Energy efficiency	Medium	Low	High

6.11 Mining Pool Considerations

Mining pools aggregate hashpower from multiple miners, distributing rewards proportionally. Botho's hybrid consensus requires adaptations to traditional pooling protocols.

6.11.1 Pool Protocol Compatibility

Standard Stratum-style protocols require modification for PoW+SCP:

```
PoolWorkAssignment
    // Standard PoW fields
    job_id: u64,
    prev_hash: Hash,
    coinbase_template: Vec<u8>,
    merkle_branches: Vec<Hash>,
    target: Difficulty,

    // Botho-specific: SCP context
    slot_index: u64,
    quorum_set_hash: Hash,
    nomination_deadline: Timestamp,
```

Key difference: Pools must track SCP slot progress and coordinate work assignments with nomination deadlines. Stale work (missed nomination) produces no reward even if PoW is valid.

6.11.2 Block Reward Attribution

Minting transactions in Bothoinclude the minter's public key, creating challenges for pool reward distribution:

- **Pool-controlled keys**: Pool creates minting transaction, distributes rewards via separate payment transactions
- **Pay-per-share (PPS)**: Pool pays from reserve for valid shares, assumes variance
- **Proportional (PROP)**: Miners receive proportion of actual block rewards found

Cluster tag implications: Pool-minted coins carry the pool's cluster tag. Miners receiving payouts inherit blended tags from the pool's operations.

6.11.3 SCP Participation

Pools face a choice in SCP participation:

1. **Pool as SCP node**: Pool runs consensus, coordinates with quorum. Requires high uptime and expertise.
2. **Delegate SCP**: Pool submits valid blocks to SCP-participating nodes for finalization. Simpler but adds trust.

Most pools will likely delegate SCP participation to infrastructure nodes while focusing on PoW coordination.

6.11.4 Pool Centralization Risks

Pooled mining creates centralization pressure:

- **Hash rate concentration**: Large pools may approach majority hashpower
- **Censorship capability**: Pools could refuse to include certain transactions
- **Network attacks**: Colluding pools could attempt consensus manipulation

Mitigations:

- SCP finality requires quorum agreement beyond PoW majority
- Transparent pool policies and open-source implementations encouraged
- Economic incentives favor decentralization (smaller pools have lower variance for participants)

6.11.5 Solo Mining Viability

Solo mining remains viable under certain conditions:

Table 7: Solo vs. pool mining trade-offs

Factor	Solo	Pool
Variance	Very high	Low
Cluster purity	100% own cluster	Blended with pool
Privacy	Maximum	Pool sees hashrate
Minimum hashrate	$\sim$1% network	Any
Technical complexity	Higher	Lower

Recommendation: Users prioritizing cluster tag purity (to minimize progressive fees) should consider solo mining despite higher variance.

6.11.6 Decentralized Pool Alternatives

P2Pool-style decentralized pools offer middle ground:

- **Share chain**: Miners build separate share chain proving work
- **Trustless payouts**: Block rewards distributed automatically based on share chain
- **No central operator**: Removes single point of failure/censorship

Botho adaptation: P2Pool would need SCP integration for share chain finalization. Research and development ongoing.

7 Monetary Policy

Botho's monetary policy balances long-term security sustainability with controlled inflation, using dynamic mechanisms that respond to network conditions.

7.1 Design Goals

1. **Sustainable security**: Perpetual miner incentive without relying solely on transaction fees.
2. **Fair distribution**: Rewards distributed over time, not front-loaded to early adopters.
3. **Anti-hoarding**: Economic pressure toward circulation rather than accumulation.
4. **Predictability**: Clear, auditable emission schedule.

7.2 Emission Schedule

7.2.1 Initial Emission

Block rewards follow a halving schedule:

$$R(h) = R_0 \cdot \left(\frac{1}{2}\right)^{\lfloor h/H \rfloor} \tag{33}$$

where:

- $R(h)$ is the block reward at height h
- $R_0 = 50$ BTH is the initial block reward
- $H = 6{,}307{,}200$ blocks (≈ 1 year at 5s blocks) is the halving period

After $K = 5$ halvings (≈ 5 years), the per-block reward has fallen from 50 BTH to 3.125 BTH and the chain transitions to perpetual tail emission. The geometric sum of the five halving epochs fixes the *Phase-1 supply* at

$$S_1 = R_0 H \sum_{i=0}^{K-1} 2^{-i} = 50 \times 6{,}307{,}200 \times \tfrac{31}{16} = 611{,}010{,}000 \text{ BTH } (\approx 611\text{M}). \tag{34}$$

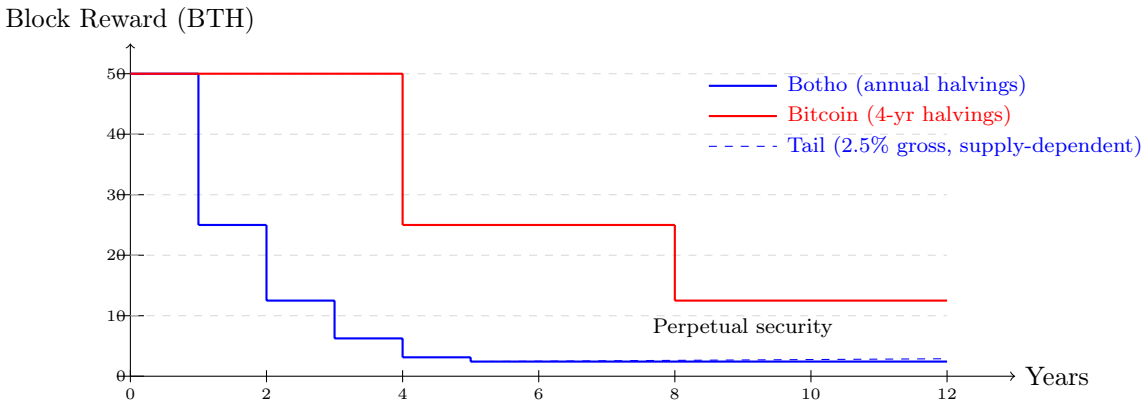


Figure 11: Block reward emission schedule. Botho halves the reward each year (50, 25, 12.5, 6.25, 3.125 BTH) for 5 years, then transitions to perpetual supply-dependent tail emission at a 2.5% gross annual rate (≈2.42 BTH/block at the ≈611M tail-onset supply, rising as supply grows), compared to Bitcoin's 4-year halvings toward zero. Tail emission ensures long-term security funding.

7.2.2 Tail Emission

After the five halvings complete, perpetual tail emission begins. Rather than a fixed per-block constant, the tail reward is a fixed *annual rate* on the cumulative supply, so emission scales with supply rather than being pinned to an absolute number:

$$R_{\text{tail}} = \frac{S \cdot (\beta_{\text{net}} + \beta_{\text{burn}})}{N_{\text{year}}} \tag{35}$$

where:

- S is the cumulative gross issuance so far (every block reward ever minted, not reduced by burns), *recomputed every block*
- $\beta_{\text{net}} = 0.02$ (200 bps) is the design net annual inflation
- $\beta_{\text{burn}} = 0.005$ (50 bps) is an *assumed* annual fee-burn rate (the 20% burn, see Section 10.6), added so that *gross* emission would net to 2% if burns reached it
- $N_{\text{year}} = 6{,}307{,}200$ is the number of blocks per year at 5s blocks

Anchoring this to the Phase-1 supply $S_1 = 611{,}010{,}000$ BTH gives, at tail onset:

$$\text{Gross annual emission} = 611{,}010{,}000 \times 0.025 = 15{,}275{,}250 \text{ BTH} \tag{36}$$

$$\text{Net annual emission} = 611{,}010{,}000 \times 0.020 = 12{,}220{,}200 \text{ BTH} \tag{37}$$

$$R_{\text{tail}} = \frac{15{,}275{,}250}{6{,}307{,}200} \approx 2.42 \text{ BTH/block (gross)} \tag{38}$$

or ≈ 1.94 BTH/block net of the assumed fee burn. Because each tail reward is added back into S, the per-block reward *grows* as the supply grows and gross issuance compounds at ≈2.53% per N_{year} blocks.

Realized net rate. The 0.5% burn is a calibration assumption, not an expected outcome. Only 20% of fees are burned, and a typical 1-input/2-output transaction pays the 0.0001 BTH relay minimum, so burning 0.5% of the tail-onset supply each year would take about 4,800 transactions per second, sustained. At the loads the timing tiers anticipate, the base-fee burn is a few thousandths of a percent of supply. Demurrage is collected as part of the fee, so 20% of it is

burned too; that term is at most ≈0.4%/year, and only if every coin sat at the maximum cluster factor and was settled yearly. The realized net tail is therefore between ≈2.1% and ≈2.53% (compounded) per N_{year} blocks, near the top in practice, rather than 2%. The parameters are unchanged (ratified); this restates what they imply. Whether to keep them and restate the headline figure, or recalibrate the tail (a consensus change), is an open operator decision tracked in issue #1788.

7.2.3 Supply Projection

Table 8: Supply projection

Year	Circulating	New Emission	Gross Inflation
1	315.4M BTH	315.4M	—
2	473.0M BTH	157.7M	50%
3	551.9M BTH	78.8M	17%
5	611.0M BTH	19.7M	3.3%
6	626.5M BTH	15.5M	2.5%
10	692.4M BTH	17.1M	2.5%
20	889.0M BTH	22.0M	2.5%

Note: Years are *policy-years* of 6,307,200 blocks (calendar years only at 5 s blocks; see Emission Bounds). Years 1–5 are the halving phase (rewards 50, 25, 12.5, 6.25, 3.125 BTH/block), which fix the Phase-1 supply at ≈611M BTH. From year 6 onward the tail reward is recomputed every block from cumulative issuance, so gross issuance compounds at ≈2.53% per policy-year (2.5% calibrated, compounded per block). Burns are omitted: at realistic fee volumes they are far below the assumed 0.5%/year (see Realized net rate above). Figures are the node's exact issuance (`cluster-tax-sim emission-sweep`, node-parity section).

7.3 Dynamic Block Timing

Bothointroduces adaptive block intervals that respond to network utilization, creating natural inflation dampening.

7.3.1 Mechanism

The target block time is a deterministic function of recent transaction load, computed over a smoothing window of the last 10 blocks. Rather than a continuous curve, the protocol selects from five discrete levels keyed on the measured transaction rate (transactions per second):

$$T_{\text{target}}(\rho) = \begin{cases} 3\text{s} & \rho \geq 20 \text{ tx/s} \\ 5\text{s} & \rho \geq 5 \text{ tx/s} \\ 10\text{s} & \rho \geq 1 \text{ tx/s} \\ 20\text{s} & \rho \geq 0.2 \text{ tx/s} \\ 40\text{s} & \text{otherwise (idle)} \end{cases} \tag{39}$$

where ρ is the transaction rate measured over the smoothing window. The absolute consensus floor is $T_{\min} = 3$ seconds (SCP needs time to externalize) and the idle ceiling is $T_{\max} = 40$ seconds. Because the level is computed from committed chain state, every validator derives the same target for a given tip.

The 5-second level is the *monetary reference*: all emission and inflation figures in this section are computed at 5 s blocks ($N_{\text{year}} = 6{,}307{,}200$), the pace the network holds under sustained high load. Slower levels produce proportionally fewer blocks per year and therefore lower effective

Table 9: Block timing by transaction load

Network State	Tx Rate	Block Time
Idle	<0.2 tx/s	40s
Low usage	≥0.2 tx/s	20s
Medium usage	≥1 tx/s	10s
High usage	≥5 tx/s	5s
Very high usage	≥20 tx/s	3s

emission (Section 7.3, Emission Bounds); the 3-second floor is reached only under transient maximum demand.

7.3.2 Economic Implications

Dynamic timing creates feedback loops:

Low utilization:

- Longer block times → fewer blocks per hour
- Fewer blocks → less emission
- Less emission → reduced inflation when currency is unused

High utilization:

- Shorter block times → more blocks per hour
- More blocks → higher throughput
- Higher throughput → more rewards when currency is actively used

This aligns miner incentives with network utility.

7.3.3 Emission Bounds

Because the tail reward is set per block, not per unit of time (at ≈2.42 BTH/block gross for the ≈611M tail-onset supply), slower blocks produce fewer blocks per year and therefore proportionally lower annual emission. The effective annual tail emission thus varies with utilization:

Table 10: Tail emission by load level (at ≈611M tail-onset supply)

Load Level	Blocks/Year	Gross Emission/Year
Idle (40s)	788,400	1.91M BTH
Medium (10s)	3,153,600	7.63M BTH
High, reference (5s)	6,307,200	15.28M BTH

At the 5 s monetary reference this is the 2.5% gross tail rate (2% net only if fee burns reached the assumed 0.5%); sustained slower operation acts as a natural inflation dampener. Blocks are load-paced, so the same dampening stretches the calendar: the 5 policy-years to tail take 10 years at 10 s blocks and 40 years when idle. The 3-second tier (sustained ≥20 tx/s) raises the block count to 10.5M/year, about 4.2% gross per calendar year, and reaches the tail in 3 years. The difficulty controller targets 5 s blocks but cannot make blocks faster than load pacing allows; how it behaves at sustained low load on the live network has not yet been measured.

7.4 Fee Economics

7.4.1 Base Fee

The minimum fee per byte is:

$$f_{\text{base}} = 1 \text{ pico-BTH per byte} \tag{40}$$

(Botho's smallest unit is the picocredit, 1 BTH $= 10^{12}$ pico-BTH.) The size term is multiplied by the cluster factor and by an output-count penalty ($\min(n_{\text{out}}, 10)^2$). For a typical 1-input/2-output transaction (990 bytes) of factor-1 coins, the consensus floor is

$$f_{\text{floor}} = 990 \times 1 \times 2^2 = 3{,}960 \text{ pico-BTH} \approx 4 \times 10^{-9} \text{ BTH}. \tag{41}$$

This is the block-validity floor. Default nodes also enforce a relay minimum of 10^8 pico-BTH (0.0001 BTH) per transaction, so that is what an ordinary wallet pays; the floor is what a miner including its own transactions pays.

The 1 pico-BTH/byte figure is the congestion *floor*. Under load the dynamic base rises multiplicatively (a congestion response bounded by a 100× safety clamp, i.e. 100 pico-BTH/byte), so the enforced minimum is

$$f_{\text{min}} = \underbrace{b_{\text{dyn}}(\text{congestion})}_{1\ldots100 \text{ pico/byte}} \times \text{size} \times \text{cluster_factor} \; + \; d(\vec{t}, v), \tag{42}$$

where $d(\vec{t}, v)$ is the *cluster demurrage* stock-level term (Section 5.4): a per-transfer charge proportional to the spent value, the annual demurrage rate, and the coins' cluster factor, so idle high-factor wealth carries a small holding cost that circulating low-factor coins avoid. Demurrage is exempt for factor-1 coins.

7.4.2 Progressive Fee Multiplier

Transactions pay additional fees based on cluster wealth (see Section 5.4):

$$f_{\text{total}} = f_{\text{base}} \times \text{size} \times \text{cluster_factor} \tag{43}$$

The cluster factor ranges from 1.0× (bottom 50% wealth) to 6.0× (top 1% wealth).

7.4.3 Demurrage Class-Transition Charge

Ordinary demurrage accrues *over holding time* and is paid on each spend. On its own this leaves a leak: a wealthy coin can be spent while young — before demurrage has accrued (`elapsed` ≈ 0) — into a background (factor-1) output, permanently escaping its class's future holding cost at near-zero cost. Botho closes this by charging, on every spend,

$$d_{\text{spend}} = \max\Big(\underbrace{d_{\text{accrued}}}_{\text{time-based}}, \quad \underbrace{d_{\text{cap}}}_{\text{capitalized reset}} \Big), \tag{44}$$

where d_{accrued} is the ordinary accrued demurrage and the *capitalized reset charge* d_{cap} prices a permanent *class downgrade* — the case where a spend declares an output factor *below* the input's ring-implied class floor (the factor the spender cannot rewrite). It equals the net future demurrage escaped by the downgrade, capitalized over a settlement horizon $H = 5$ years (`SETTLEMENT_HORIZON_BLOCKS` $= 31{,}536{,}000$ blocks at the 5 s reference):

$$d_{\text{cap}} = d(\text{value}, \phi_{\text{floor}}, H) - d(\text{value}, \phi_{\text{out}}, H) \quad \text{if } \phi_{\text{out}} < \phi_{\text{floor}}, \text{ else } 0. \tag{45}$$

A genuine background→background spend floors at 1× on both sides, so $d_{\text{cap}} = 0$ and honest commerce is never touched; only an actual downgrade relative to the ring floor is priced. The charge folds into the consensus fee floor and thus flows, via the 80/20 split, into the same lottery redistribution pool (ADR 0009; issue #925).

The ring floor is the *value-weighted centroid* of the ring members' effective cluster wealth, mapped through the factor curve (`ring_centroid_floored_factor`, `botho/src/ledger/store/validation.rs`). The floor is therefore not exact in either direction. A high-value wealthy decoy in an honest background spender's ring can lift it above 1× and trip the charge. In the other direction, a spender who pads the ring with high-value background decoys can pull it toward 1× and dilute the charge (as built, pending #1686). A ±10% **factor-similarity band** picker (`FACTOR_SIMILARITY_SPREAD`, `select_decoys_cluster_aware` in `botho/src/decoy_selection.rs`) was written to keep honest background rings near 1×, but it has no production caller. It is wallet policy at most, and consensus does not check it. The charge also uses the epoch's demurrage rate, which is zero for the first halving epoch (`demurrage_rate_bps`, `cluster-tax/src/monetary.rs`), so the class-transition and settlement charges are zero until block 6,307,200 (as built, pending #1685).

A holder may also *opt in* to a **demurrage-settlement operation** (issue #831): a transaction declaring all-background (factor-1) outputs pays the same capitalized reset charge — evaluated from the ring-floored input class down to 1× over the same horizon H — to permanently reclassify a wealthy coin to factor-1 and make it wrap-eligible for the bridge on-ramp (Section 11). Both doors (the implicit #925 downgrade charge and the explicit #831 settlement charge) read the *one* shared horizon H and reuse the same charge formula, so neither is a cheaper route out of the demurrage regime than the other.

7.4.4 Lottery-Based Fee Redistribution

Transaction fees are split between a redistribution pool and burning:

- **80% to the lottery pool**: Added to a persistent, cumulative pool and distributed to 4 UTXOs per block via a verifiable uniform lottery over a circulation window (*Path C*; ADR 0009)
- **20% burned**: Permanently removed from supply

The lottery pool is not fee-funded alone. Each block also routes a height-scheduled *emission share* of the block reward into the same pool: the share ramps from 0 at genesis by 10 percentage points per halving epoch to a 50% cap (the miner keeps the remainder). Emission-routing is what lets redistribution reach idle wealth that never transacts — transaction fees are a consumption tax and cannot, by themselves, touch coins that sit still (Section 10.6). The pool balance is cumulative (carried over across blocks) and each block pays out

$$\text{payout} = \min\Big(\underbrace{\text{carryover} + \text{emission_share} + 0.8\, f_{\text{block}}}_{\text{available}},\ R\Big), \qquad R = \min\big(\rho \cdot f_{\text{lott}},\ R(h)\big), \quad (46)$$

where the *reward cap* R is the smaller of the Path C cap $\rho \cdot f_{\text{lott}}$ and one block reward $R(h)$ (`reward_cap`, `botho/src/consensus/lottery.rs`; `compute_pool_accounting`, `lottery.rs`). Here ρ is the count of eligible outputs in the candidate set (up to 10,000, gathered over the whole circulation window), and $f_{\text{lott}} = 0.25$ BTH (`LOTTERY_BASE_FEE_PICO`, `cluster-tax/src/lottery.rs`) is a per-ticket *cap unit*. It is used only to size the cap; no transaction is charged 0.25 BTH, per output or otherwise. Any surplus *carries forward* to later blocks rather than being burned, so ≈100% of collected fees are eventually redistributed. The block-reward term $R(h)$ is the anti-grinding bound (Section 9). Because ρ counts the whole window, $\rho \cdot f_{\text{lott}}$ exceeds $R(h)$ whenever more than a few hundred outputs are eligible, so in practice the binding terms

are the available pool and $R(h)$ (pending #1682). A large carried-forward pool, for example a demurrage lump, therefore drains at up to one block reward per block to current ticket holders (pending #1691).

This creates:

- **Uniform selection**: Winners are drawn *uniformly* among eligible outputs: no value and no cluster factor enters the draw weight (`SelectionMode::Uniform`, `cluster-tax/src/lottery.rs`). Selection is not value-free, however: eligibility reads each output's amount against the 10^6 pico-BTH minimum (`min_utxo_value` in `lottery_candidates`, `botho/src/ledger/store/validation.rs`), which a confidential-amounts design would have to replace
- **Splitting (design intent, not met as built)**: Path C was designed so that a holder who splits a position into many outputs raises the cap by one f_{lott} per extra ticket and pays one f_{lott} per output, making splitting net-zero. As built, neither premise holds: nothing charges f_{lott} (an extra output costs roughly the 10^8 pico-BTH relay minimum, about 2,500× less), and ρ counts the whole window rather than one block's new outputs. Ticket-splitting is therefore profitable: a code-faithful re-run finds it profitable at all 192 points of its parameter grid (#1761, pending #1682). Coinbase and lottery payout outputs also become tickets at no fee, so past winners compound their share (pending #1690)
- **Progressive intake**: Redistribution progressivity now lives on the *source* side — the demurrage and progressive-fee terms that fill the pool are value- and factor-dependent. At today's constants the flat 10^8 pico-BTH relay minimum exceeds the factor-scaled size fee for ordinary transactions, so demurrage (from the second halving epoch on) is the term that is progressive in practice (pending #1688)
- **Deflationary pressure**: 20% burn creates supply reduction
- **Anti-MEV**: No miner incentive to reorder transactions
- **Alignment**: Miner income comes from its share of the block reward only

Earlier designs drew winners with weight $w_i = v_i \times (\phi_{\max} - \phi_i + 1)$ — proportional to UTXO value and tilted by the inverse cluster factor. That weight is *split-invariant* (splitting conserves value hence weight), but it reads the UTXO value directly, which is incompatible with the confidential amounts of ADR 0006/0009, and adversarial simulation (the `cluster-tax` lottery-selection sweep; Section 10.6) showed the value term dominates the factor tilt in realistic populations, routing much of the pool *back* to whales. Path C (ADR 0009, `docs/research/ct-compatible-lottery-selection.md`) retires the value tilt entirely: the payout is a uniform draw whose split-resistance was meant to come from the endogenous reward cap, not the weight. The research memo's net-zero bound assumes a per-block ρ and a per-output charge of f_{lott}; the implementation has neither, so the bound does not currently hold (pending #1682). In the earlier simulation's honest populations the uniform draw was more equalizing than the retired tilt; at production parameters it is not measurably equalizing (below).

Status of the redistribution claim. The mechanisms above are specified and implemented as described: fees and an emission share fill the pool, demurrage charges high-factor coins, and the pool pays uniformly drawn outputs. A measurable reduction in inequality from them is *not* demonstrated at production parameters, lottery ticket-splitting is profitable, and a redesign is under study (Section 10.6.2).

7.4.5 Lottery Mechanism

The draw runs once *per block*, over the block's accumulated pool:

1. Compute the fee split: burn $0.2\, f_{\text{block}}$; add $0.8\, f_{\text{block}}$ plus the emission share to the cumulative pool

2. Derive the seed from the parent block's proof-of-work fields (protocol 7.0.0, `lottery_seed_v2`, `botho/src/consensus/lottery.rs`):

$$s = \text{SHA256}(\text{“BOTHO_LOTTERY_SEED_V2”} \,\|\, \text{pow_hash}_{h-1} \,\|\, \text{prev_block_hash}_{h-1} \,\|\, h), \tag{47}$$

   where pow_hash_{h-1} is the parent's RandomX output and $\text{prev_block_hash}_{h-1}$ is the field of its PoW preimage. The parent's header hash is deliberately not an input, because its timestamp and coinbase output can be changed without new proof of work

3. Gather candidates: walk the UTXO set from a seed-derived offset, keeping outputs at least 720 blocks old, at most $N = 10{,}000$ blocks old (the circulation window), and worth at least 10^6 pico-BTH, up to 10,000 candidates (`lottery_candidates`, `botho/src/ledger/store/validation.rs`). Eligibility is by creation height, so coins that have not moved for 10,000 blocks drop out. Let ρ be the resulting count of eligible outputs

4. Set the reward cap $R = \min(\rho \cdot f_{\text{lott}}, R(h))$ and pay out min(available, R) this block, carrying any surplus forward

5. Select 4 winners *uniformly* without replacement: every eligible output has equal probability, with no value or cluster-factor weight

6. Split the capped payout equally among the winners (integer remainder to the last winner)

The seed is fixed once the parent's proof of work is found, and re-rolling it requires a new PoW solution. The candidate set, however, is read from the state after block $h-1$, and the PoW does not commit to $h-1$'s transactions. Whoever assembles block $h-1$'s body after its PoW is solved can choose which eligible outputs it spends, which changes block h's candidates and winners at roughly one transaction fee per alternative (as built, pending #1689). The grinding bound and its caveats are in Section 9 (Lottery Grinding); the splitting bound is not met as built (above, #1682).

7.4.6 Effective Inflation

The effective inflation rate becomes:

$$\text{inflation}_{\text{effective}} = \frac{\text{emission} - \text{fees_burned}}{\text{supply}} \tag{48}$$

where $\text{fees_burned} = 0.2 \times \text{total_fees}$. Under high utilization with progressive fees, burning may exceed tail emission, creating net deflation.

7.4.7 Fee Flow Analysis

Table 11: Fee flow scenarios (1M daily transactions)

Wealth Distribution	Daily Fees	Redistributed	Burned
Equal (factor 1.0)	24.7 mBTH	19.8 mBTH	4.9 mBTH
Current inequality	74.1 mBTH	59.3 mBTH	14.8 mBTH
High inequality	148.2 mBTH	118.6 mBTH	29.6 mBTH

Assumptions and derivation: the per-transaction figure is the congestion-floor consensus minimum for a standard transfer, computed directly from the fee formula (`minimum_fee_dynamic_with_outputs`

`botho/src/ledger/store.rs`). Take a standard 2-output transfer (payment + change) from a factor-1 cluster. Under the universal ML-KEM-768 stealth format (Section 4.2) every output carries a mandatory 1,088-byte encapsulation ciphertext, so a ≈4,000-byte transaction body grows by $2 \times 1{,}088 = 2{,}176$ bytes to size $\approx 6{,}176$ bytes. At the congestion floor $b = 1$ pico-BTH/byte:

$$f_{\text{tx}} = \underbrace{b}_{1} \times \underbrace{\text{size}}_{6{,}176} \times \underbrace{\text{cluster_factor}}_{1.0} \times \underbrace{\text{output_penalty}}_{2^2=4} = 24{,}704 \text{ pico-BTH} = 2.47 \times 10^{-8} \text{ BTH}, \tag{49}$$

i.e. ≈24.7 nano-BTH per transaction (the 2-output superlinear penalty $\min(2, 10)^2 = 4$ is part of the enforced minimum). Over 1M transactions/day the factor-1 aggregate is $10^6 \times 24{,}704$ pico-BTH $= 2.47 \times 10^{10}$ pico-BTH $= 2.47 \times 10^{-2}$ BTH ≈ 24.7 mBTH. The remaining rows scale the aggregate by each distribution's mean cluster factor ($\approx 3\times$ and $\approx 6\times$, within the 1–6 range). These are floor-level flow magnitudes; a rising dynamic base (up to the 100× clamp) or larger, multi-output transactions raise them proportionally.

7.5 Minter Incentives

7.5.1 Block Reward Composition

Minters receive the block reward minus the lottery emission share, and no fees:

$$\text{minter_income} = (1 - f_{\text{em}}(h)) \cdot R(h) \tag{50}$$

where $f_{\text{em}}(h)$ is 0 in the first halving epoch and rises 10 percentage points per epoch to a 50% cap (`lottery_emission_bps`, `cluster-tax/src/monetary.rs`; `miner_payout`, `botho/src/block.rs`).

This simplifies minting economics and eliminates fee-based attacks.

7.5.2 Profitability Analysis

Minting profitability depends on:

- Hardware cost (one-time)
- Electricity cost (ongoing)
- Block reward value
- Network hashrate (competition)

The tail emission ensures perpetual profitability for efficient miners, unlike Bitcoin where fee-only income creates uncertainty.

7.5.3 Decentralization Incentives

Several mechanisms encourage mining decentralization:

- **CPU-friendly PoW**: Algorithm resists ASIC development (RandomX-based [62])
- **Solo mining viable**: Tail emission ensures consistent income even at low hashrate
- **No economy of scale**: Linear scaling of rewards with hashpower

7.6 Long-Term Sustainability

7.6.1 Security Budget

The perpetual security budget (in BTH terms) is:

$$\text{security_budget} = R_{\text{tail}} \times \text{blocks_per_year} \tag{51}$$

At the ≈611M tail-onset supply this ranges from ≈1.9M BTH (idle) to ≈15.3M BTH (maximum utilization) annually, and grows with supply thereafter.

7.6.2 Comparison with Fixed Supply

Table 12: Security model comparison: fixed supply vs. tail emission

Aspect	Fixed Supply	Tail Emission
Long-term miner income	Fees only	Rewards + fees
Security if fees low	Degraded	Maintained
Inflation	0%	∼2–3%
Predictability	High	High

7.6.3 Wealth Tax Equivalence

Tail emission functions as a mild wealth tax:

$$\text{effective_tax} = \frac{\text{tail_emission}}{\text{total_supply}} \approx 2\% \tag{52}$$

This encourages circulation: holding idle currency means gradual dilution, while active participation maintains relative wealth.

7.7 Monetary Constants

8 Network Protocol

Bothoemploys a peer-to-peer network protocol optimized for privacy transaction propagation and consensus message delivery.

8.1 Network Architecture

8.1.1 Node Types

- **Full nodes**: Store complete blockchain, validate all transactions, participate in consensus
- **Minting nodes**: Full nodes that additionally perform PoW and propose blocks
- **Light clients**: Store block headers only, verify transactions via inclusion proofs

8.1.2 Transport Layer

All peer-to-peer communication uses:

- **TCP**: Reliable delivery for consensus messages
- **Noise Protocol** [53]: Authenticated encryption with forward secrecy
- **Multiplexed streams**: Separate channels for different message types

Table 13: Monetary policy constants

Parameter	Value	Description
Initial reward	50 BTH	Block reward at genesis
Halving period	6,307,200 blocks	$\approx$ 1 year (at 5s blocks)
Halving count	5	Time-to-tail $\approx$ 5 years
Phase-1 supply	$\approx$611M BTH	Geometric sum of 5 halvings
Tail inflation	2.5% gross/yr	2% net + 0.5% assumed burn; realized net $\approx$2.1–2.53%
Tail reward	$\approx$2.42 BTH/block	Gross, at 611M supply (recomputed each block)
Min block time	3 seconds	Consensus floor (very high load)
Reference block time	5 seconds	Monetary-calculation reference
Max block time	40 seconds	Idle ceiling
Base fee rate	1 pico-BTH/byte	Congestion floor (dynamic, $\leq$100$\times$)
Lottery winners/block	4	Uniform draw (Path C); eligibility reads amount
Lottery circulation window	10,000 blocks	Eligibility horizon ($\approx$14 h)
Lottery cap unit	0.25 BTH	Sizes $R = \min(\rho f_{\text{lott}}, R(h))$; not charged
Emission-to-lottery cap	50% of reward	Ramps 10%/halving epoch
Settlement horizon	5 years	Capitalized reset / class-downgrade charge
Decoy factor band	$\pm$10%	Unwired wallet picker; not consensus (#1686)
Max cluster factor	6.0$\times$	Top 1% wealth multiplier
Decimals	12	Smallest unit: 1 picocredit

8.2 Peer Discovery

8.2.1 Bootstrap Nodes

New nodes connect to hardcoded bootstrap nodes to discover initial peers:

```
BootstrapNodes
    dns_seeds: [
        "seed1.botho.org",
        "seed2.botho.org",
        "seed3.botho.org",
    ],
    static_peers: [
        "203.0.113.1:9732",
        "203.0.113.2:9732",
    ],
```

8.2.2 Kademlia DHT

Peer discovery uses a modified Kademlia DHT [37]:

- Node IDs derived from public keys (not chosen)
- XOR distance metric for routing
- Iterative lookup with parallel queries
- Periodic bucket refresh

Table 14: Connection limits

Category	Limit
Outbound connections	8
Inbound connections	117
Total connections	125
Connections per IP	2

8.2.3 Peer Limits

8.3 Message Protocol

8.3.1 Message Types

```
enum Message
    // Handshake
    Hello  version, genesis_hash, best_height ,
    HelloAck  version, genesis_hash, best_height ,

    // Block propagation
    NewBlock  block ,
    GetBlocks  locator, stop_hash ,
    Blocks  blocks ,

    // Transaction propagation
    NewTransaction  tx ,
    GetTransactions  hashes ,
    Transactions  txs ,

    // Consensus (SCP)
    SCPNomination  msg ,
    SCPPrepare  msg ,
    SCPCommit  msg ,
    SCPExternalize  msg ,

    // Compact blocks
    CompactBlock  header, short_ids ,
    GetBlockTxs  block_hash, indices ,
    BlockTxs  block_hash, txs ,
```

8.3.2 Message Serialization

Messages are serialized using a canonical binary format:

- Little-endian byte order
- Varint encoding for lengths
- No padding or alignment
- Deterministic ordering of map keys

8.3.3 Wire Format Specification

Each message follows the structure:

```
Message
    magic: [u8; 4],        // Network identifier: 0x00 0x0B 0x07 0xB0
    type_id: u8,           // Message type discriminant
    length: varint,        // Payload length in bytes
    checksum: [u8; 4],     // First 4 bytes of SHA256(payload)
    payload: [u8; length], // Type-specific data
```

Message type identifiers:

Table 15: Message type encoding

ID	Message	Max Size
0x00	Hello	256 B
0x01	HelloAck	256 B
0x10	NewBlock	2 MB
0x11	GetBlocks	4 KB
0x12	Blocks	10 MB
0x20	NewTransaction	100 KB
0x21	GetTransactions	4 KB
0x22	Transactions	1 MB
0x30	SCPNomination	64 KB
0x31	SCPPrepare	64 KB
0x32	SCPCommit	64 KB
0x33	SCPExternalize	64 KB
0x40	CompactBlock	64 KB
0x41	GetBlockTxs	4 KB
0x42	BlockTxs	1 MB
0xFF	Ping/Pong	8 B

Field-level encoding:

```
// Varint: 7 bits per byte, MSB = continuation
fn encode_varint(n: u64) -> Vec<u8>
    let mut bytes = Vec::new();
    let mut value = n;
    while value >= 0x80
        bytes.push((value & 0x7F) | 0x80);
        value >>= 7;

    bytes.push(value as u8);
    bytes


// Hash: 32 bytes, raw
Hash = [u8; 32]

// CompressedPoint: 32 bytes (Ristretto255)
Point = [u8; 32]

// Signature: Variable (CLSAG ~700B at ring 20)
Signature = varint(len) ++ [u8; len]

// Block header: Fixed 120 bytes
BlockHeader
    version: u8,             // 1 byte
```

```
    prev_hash: [u8; 32],     // 32 bytes
    merkle_root: [u8; 32],   // 32 bytes
    timestamp: u64,          // 8 bytes (little-endian)
    height: u64,             // 8 bytes
    difficulty: u64,         // 8 bytes
    nonce: u64,              // 8 bytes
    minter_key_hash: [u8; 32], // 32 bytes
```

8.4 Protocol State Machine

8.4.1 Connection States

Each peer connection transitions through defined states:

```
enum ConnectionState
    Connecting,      // TCP handshake in progress
    Authenticating,  // Noise protocol handshake
    Handshaking,     // Hello/HelloAck exchange
    Active,          // Normal operation
    Syncing,         // Initial block download
    Stale,           // No activity timeout
    Disconnecting,   // Graceful shutdown


// State transitions
Connecting     -> Authenticating  [TCP established]
Authenticating -> Handshaking     [Noise complete]
Handshaking    -> Active          [HelloAck received, compatible]
Handshaking    -> Disconnecting   [Incompatible version/genesis]
Active         -> Syncing         [Peer ahead by >10 blocks]
Syncing        -> Active          [Caught up]
Active         -> Stale           [No messages for 5 minutes]
Stale          -> Active          [Pong received]
Stale          -> Disconnecting   [No Pong after 30 seconds]
*              -> Disconnecting   [Error or ban]
```

8.4.2 Message Sequences

Connection establishment:

1. Initiator: TCP connect to peer:9732
2. Both: Noise XX handshake (mutual authentication)
3. Initiator: Send Hello {version, genesis_hash, best_height}
4. Responder: Send HelloAck (if compatible) or disconnect
5. Both: Transition to Active state

Block synchronization:

1. Node A: Send GetBlocks {locator, stop_hash}
2. Node B: Find common ancestor from locator
3. Node B: Send Blocks {blocks[ancestor+1..min(ancestor+500, stop)]}
4. Node A: Validate and apply blocks

5. Node A: Repeat from step 1 if more blocks needed

Transaction relay:

1. Sender: Receive transaction (local or from peer)
2. Sender: Validate transaction fully
3. If in stem phase: Forward to single stem peer
4. If in fluff phase: Announce via NewTransaction to all peers
5. Receivers: Request full transaction if not in mempool
6. Receivers: Validate and continue relay

Consensus message flow:

1. Proposer: Broadcast NewBlock upon PoW solution
2. Validators: Validate block, enter nomination
3. Validators: Exchange SCPNomination messages
4. Validators: Upon nomination convergence, send SCPPrepare
5. Validators: Upon prepare quorum, send SCPCommit
6. Validators: Upon commit quorum, send SCPExternalize
7. All nodes: Apply externalized block as final

8.4.3 Error Handling

Protocol errors trigger specific responses:

Table 16: Protocol error handling

Error	Action	Score Penalty
Invalid checksum	Drop message	-1
Unknown message type	Drop message	0
Message too large	Drop + warn	-10
Invalid block header	Drop block	-50
Invalid transaction	Drop tx	-10
Invalid signature	Drop + ban	-1000
Rate limit exceeded	Throttle	-5/excess
Protocol violation	Disconnect	-1000

Reconnection policy:

- Disconnected peers: Retry after exponential backoff (1s, 2s, 4s, ..., 1h max)
- Banned peers: No reconnection for 24 hours
- Repeated bans: Permanent blacklist after 3 bans

8.5 Block Propagation

8.5.1 Compact Block Relay

To reduce bandwidth, blocks are relayed in compact form:

1. Sender transmits compact block (header + short transaction IDs)
2. Receiver reconstructs from mempool
3. Receiver requests missing transactions by index
4. Sender provides requested transactions

Short ID computation:

$$\text{short_id} = \text{SipHash}(\text{nonce} \| \text{txid})_{[0:6]} \tag{53}$$

Average bandwidth savings: 90% (most transactions already in mempool).

8.5.2 Block Validation

Upon receiving a block:

1. Validate header (PoW, timestamp, difficulty)
2. Verify Merkle root
3. Validate each transaction
4. Verify minting transaction signature
5. Verify SCP proof (quorum agreement)
6. Apply to local state

8.6 Transaction Propagation

8.6.1 Dandelion++

Transaction propagation uses Dandelion++ [19] to protect sender network identity:

Stem phase:

- Transaction relayed along a random path
- Each hop has probability p to switch to fluff phase
- Path length follows geometric distribution

Fluff phase:

- Standard flooding to all peers
- Cannot trace back to stem origin

8.6.2 Transaction Validation

Transactions are validated before relay:

1. Syntactic validity (correct encoding)
2. Semantic validity (signatures verify)
3. Contextual validity (references exist, no double-spend)
4. Fee sufficiency

Invalid transactions are dropped; peers sending invalid transactions are deprioritized.

8.7 Synchronization

8.7.1 Initial Block Download

New nodes synchronize via:

1. Connect to peers and exchange best heights
2. Download block headers (verify PoW chain)
3. Request blocks in parallel from multiple peers
4. Validate and apply blocks sequentially
5. Join consensus once synchronized

8.7.2 Block Locator

Block requests include a locator to identify common ancestor:

```
fn block_locator(tip: Height) -> Vec<Hash>
    let mut locator = Vec::new();
    let mut step = 1;
    let mut height = tip;

    while height > 0
        locator.push(hash_at(height));
        if locator.len() > 10
            step *= 2;

        height = height.saturating_sub(step);

    locator.push(genesis_hash());
    locator
```

This logarithmic structure efficiently identifies divergence point.

8.7.3 Pruning

Full nodes may prune old blocks while retaining:

- Recent blocks (configurable, default 10,000)
- Block headers (all)
- UTXO set (current state)
- Key image set (all, for double-spend detection)

Pruned nodes cannot serve historical blocks but can fully validate new transactions.

8.8 Denial-of-Service Protection

8.8.1 Rate Limiting

Per-peer rate limits:

- Block requests: 10/second
- Transaction announcements: 100/second
- Consensus messages: 50/second

8.8.2 Peer Scoring

Peers maintain reputation scores:

- +1 for valid blocks/transactions
- -10 for invalid data
- -100 for protocol violations
- Disconnect at score $<$ -1000

8.8.3 Resource Bounds

- Maximum message size: 100 KB
- Maximum block size: 2 MB
- Maximum mempool size: 100 MB
- Maximum pending requests: 1000

8.9 Privacy Considerations

8.9.1 Traffic Analysis Resistance

- **Dandelion++**: Hides transaction origin
- **Encrypted transport**: Prevents content inspection
- **No timing correlation**: Messages are batched and jittered

8.9.2 Tor/I2P Support

Optional routing through anonymity networks:

- Tor hidden service support
- I2P integration planned
- Mixed clearnet/onion peer connections

8.10 Light Client Security

Light clients enable mobile and resource-constrained devices to participate in the network with reduced trust assumptions compared to custodial solutions.

8.10.1 Verification Capabilities

Light clients can verify:

- **Transaction inclusion**: Merkle proofs demonstrate transaction is in a specific block
- **Block header chain**: PoW verification confirms work was expended
- **SCP finality**: Externalization proofs confirm consensus agreement
- **Output ownership**: Local key derivation identifies owned outputs

Light clients *cannot* verify:

- **Transaction validity**: Ring signature verification requires full UTXO context
- **Double-spend prevention**: Requires complete key image database
- **Value conservation**: Commitment arithmetic needs all inputs

8.10.2 Trust Model

Light clients trust full nodes for:

1. **Transaction validity**: Full nodes could relay invalid transactions (mitigated by connecting to multiple independent nodes)
2. **Completeness**: Full nodes could withhold transactions (mitigated by querying multiple nodes)
3. **Privacy**: Query patterns reveal interest (mitigated by fetching broader data ranges)

Security level: Comparable to SPV in Bitcoin—economically secure against profit-motivated attackers but vulnerable to targeted attacks by well-resourced adversaries.

8.10.3 Output Scanning

Light clients scan for received funds:

```
LightClientScan
    // Request outputs in block range
    GetOutputs
        start_height: u64,
        end_height: u64,
    ,

    // Response: encrypted output data
    Outputs
        outputs: Vec<(Height, TxIndex, EncryptedOutput)>,
    ,
```

For each output, the client:

1. Attempts ML-KEM decapsulation with view key
2. If successful, derives one-time key and checks match
3. Decrypts amount if output belongs to wallet

Privacy consideration: Downloading all outputs in a range (rather than requesting specific ones) prevents servers from learning which outputs the client owns.

8.10.4 Transaction Submission

Light clients construct transactions locally:

1. Select ring members from cached output set
2. Build transaction with local keys (never transmitted)
3. Submit to multiple full nodes via Dandelion++ stem

Ring member selection: Light clients maintain a local cache of recent outputs sufficient for ring construction. This cache is populated during scanning and can be refreshed from any full node.

8.10.5 Privacy Implications

Light client operations leak information:

Table 17: Light client information leakage

Operation	Information Leaked
Block header sync	Active time periods
Output scanning	Block ranges of interest
Ring member fetch	Potential transaction construction
TX submission	Approximate location (IP-based)

Mitigation strategies:

- Connect via Tor/I2P for IP privacy
- Scan broader ranges than necessary
- Cache ring members proactively
- Submit transactions to random nodes

8.10.6 Comparison to Full Node Security

Table 18: Full node vs. light client security

Property	Full Node	Light Client
Validate all transactions	Yes	No
Detect double-spends	Yes	No
Verify consensus	Yes	Partial
Independent operation	Yes	No
Receive funds securely	Yes	Yes*
Send funds securely	Yes	Yes
Privacy (network)	High	Medium

**Light clients can verify they received funds but must trust that the sending transaction was valid.*

8.10.7 Mobile Wallet Recommendations

For mobile deployments:

- Use light client with multiple independent servers
- Prefer Tor connections when available
- Cache sufficient outputs for offline transaction construction
- Verify transaction inclusion before considering funds confirmed
- Consider running personal full node as trusted backend

8.11 Network Constants

Table 19: Network protocol constants

Parameter	Value	Description
Default port	9732	Main network
Testnet port	19732	Test network
Protocol version	1	Current version
Magic bytes	`0x000B07B0`	Network identifier
Handshake timeout	10s	Connection setup
Ping interval	60s	Keepalive

9 Security Analysis

This section analyzes Botho's security properties, threat models, and resistance to various attacks.

9.1 Threat Model

9.1.1 Adversary Capabilities

We consider adversaries with the following capabilities:

- **Network**: Can observe, delay, and inject messages (but not break encryption)
- **Computational**: Polynomial-time bounded (or BQP for quantum adversaries)
- **Economic**: Can acquire significant hashpower or stake
- **Passive**: Can collect and analyze blockchain data indefinitely

9.1.2 Security Goals

1. **Safety**: No double-spending; no unauthorized coin creation
2. **Liveness**: Valid transactions eventually confirm
3. **Privacy**: Transaction graph analysis does not reveal sender, recipient, or amount
4. **Quantum resistance**: Long-term data remains protected against quantum attacks

9.2 Privacy Analysis

9.2.1 Recipient Unlinkability

Theorem 9.1 (Post-Quantum Recipient Privacy)**.** *Under the IND-CCA2 security of ML-KEM-768, an adversary with access to a quantum computer cannot link outputs to recipient addresses with probability better than negligible.*

Proof. Each output contains:

- ML-KEM ciphertext c encapsulating shared secret K
- One-time key $P = sG + B$ where $s = H_s(K \| \text{index})$

Without the recipient's secret key, the shared secret K is computationally indistinguishable from random (IND-CCA2 security holds against quantum adversaries for ML-KEM-768). Thus s is random, and P reveals nothing about the recipient's public key B. □

9.2.2 Sender Anonymity

Theorem 9.2 (Ring Signature Anonymity)**.** *Under the DLP assumption in the random oracle model, the probability that an adversary identifies the true signer among ring members is at most* $1/n + \mathsf{negl}(\lambda)$*, where* $n = 20$ *is the ring size.*

Caveat: This is classical security. A quantum adversary with access to a sufficiently large quantum computer could potentially solve DLP and identify signers. However:

- Sender identity has ephemeral value (see Section 4)
- Ring signature data is not uniquely attributable even with private key recovery
- Retroactive deanonymization provides limited advantage

9.2.3 Amount Confidentiality

Theorem 9.3 (Information-Theoretic Amount Hiding)**.** *Transaction amounts are unconditionally hidden: no computational power (including quantum computers) can determine amounts from Pedersen commitments alone.*

Proof. A Pedersen commitment $C = vH + rG$ with random blinding factor r is uniformly distributed over the curve, independent of v. For any commitment C and any amount v', there exists r' such that $C = v'H + r'G$. Without the discrete log of H base G, the commitment is perfectly hiding. □

9.2.4 Anonymity Set Analysis

The effective anonymity set depends on decoy selection quality. The reduction factors below are consistent with empirical traceability studies of ring-signature systems, notably Möser et al.'s analysis of Monero, which found that temporal and zero-decoy heuristics can sharply cut the effective ring size [42]:

Even with degradation, the effective anonymity set provides meaningful privacy protection for typical use cases.

9.3 Formal Privacy Framework

This section establishes formal definitions and analysis of Botho's privacy properties, following standard cryptographic privacy definitions.

Table 20: Anonymity degradation factors

Attack	Reduction	Mitigation
Timing analysis	$\sim$2–3$\times$	Age distribution matching
Output age heuristic	$\sim$1.5$\times$	Recent output bias
Cluster analysis	$\sim$1.2$\times$	Cluster similarity requirement
Repeated use	Cumulative	Transaction spacing
Effective ring size	$\sim$5–8	(from nominal 20)

9.3.1 Privacy Definitions

We formalize the privacy goals following the terminology of Pfitzmann and Hansen [54].

Definition 9.4 (Unlinkability)**.** Two items of interest (transactions, addresses, users) are *unlinkable* if, within a given anonymity set, the adversary cannot determine with probability better than random whether the items are related.

Formally, let X and Y be events. (X, Y) are ϵ-unlinkable if:

$$|\Pr[\text{Linked}(X, Y)|\text{View}_{\mathcal{A}}] - \Pr[\text{Linked}(X, Y)]| \leq \epsilon$$

where $\text{View}_{\mathcal{A}}$ is the adversary's view and ϵ is negligible.

Definition 9.5 (Untraceability)**.** A transaction is *untraceable* if, given the transaction, the adversary cannot determine its inputs (sender) or outputs (recipient) with probability better than uniform over the respective anonymity sets.

Formally, for ring $\mathcal{R} = \{P_0, \ldots, P_{n-1}\}$ with real index π:

$$\forall i \in [n] : \Pr[\text{Real} = i|\sigma, \mathcal{R}] = \frac{1}{n} + \mathsf{negl}(\lambda)$$

Definition 9.6 (Confidentiality)**.** An attribute (amount, memo, metadata) is *confidential* if the adversary gains no information about it from public transaction data.

Formally, for amount v hidden in commitment C:

$$H(\text{Amount}|C) = H(\text{Amount})$$

where H denotes Shannon entropy.

9.3.2 BothoPrivacy Properties

We now prove Bothosatisfies these definitions.

Theorem 9.7 (Transaction-Address Unlinkability)**.** *Outputs belonging to different subaddresses of the same master address are* (2^{-128})*-unlinkable without knowledge of the view key.*

Proof. Subaddresses are derived as $(C_i, D_i) = (A+\delta_i G, B+\delta_i G)$ where $\delta_i = H_s(\text{“SubAddr”}\|a\|i)$.

Without view key a, the adversary cannot compute δ_i for any i. The hash function output is computationally indistinguishable from random. Thus (C_i, D_i) and (C_j, D_j) for $i \neq j$ appear as independent random points. The reduction from hash collision gives $\epsilon \leq 2^{-128}$. □

Theorem 9.8 (Input-Output Unlinkability)**.** *Given a transaction with inputs* $\mathcal{I}$ *and outputs* $\mathcal{O}$*, the adversary cannot determine which output corresponds to change.*

Proof. All outputs use identical encoding:

- Same commitment structure ($C = vH + rG$)

- Same one-time key derivation (ML-KEM)
- Identical sizes (including KEM ciphertext)

No syntactic distinguisher exists. Semantic distinguishers (amount ranges, timing) are addressed via uniform output ordering and standardized amounts. □

9.3.3 Privacy Budget Analysis

Privacy degrades with usage. We model cumulative privacy loss as a "budget" that depletes over time.

Model. Let $\pi_0 = 1/n$ be the initial probability an adversary correctly identifies the real input (uniform over ring). After observing k transactions from the same cluster:

$$\pi_k = 1 - (1 - 1/n)^k \cdot \prod_{i=1}^{k}(1 - \text{leak}_i) \tag{54}$$

where leak_i represents information leakage from transaction i (timing, amount correlation, etc.).

Privacy half-life. The number of transactions $k_{1/2}$ until $\pi_k = 0.5$:

$$k_{1/2} \approx \frac{\ln 2}{\ln(1 + 1/n) + E[\text{leak}]} \tag{55}$$

For $n = 20$ and $E[\text{leak}] = 0.05$:

$$k_{1/2} \approx \frac{0.693}{0.049 + 0.05} \approx 7 \text{ transactions}$$

Implications:

- Users making many transactions should use churning (self-sends) periodically to reset privacy budget
- Long-term passive holders enjoy persistent privacy
- High-frequency users should consider privacy-time-preference tradeoff

9.3.4 Metadata Leakage Quantification

Beyond cryptographic privacy, transactions leak metadata:

Transaction size reveals input/output structure:

$$\text{Inputs} \approx \frac{\text{size} - \text{base} - \text{outputs} \cdot s_o}{s_i} \tag{56}$$

where $s_i \approx 700$ bytes/input, $s_o \approx 1,152$ bytes/output.

Timing reveals transaction patterns:

- Time-of-day distribution (timezone inference)
- Inter-transaction intervals (behavioral fingerprinting)
- Response to external events (market correlation)

Network layer reveals source information:

- IP address (mitigated by Dandelion++)
- First-seen node (geographic inference)

Quantification table:

Total information leakage: With mitigations, approximately 5–8 bits per transaction. Over 1000 transactions, this accumulates to 5–8 KB of potentially correlatable information.

Table 21: Metadata leakage channels

Metadata	Bits Leaked	Mitigation	Residual
Tx size	3–5	Fixed-size outputs	1–2
Timing (hour)	4	Random delay	1–2
Network source	10+	Dandelion++	0–2
Fee amount	2–4	Min fee policy	1

9.3.5 Information-Theoretic Bounds

We compare Botho's achieved privacy to theoretical limits.

Optimal unlinkability requires entropy equal to the anonymity set:

$$H_{\text{opt}} = \log_2(n) = \log_2(20) \approx 4.32 \text{ bits} \tag{57}$$

Achieved unlinkability accounting for degradation:

$$H_{\text{achieved}} = \log_2(n_{\text{effective}}) \approx \log_2(6) \approx 2.58 \text{ bits} \tag{58}$$

Privacy efficiency:

$$\eta = \frac{H_{\text{achieved}}}{H_{\text{opt}}} = \frac{2.58}{4.32} \approx 60\% \tag{59}$$

This efficiency reflects the cost of practical decoy selection constraints (age distribution matching, cluster similarity).

Comparison to alternatives:

Table 22: Privacy efficiency comparison

System	Nominal Set	Effective	Efficiency
Botho	20	∼6	60%
Monero	16	∼4	50%
Zcash (shielded)	∞	Large	∼95%
MimbleWimble	Variable	∼1	Low

Notes: Zcash achieves higher efficiency but with computational cost and opt-in privacy reducing practical anonymity sets. MimbleWimble's cut-through provides limited effective privacy due to interaction requirements and transaction graph attacks [20].

9.4 Consensus Security

9.4.1 Double-Spend Resistance

Theorem 9.9 (Double-Spend Prevention)**.** *Under SCP's safety guarantees and assuming quorum intersection, no valid double-spend can be confirmed.*

Proof. We prove by strong induction on block height that no output can be spent twice.

Base case ($h = 0$): The genesis block contains no transactions, hence no double-spends.

Inductive hypothesis: Assume for all heights $h' < h$, no double-spend has been confirmed.

Inductive step: Consider height h. Suppose transactions T_1 and T_2 both attempt to spend output O, producing key image I.

Case 1: T_1 and T_2 are in the same block B_h.

- Block validation requires all key images in B_h to be distinct
- T_1 and T_2 share key image I

- B_h fails validation and is rejected

Case 2: T_1 is in block B_{h_1} and T_2 is in block B_{h_2} where $h_1 < h_2 \leq h$.

- By inductive hypothesis, T_1 is validly included at h_1
- Key image I is added to the key image set $\mathcal{I}_{h_1}$
- For all $h' > h_1$: $\mathcal{I}_{h'} \supseteq \mathcal{I}_{h_1}$
- Block B_{h_2} validation checks $I \notin \mathcal{I}_{h_2-1}$
- Since $I \in \mathcal{I}_{h_1} \subseteq \mathcal{I}_{h_2-1}$, check fails
- B_{h_2} (containing T_2) is rejected

Case 3: T_1 and T_2 are in competing blocks at the same height.

- SCP safety guarantees: if quorum intersection holds, only one block can be externalized per slot
- Let B_h be the unique externalized block at height h
- At most one of T_1, T_2 can be in B_h
- The other is never confirmed

By induction, double-spending is impossible at any height. □

Corollary 9.10 (Key Image Uniqueness)**.** *Each key image appears in at most one confirmed transaction.*

Proof. Immediate from the double-spend prevention theorem: the key image set $\mathcal{I}_h$ at height h contains exactly the key images from all confirmed transactions in blocks $0, \ldots, h-1$, with no duplicates. □

9.4.2 Byzantine Fault Tolerance

The system tolerates Byzantine nodes within quorum thresholds:

Theorem 9.11 (Byzantine Resilience)**.** *If quorum intersection holds and each quorum can tolerate its failure threshold, honest nodes agree on the same externalized values.*

With the default tiered structure (3-of-4 infrastructure + 2-of-3 community), the system tolerates:

- 1 Byzantine infrastructure node, OR
- 1 Byzantine community node, OR
- Combinations below both thresholds

9.4.3 51% Attack Resistance

Unlike pure PoW, Bothois resistant to hashpower-majority attacks:

- **Block proposal**: Majority hashpower can propose blocks more frequently
- **Finalization**: SCP requires quorum agreement regardless of hashpower
- **Result**: Attacker can flood proposals but cannot force finalization

An attacker would need to compromise both hashpower majority AND sufficient quorum members.

9.5 Cryptographic Security

9.5.1 Hash Function Security

Bothouses SHA3-256 [44] and BLAKE3 [50] for hashing:

- Collision resistance: 2^{128} security (quantum: 2^{85})
- Preimage resistance: 2^{256} security (quantum: 2^{128})
- Sufficient for all security requirements

9.5.2 Signature Security

Table 23: Signature security levels

Algorithm	Classical	Quantum	Use
CLSAG	128-bit	0	Ring signatures
PoW preimage binding	256-bit	128-bit (Grover)	Minting attribution

9.5.3 Key Encapsulation Security

ML-KEM-768 provides:

- IND-CCA2 security against quantum adversaries
- 192-bit classical security / 128-bit quantum security
- Based on hardness of MLWE problem

9.6 Attack Resistance

9.6.1 Timing Attacks

Transaction timing:

- Dandelion++ randomizes propagation timing
- Batching obscures submission time
- Mempool privacy prevents timing correlation

Decoy selection:

- Age distribution matches empirical spend patterns
- Randomization within constraints
- No deterministic selection

9.6.2 Transaction Graph Analysis

Several features resist graph analysis:

- **Ring signatures**: True input hidden among 20 possibilities
- **Stealth addresses**: Each output has unique one-time key
- **Hidden amounts**: Cannot trace by amount matching
- **Multiple outputs**: Change output indistinguishable

9.6.3 Sybil Attacks

Cluster-based progressive fees resist Sybil attacks:

- Splitting coins preserves cluster ancestry
- Creating new identities doesn't reduce fees
- Only genuine economic activity changes cluster factor

9.6.4 Denial of Service

Network-level:

- Rate limiting per peer
- Reputation scoring
- Connection limits
- Resource bounds

Consensus-level:

- PoW requires resources to propose blocks
- Invalid proposals rejected before propagation
- SCP messages bounded by quorum size

Transaction-level:

- Minimum fee requirement
- Validation before relay
- Mempool size limits

9.6.5 Eclipse Attacks

Eclipse attacks monopolize a victim's peer connections to control its view of the network [29].
Mitigation:

- Diverse peer selection (multiple ASNs, countries)
- Outbound connection preference
- Bootstrap node diversity
- Detection of peer manipulation

9.6.6 Selfish Mining

Botho's hybrid consensus changes the selfish-mining dynamics first analyzed for longest-chain PoW [17]:

- Block withholding delays finalization but doesn't create advantage
- SCP selects among available proposals, not necessarily first
- No "longest chain" to game
- Withholding risks losing the block entirely

9.7 Formal Security Properties

9.7.1 Safety

Property 9.12 (Value Conservation). For any valid block, the sum of all transaction outputs plus fees equals the sum of all inputs plus block reward.

Property 9.13 (No Inflation). The total supply at height h is exactly the sum of all block rewards up to h, minus all burned fees.

Property 9.14 (Key Image Uniqueness). Each output can be spent exactly once; the key image uniquely identifies the spent output.

9.7.2 Liveness

Property 9.15 (Transaction Inclusion). A valid transaction with sufficient fee will be included in a block within bounded time, assuming network synchrony and honest quorum majority.

Property 9.16 (Consensus Progress). If the network is eventually synchronous and quorum intersection holds, SCP will eventually externalize a value for each slot.

9.8 Security Comparison

Table 24: Security comparison with other privacy coins

Property	**Botho**	**Monero**	**Zcash**	**Grin**
PQ recipient privacy	✓	—	—	—
Ring size	20	16	N/A	N/A
Amount hiding	IT-secure	IT-secure	IT-secure	IT-secure
Mandatory privacy	✓	✓	—	✓
Deterministic finality	✓	—	—	—
Trusted setup	—	—	✓	—

(IT-secure = information-theoretically secure)

9.9 Detailed Attack Scenarios

This section provides worked examples of common attacks and their outcomes against Botho.

9.9.1 Scenario: Timing-Based Deanonymization

Setup: An adversary operates nodes connected to a significant fraction of the network (e.g., 30% of peers).

Attack vector:

1. Monitor transaction first-seen times across controlled nodes
2. Correlate timing with known user activity patterns
3. Attempt to identify transaction origin

Defense mechanisms:

1. **Dandelion++ stem phase**: Transaction travels through 1–10 intermediary nodes before broadcast
2. **Random delays**: Each hop adds 0–5 second random delay

3. **Stem length**: Geometric distribution with $p = 0.1$ (expected length 10 hops)

Analysis:

$$P(\text{origin identified}) < P(\text{first node observed}) \times P(\text{that node is origin})$$

$$= 0.3 \times 0.1 = 0.03$$

With 3% probability per transaction, an adversary would need to observe many transactions from the same user to achieve statistical confidence.

Additional mitigation: Users can further protect themselves by submitting transactions via Tor.

9.9.2 Scenario: Chain Analysis Attack

Setup: A chain analysis company maintains a database of "poisoned" outputs (outputs they created and can identify when spent). This is the zero-decoy / poisoned-output attack characterized empirically against Monero [42].

Attack vector:

1. Create many small outputs across many addresses
2. When outputs appear as decoys in other transactions, those are eliminated as possible real inputs
3. Remaining ring members are more likely to be real

Defense mechanisms:

1. **Ring size 20**: Even eliminating known decoys leaves significant anonymity
2. **Decoy selection algorithm**: Prefers outputs with similar cluster tags (70% similarity threshold)
3. **Output distribution**: Natural distribution of outputs dilutes poisoned set

Analysis: Suppose adversary controls 10% of all outputs (unrealistically high). For a ring of 20:

$$E[\text{poisoned decoys}] = 19 \times 0.1 = 1.9$$

Expected effective ring size ≈ 18, still providing substantial anonymity. In practice, controlling even 1% of outputs requires enormous capital investment.

9.9.3 Scenario: Flood-and-Loot Attack

Classic attack [28] (against pure PoW):

1. Deposit funds at exchange
2. Trade for another asset and withdraw
3. Use hashpower to reorg chain and double-spend deposit

Why it fails against Botho:

1. Exchange waits for transaction to be externalized (SCP finalized)
2. Externalization requires quorum agreement, not just PoW

3. Reverting requires corrupting quorum intersection
4. Even with majority hashpower, attacker cannot force quorum to finalize different block

Cost analysis: To execute this attack, adversary would need:

- >50% hashpower (cost: millions in hardware + electricity)
- Corrupt 2+ infrastructure nodes AND 2+ community nodes
- All this for a single double-spend opportunity

Conclusion: Attack cost far exceeds any reasonable double-spend profit.

9.9.4 Scenario: Lottery Grinding Attack

Hypothesis: Can a miner manipulate block construction to direct lottery rewards to their own UTXOs?

Attack vector:

1. Control minting of blocks
2. Modify transaction ordering/selection to change the randomness seed
3. Bias the lottery toward controlled UTXOs

Defense mechanism: Since protocol 7.0.0 the lottery seed is derived from the parent block's proof-of-work fields (`lottery_seed_v2`, `botho/src/consensus/lottery.rs`; issue #1676):

$$\text{seed}_h = \text{SHA256}(\text{“BOTHO_LOTTERY_SEED_V2”} \,\|\, \text{pow_hash}_{h-1} \,\|\, \text{prev_block_hash}_{h-1} \,\|\, h),$$

where pow_hash_{h-1} is the parent's RandomX output over nonce ‖ prev ‖ minter keys (`BlockHeader::pow_hash`, `botho/src/block.rs`).

Analysis:

- The seed does not depend on block h's transactions or ordering, so the proposer of block h cannot bias the draw through its own body.
- The parent's header hash is not an input. It covers the timestamp and the coinbase output, which a minter can change without new proof of work; binding only PoW-preimage fields means every change of seed costs a fresh PoW solution.
- **Candidate-set channel (open).** Block h's candidates are read from the state after block $h-1$, and pow_hash_{h-1} does not commit to $h-1$'s transactions. Once $h-1$'s PoW is solved and the seed is known, whoever assembles $h-1$'s body can choose which eligible outputs to spend in it, changing block h's candidate list and winners. Each alternative costs about one transaction fee, not a PoW solution. Whether this is feasible within SCP nomination timing is unverified (as built, pending #1689).

Cost-benefit for PoW regrinding: A miner that discards a valid solution to try for a different seed forfeits that block's coinbase. The coinbase is the reward minus the lottery emission share, so the forfeit is $(1 - f_{\text{em}})\, R(h)$, where f_{em} is 0 in the first halving epoch and rises to a 50% cap (`miner_payout`, `botho/src/block.rs`). The payout the new seed can redirect is at most one block reward:

$$\text{grinding_cost} \;\approx\; (1 - f_{\text{em}})\, R(h), \qquad \Delta_{\text{payout}} \;\le\; R(h) \times P(\text{controlled UTXO wins}).$$

In the first epoch ($f_{\text{em}} = 0$) the forfeit is a full reward and exceeds any redirectable payout. As f_{em} rises to 50%, the forfeit falls to half a reward while the redirectable payout stays up to one reward, so the bound no longer holds by construction for a grinder whose controlled outputs hold a large share of the eligible tickets. Ticket share is cheap to acquire while the splitting cost premises are unmet (#1682), so the grinding bound depends on resolving that issue as well.

9.9.5 Scenario: Eclipse Attack on Light Client

Setup: Adversary controls all connections to a light client.
Attack vector:

1. Feed client fabricated block headers
2. Show non-existent or double-spent transactions as confirmed
3. Trick user into accepting invalid payment

Defense mechanisms:

1. **Multiple connections**: Light clients connect to 8+ diverse peers
2. **PoW verification**: Fake headers require real PoW
3. **SCP proofs**: Externalization proofs verify quorum agreement
4. **Checkpoint verification**: Known checkpoints prevent deep reorgs

Attack cost: To eclipse a light client:

- Must control all 8+ outbound connections
- Must produce valid PoW for fake headers
- Must forge SCP externalization proofs (requires quorum keys)

Practical impossibility: Forging SCP proofs requires private keys of quorum members. Without these, even a fully-eclipsed client will reject blocks that lack valid externalization.

9.9.6 Scenario: Progressive Fee Evasion

Goal: Large holder wants to pay base fees despite high cluster factor.
Attempted strategies:

1. **Splitting**: Divide holdings into many small UTXOs
   - *Result*: All child UTXOs inherit same cluster tag
   - *Outcome*: No fee reduction
2. **Multiple addresses**: Create many addresses
   - *Result*: Cluster factor based on coin ancestry, not address
   - *Outcome*: No fee reduction
3. **Wash trading**: Rapid self-transfers to decay tags
   - *Result*: Consensus applies no decay and does not rate-limit it; it rejects only tag *inflation* (`CONSENSUS_TAG_DECAY_RATE` = 0, `botho/src/ledger/store.rs`)
   - *Outcome*: Stock wallets decay 5% per transfer with no age gate (half-life ≈13.5 transfers); a modified wallet can write empty tags in one transaction (as built, pending #1687)
   - *Cost*: Transaction fees plus the class-downgrade charge, which is zero in the first halving epoch (pending #1685) and can be diluted by decoy choice (pending #1686)
4. **Exchange laundering**: Deposit to exchange, withdraw different coins

- *Result*: Works, but requires real economic activity
- *Outcome*: Legitimate use case (this is the intended mechanism for tag blending)

Conclusion: Splitting and new addresses provide no fee benefit. Splitting does buy lottery tickets, which is currently profitable (#1761, pending #1682; Section 10.6.2). Wash trading is bounded only by the class-downgrade charge, not by a decay rate limit; whether consensus should guarantee tag persistence is open (#1687).

9.10 Known Limitations

9.10.1 Classical Ring Signature Vulnerability

A sufficiently powerful quantum computer could break CLSAG and identify signers retroactively. Mitigation:

- Ring signature data alone doesn't prove spending
- Economic value of retroactive deanonymization is limited
- Future protocol upgrade to post-quantum rings is possible

9.10.2 Metadata Leakage

Despite cryptographic privacy:

- Transaction size reveals input/output count
- Timing correlation possible with powerful adversary
- Network-level deanonymization without Tor/I2P

9.10.3 Implementation Risks

- Side-channel attacks on key material
- Random number generator quality
- Memory safety issues
- Timing side channels in cryptographic operations

All reference implementation cryptographic code uses constant-time algorithms and is subject to ongoing security audit.

10 Economic Analysis

This section analyzes the game-theoretic properties of Botho's economic mechanisms and their long-term sustainability.

10.1 Incentive Alignment

10.1.1 Miner Incentives

Miners (block proposers) are incentivized to:

1. **Include transactions**: Larger blocks have no disadvantage (fees are burned, not captured)
2. **Maintain network health**: Block rewards depend on network value, not transaction ordering
3. **Stay honest**: Invalid blocks are rejected by SCP

Why fee burning works:
Without fee capture, miners have no incentive to:

- Reorder transactions (MEV elimination)
- Censor low-fee transactions beyond minimum
- Create artificial congestion

10.1.2 User Incentives

Users are incentivized to:

1. **Circulate rather than hoard**: Progressive fees and tail emission create holding costs
2. **Maintain privacy**: Default privacy eliminates coordination problems
3. **Run nodes**: Reduced resource requirements enable participation

10.1.3 Validator Incentives

SCP validators (consensus participants) are incentivized by:

1. **Reputation**: Being in quorum slices requires trustworthiness
2. **Self-interest**: Validators typically also hold currency
3. **Community standing**: Non-economic incentives matter for infrastructure nodes

10.2 Progressive Fee Analysis

10.2.1 Fee Distribution

Under the cluster-based progressive fee system:

$$E[\text{fee}] = f_{\text{base}} \times \text{size} \times E[\text{cluster_factor}] \tag{60}$$

The expected cluster factor depends on wealth distribution:

Fees are the size term of the congestion-floor consensus minimum (1 pico-BTH/byte × 4,096 bytes × factor), before the output penalty and demurrage; 4,000 *pico-BTH* $= 4 \times 10^{-9}$ *BTH. Under congestion the dynamic base multiplies these by up to* 100×*. Default nodes also require the flat relay minimum* `MIN_TX_FEE` $= 10^8$ *pico-BTH (*`transaction/clsag/src/lib.rs`*), which exceeds every row, so at today's constants an ordinary wallet pays the same fee at every percentile (pending #1688; the relay minimum is not a consensus rule, #1684).*

Table 25: Expected fees by wealth percentile

Percentile	Factor	4KB Tx Fee (floor)
0–50	1.0–1.5	4,000–6,000 pico
50–90	1.5–3.0	6,000–12,000 pico
90–99	3.0–5.0	12,000–20,000 pico
99–100	5.0–6.0	20,000–24,000 pico

10.2.2 Redistribution Flows

The lottery routes fees from fee payers to the holders of randomly drawn outputs:

$$\text{redistribution}_{\text{direct}} = 0.8 \times \sum_{\text{tx}} f_{\text{total}} \tag{61}$$

Fee burning (20%) reduces supply, which benefits all holders in proportion to their holdings and so does not by itself change the distribution:

$$\text{redistribution}_{\text{indirect}} = 0.2 \times \sum_{\text{tx}} f_{\text{total}} \tag{62}$$

These are flows, not outcomes. Whether they reduce inequality depends on who pays and who wins; at production parameters they do not measurably do so (Section 10.6.2).

Lottery selection as built:

- Winners are drawn uniformly among eligible outputs (`SelectionMode::Uniform`, `cluster-tax/src/lottery.rs`); every eligible output is one ticket, regardless of value or cluster factor (Section 7)
- A uniform per-output draw depends on UTXO count, which a holder controls. Its split resistance rests on the Path C reward cap, whose premises the implementation does not meet, so splitting is profitable (#1761, pending #1682); coinbase and payout outputs are also free tickets (pending #1690)
- Any progressivity therefore has to come from the intake side (demurrage, and the factor-scaled fee where it binds), not from the payout weight; at production parameters that intake does not produce a measurable reduction in inequality (Section 10.6.2)

10.2.3 Anti-Hoarding Dynamics

The combination of

- Tail emission (∼2% annual supply increase)
- Progressive fees (wealth-based costs)
- Lottery redistribution (80% of fees to random UTXOs)
- Fee burning (20% deflationary pressure)

is designed to create net pressure toward circulation:

$$\text{holding_cost} = \text{dilution} - \text{lottery_income} + \text{opportunity_cost} \tag{63}$$

Lottery income is per eligible output, not tilted by cluster factor (as built, Section 7). Splitting does not change cluster attribution, but shedding attribution is not rate-bounded by consensus: tags can be dropped at the cost of the class-downgrade charge (as built, pending #1687, #1686, #1685; Section 5.4.4).

Intuition: why a holding cost, and why a targeted one. A fixed-supply currency in the Bitcoin mold makes hoarding costless, indeed rewarding: a coin left untouched keeps its full claim on a supply that will never grow, so the rational move for the largest holders is to sit still and let scarcity appreciate their stake. That is good for a store of value but poor for a medium of exchange, and it concentrates wealth over time because those best able to wait accrue the appreciation. A pure tail-emission design (perpetual issuance, no offsetting drag) softens the deflation but does not change the incentive: new coins accrue to whoever mines, and idle wealth is diluted only uniformly, which large holders can hedge simply by holding a proportional share of hashpower. In neither design does sitting on wealth cost the holder anything relative to their peers.

Botho's anti-hoarding design makes idle wealth carry a small, *cluster-scaled* holding cost while leaving circulating money alone. The intent is that the holding_cost term above is a net drag for the high-factor clusters that dominate the supply and positive-leaning for ordinary factor-1 balances, which would recover much of their dilution through lottery income: a continuous incentive to *spend* rather than accumulate, without penalizing the everyday user the way classical uniform demurrage does. At production parameters that targeting does not hold: lottery income tracks block production and output count rather than holdings, and decayed or coinbase-lineage tags price nearly all value at factor 1 within a decade (Section 10.6.2). Historical demurrage currencies (Freigeld, the Chiemgauer) taxed every balance equally and were rejected as user-hostile for exactly this reason (Section 2); Botho's contribution is to aim the same circulation incentive only where concentration actually occurs, using the cluster factor as the targeting signal (Section 10.6).

10.3 Game-Theoretic Analysis

10.3.1 Miner Strategy

Proposition: Honest mining is a Nash equilibrium.

Proof sketch. Consider miner M with hashpower fraction α:

- Honest strategy: Expected reward $\alpha \times R$ per block
- Withholding: Risk of block rejection by SCP (no benefit)
- Invalid blocks: Rejected, wasted computation
- Transaction censorship: No benefit (fees burned anyway)

No deviation improves expected payoff. □

10.3.2 Validator Strategy

Proposition: Honest validation is incentive-compatible for economically invested validators.

Proof sketch. Validators typically hold currency. Byzantine behavior risks:

- Network failure (total loss of holdings)
- Exclusion from quorum slices (loss of influence)
- Reputation damage (non-economic but real)

The expected loss exceeds any potential gain from misbehavior. □

10.3.3 User Strategy

Proposition: Using the system as designed is individually rational.

Users face choices:

- **Privacy**: Mandatory, so no choice to make
- **Fee payment**: Required for transaction inclusion
- **Holding vs. spending**: Mild pressure toward spending, but not coercive

10.4 Long-Term Sustainability

10.4.1 Security Budget

The annual security budget (miner revenue) is:

$$\text{budget} = R_{\text{tail}} \times \text{blocks_per_year} \times \text{BTH_price} \tag{64}$$

With tail emission, this is guaranteed to be positive regardless of transaction volume. This directly addresses the fee-only security concern: fee-funded security is subject to instability and potential underinvestment [7, 32], whereas a perpetual emission floor decouples the security budget from fee-market volatility.

10.4.2 Network Effect

Privacy creates positive network effects:

- More users → larger anonymity sets
- Larger anonymity sets → better privacy
- Better privacy → more users

This creates a virtuous cycle encouraging adoption.

10.4.3 Fee Market Stability

Fee burning eliminates fee market volatility:

- No bidding wars for block inclusion
- Predictable costs for users
- No miner incentive to manipulate fees

The minimum fee provides sufficient spam resistance while remaining affordable.

10.5 Comparison with Alternatives

10.5.1 Bitcoin Model

10.5.2 Monero Model

10.6 Quantitative Economic Modeling

This section presents formal modeling and simulation results for Botho's economic mechanisms.

Table 26: Bitcoin vs Botho economic comparison

Property	Bitcoin	Botho
Supply cap	21M	None (tail)
Long-term inflation	0%	∼2%
Security funding	Fees only	Emission + burn
Fee predictability	Low	High
Wealth concentration	High	Reduction not demonstrated

Table 27: Monero vs Botho economic comparison

Property	Monero	Botho
Tail emission	Yes	Yes
Fee destination	Miners	Burned
Progressive fees	No	Yes
Dynamic timing	No	Yes

10.6.1 Adversarial Redistribution Simulation

An earlier agent-based simulation evaluated candidate redistribution designs. It does not model the implemented mechanism and does not demonstrate it (Section 10.6.2). Every candidate mechanism was tested twice: once with honest participants, and once against a *strategic whale* that splits its position into 1,000 UTXOs and churns them periodically to maintain lottery eligibility — the optimal gaming strategy known at the time. A mechanism's redistributive effect was only credited in the *gamed* equilibrium.

Setup: 100M BTH economy; 80 small holders (5% of wealth, cluster factor 1×), 30 mid holders (25%, 2×), 9 honest whales (65%, 6×), 1 strategic whale (5%, 6×); initial Gini $G_0 = 0.77$. The operating point routes 2.5%/yr of supply from tail emission into the lottery pool and applies 2%/yr demurrage at factor 6, with the split-invariant 6:1 payout tilt. Design criterion: $\Delta G > 0.05$ versus the no-redistribution baseline. Runs are deterministic given an RNG seed (the simulation forks a `ChaCha20` stream over order-stable `BTreeMap` state, so a seed fixes the run byte-for-byte). To show the headline result is not an artefact of one lucky draw, the winning mechanism was re-run across an eight-seed set — `{2953379877, 626626626, 1, 42, 12345, 271828183, 314159265, 161803398}` — and the bold row below reports the *mean ± 95% confidence interval* (Student-t, $n = 8$) of the five-year ΔG against the no-redistribution baseline. Each seed is reproducible via `cluster-tax-sim lottery-experiment --blocks 7884000 --seed <seed>`; the other (comparison-only) rows use the first seed.

These levers are part of the specified protocol, not analysis-only add-ons. Emission-routing is the height-scheduled emission share paid into the lottery pool (Section 7, ramping to a 50% cap), and demurrage is the cluster-scaled stock-level term on the consensus fee floor (Section 5.4). The payout lever is not: the "cluster-tilted" rows below simulate the retired value-weighted, tilted draw, while the protocol draws winners uniformly (`SelectionMode::Uniform`, `cluster-tax/src/lottery.rs`). The table is therefore an earlier design's result, not a measurement of the implemented mechanism. The row closest to the implemented draw is "Uniform per UTXO + emission", which inverts under gaming; the Path C reward cap was meant to prevent that inversion, and as built it does not bind (#1682). The code-faithful re-runs (#1696 for M2 at production scale, #1761 for splitting under the implemented cap) are reported in Section 10.6.2.

Three structural results from that model:

1. **Per-UTXO payouts invert under gaming.** The uniform lottery is the strongest redistributor in the honest world and the worst mechanism overall in the gamed one: the strategic whale captures the payout stream, growing from 5% to 24% of supply in five

Table 28: Five-year ΔG vs. the no-redistribution baseline, honest and gamed. The bold winning row's honest figure is the mean ± 95% CI over the eight-seed set ($n = 8$; $1\sigma = 0.0003$); every other cell is the first-seed (`2953379877`) value.

Payout mechanism	**Honest**	**Gamed**
Uniform per UTXO + emission	+0.176	**−0.026**
Value-weighted + emission	−0.001	−0.001
Cluster demurrage only	+0.008	+0.009
Cluster-tilted + emission	+0.054	+0.055
Cluster-tilted + emission + demurrage	**+0.078 ± 0.0002**	+0.076

years while the Gini coefficient *rises*. UTXO count is a free variable; nothing may be paid on it.

2. **Emission with proportional payout is exactly Gini-neutral**, confirming that dilution redistributed in proportion to wealth changes nothing — the payout must be tilted.

3. **Cluster-anchored mechanisms are split-invariant.** Under the cluster-tilted lottery the gamed run shows marginally *more* redistribution than the honest run: the attacker's splitting and churn fees feed the lottery pool, while costing the attacker ∼19% of its position over five years. Attacking the mechanism funds it.

Expected lottery income as built. Under the uniform draw, a holder with k of the ρ eligible outputs in the candidate set expects, per block of payout P,

$$E[\text{income}] \approx P \cdot \frac{k}{\rho}, \tag{65}$$

independent of value and cluster factor. This depends on how a position is divided across outputs, which is why the splitting premises (#1682) matter. The tilted formula of earlier drafts, $P \cdot v(\phi_{\max} - \phi + 1) / \sum_j v_j(\phi_{\max} - \phi_j + 1)$, describes the retired `ClusterWeighted` mode only.

What the earlier model reported. In that model, at its parameters, the retired cluster-tilted design with emission routing and demurrage lowered the Gini coefficient by 0.078 ± 0.0002 per five years (ΔG vs. the no-redistribution baseline; mean ± 95% CI over the eight seeds), and by ≈ 0.097 in absolute terms ($0.77 \to 0.68$); the gamed run gave 0.076. The fee-only configuration (no emission routing, no demurrage) gave +0.02/5yr: transaction fees are a consumption tax and cannot by themselves touch idle wealth at realistic velocities. These figures describe a design the protocol does not implement and are kept only as a record of how the design was chosen. They are not a property of Botho.

10.6.2 Status of the Redistribution Claim

Measurable inequality reduction is not demonstrated at production parameters. This restatement (2026-11-01) follows the operator ruling on issues #1727 and #1682. It changes no constant and no consensus rule.

What the protocol does, as specified. The fee floor scales with a coin's cluster factor (1×–6×). Cluster-scaled demurrage charges high-factor coins in proportion to value and holding time, from the second halving epoch on; factor-1 coins are exempt. The lottery pool receives 80% of fees (demurrage included) and a height-scheduled emission share, and pays four outputs per block drawn uniformly from the outputs aged 720 to 10,000 blocks. Ring signatures hide which output a spend consumed, so a spent output stays a candidate until it ages out (issue

#1808). Cluster tags record each output's provenance; each coinbase starts a fresh cluster. These descriptions stand.

What is not demonstrated. A code-faithful re-run (issue #1696, PR #1790, `cluster-tax-sim m2-production`) uses the node's fee floor, 5 s blocks, the mainnet emission and lottery-share ramp, the uniform draw with payouts as new outputs, coinbase-lineage clusters and an optimal-decoy mining whale, and compares each run against the same seed with the mechanism switched off. The mechanism's own ΔG is between -0.028 and $+0.003$ (three-seed means at 10 and 20 years) against the 0.05 design bar, both under the record's original candidate rule (a spend removes its outputs from the draw) and under the node's, in which spent outputs stay eligible (issue #1808). Even at 1000× the transaction rate no case clears the bar at both 10 and 20 years: the largest value, $+0.073$, is at 20 years only, under the node's rule and wealth-proportional mining. By year 10, 100% of top-decile value is priced below 1.5×. The measured failure mechanisms are structural:

- lottery tickets track hashpower and output count, because most eligible outputs are coinbases and payouts that copy their winner;
- spent outputs stay candidates until they age out, so tickets follow output creation rather than current holdings, and re-spending the same capital adds tickets;
- tag decay and coinbase-lineage clusters bring nearly all value to factor 1, so demurrage stops, and at high transaction density it disappears altogether;
- per-output splitting buys tickets (below).

Under egalitarian mining the mechanism is mildly regressive on average. The record is `docs/research/m2-production-verdicts.md`.

Lottery splitting is profitable. A faithful-to-code re-run of the splitting experiment (PR #1761, `lottery-reward-cap-sweep --faithful`) finds splitting profitable at all 192 grid points: an extra output costs far less than the ticket share it buys, and the Path C reward cap does not bind as built (issue #1682). The record is `docs/research/ct-compatible-lottery-selection.md`, §9.1.1.

What happens next. A sim-only redesign study (issue #1801) targets the failure mechanisms above. Candidate directions include tickets weighted by value, age or cluster factor instead of output count, and any candidate must pass the production-fidelity harness of #1790 and the splitting grid of #1761 before a constant or consensus rule moves. Raising the existing levers (demurrage rate, lottery share, per-transaction floor) was considered and rejected, because the failure is structural. Until the study concludes, the redistribution mechanism should be read as specified and implemented, but not as a demonstrated means of reducing inequality.

10.6.3 Formal Nash Equilibrium Analysis

We prove that honest participation constitutes a Nash equilibrium under reasonable assumptions.

Definition 10.1 (Miner Game)**.** The miner game $\Gamma_M = (N, S, u)$ consists of:

- Players $N = \{1, \ldots, m\}$ (miners with hashpower α_i)
- Strategy space $S_i = \{\text{honest}, \text{deviate}\}$
- Payoff $u_i(s) = R \cdot \alpha_i \cdot \mathbb{1}[\text{block accepted}]$

Theorem 10.2 (Miner Nash Equilibrium)**.** *In Γ_M, the strategy profile where all miners play honest is a Nash equilibrium for any $\alpha_i < 0.5$.*

Proof. Consider miner i with hashpower α_i. Under honest strategy:

$$u_i(\text{honest}, s^*_{-i}) = R \cdot \alpha_i \cdot P(\text{valid}) = R \cdot \alpha_i$$

Under deviation (invalid block, withholding, or censorship):

1. **Invalid block**: SCP rejects with probability 1. $u_i(\text{invalid}) = 0 < R \cdot \alpha_i$.
2. **Block withholding**: Reduces probability of own block selection. $u_i(\text{withhold}) < R \cdot \alpha_i$.
3. **Transaction censorship**: No payoff change (fees burned). $u_i(\text{censor}) = R \cdot \alpha_i = u_i(\text{honest})$.

No deviation strictly improves payoff. Censorship is weakly dominated due to reputation costs not modeled here. □

Definition 10.3 (Validator Coalition Game)**.** The coalition game $\Gamma_V = (N, v)$ consists of:

- Validators N with quorum slice structure $\mathcal{Q}$
- Characteristic function $v : 2^N \to \mathbb{R}$ where $v(S)$ is the value coalition S can guarantee

Theorem 10.4 (Coalition Resistance)**.** *Under quorum intersection, no coalition S with $|S| < f_{threshold}$ can guarantee positive deviation payoff.*

Proof sketch. A deviating coalition must either:

1. Block consensus (requires breaking quorum intersection)
2. Force different values (requires $> f$ Byzantine nodes)

Both require coalition size exceeding Byzantine threshold. Below threshold, $v(S) \leq 0$ for any deviating strategy. □

10.6.4 Economic Attack Cost Analysis

We quantify the cost of various economic attacks.

51% Attack Cost. Unlike pure PoW, Bothorequires both hashpower majority AND quorum corruption:

$$\text{Cost}_{51\%} = \text{Cost}_{\text{hashpower}} + \text{Cost}_{\text{quorum}} \tag{66}$$

For hashpower (assuming \$0.05/kWh electricity, efficient mining):

$$\text{Cost}_{\text{hashpower}} = \frac{0.51 \cdot H_{\text{network}} \cdot P_{\text{unit}}}{E_{\text{unit}}} \cdot t_{\text{attack}} \tag{67}$$

For quorum corruption (must compromise $\geq f + 1$ validators):

$$\text{Cost}_{\text{quorum}} \geq (f + 1) \cdot V_{\text{stake}} \cdot P_{\text{bribe}} \tag{68}$$

where P_{bribe} represents the fraction of stake validators require to defect.

Cluster Factor Manipulation. To reduce cluster factor from f to f':

$$\text{Cost}_{\text{cluster}} = \text{fee}(\text{mixing}) + \text{slippage} + \text{counterparty_cost} \tag{69}$$

Mixing requires acquiring low-cluster UTXOs through genuine trade. With efficient markets, arbitrage prevents free cluster factor reduction.

Lottery Grinding. The seed is bound to the parent block's proof of work (protocol 7.0.0, Section 9, Lottery Grinding), so each new seed costs a PoW solution. A miner that discards a solution forfeits its coinbase, $(1 - f_{\text{em}})\, R(h)$, not the full reward $R(h)$; with the lottery emission share f_{em} at its 50% cap, that is half a reward against a redirectable payout of up to one reward. The candidate set can also be changed without new PoW through the parent block's transaction set (as built, pending #1689).

10.7 Economic Risks

10.7.1 Low Adoption

If adoption remains low:

- Tail emission maintains miner incentive
- Progressive fees have minimal impact (few large holders)
- Network can persist indefinitely at low scale

10.7.2 Mining Centralization

Risks and mitigations:

- **Risk**: ASIC development concentrates mining
- **Mitigation**: CPU-friendly PoW (RandomX-based [62])
- **Risk**: Pool concentration
- **Mitigation**: Solo mining viable with tail emission

10.7.3 Validator Cartel

If validators collude:

- SCP's tiered structure provides defense
- Community can adjust quorum slices
- Economic self-interest opposes cartels

10.8 Economic Constants Summary

Table 29: Economic parameters

Parameter	Value	Rationale
Tail inflation	∼2%	Security sustainability
Max fee multiplier	6×	Balance incentive/burden
Ring size	20	Privacy/size tradeoff
Min block time	3s	Consensus floor (very high load)
Reference block time	5s	Monetary-calculation reference
Max block time	40s	Emission floor

11 The BTH ↔ wBTH Bridge

This section is the normative specification of the Botho cross-chain bridge: the protocol by which native BTH is locked and a wrapped representation, **wBTH**, is minted on Ethereum and Solana, and by which wBTH is burned to release the locked BTH. It is authored against the ratified bridge decision records (ADR 0002–0005, 0007) and is maintained truthfully against the bridge implementation (`bridge/**/*.rs` and the two token contracts); every normative claim below either holds of that implementation today or is marked target-state in the implementation-status register (§11.7). It builds on the transaction model (§5), the hybrid PoW+SCP consensus (§6), the monetary rules (§7), and the cluster-factor anti-hoarding economics (§10).

Conformance keywords. The key words **MUST**, **MUST NOT**, **SHALL**, **SHALL NOT**, **SHOULD**, and **MAY** in this section are to be interpreted as described in RFC 2119: they denote absolute requirements, prohibitions, recommendations, and optional behavior respectively. A conformant bridge implementation MUST satisfy every **MUST** / **SHALL** obligation stated below, except where a requirement is explicitly scoped *target-state* in the implementation-status register (§11.7), in which case the register's *Live* column states the obligation the current implementation MUST meet and the *Target* column states the obligation a future implementation MUST meet.

11.1 Motivation and Scope

Botho is a privacy-preserving, anti-hoarding monetary network. Its native asset does not natively reach the deep liquidity of the Ethereum and Solana DeFi ecosystems. The bridge exists to give holders access to that liquidity: a holder locks native BTH into a reserve and receives **wBTH**, a fully-transparent ERC-20 (Ethereum) or SPL (Solana) token redeemable one-for-one for the locked BTH.

v1 chain scope (ADR 0005). Bridge v1 ships *both* Ethereum and Solana. Every privileged operation—mint, release, watching, reserve accounting—is defined for both chains. The peg invariant is stated once and generalizes across chains:

$$\underbrace{\sum \text{wBTH on Ethereum} + \sum \text{wBTH on Solana}}_{\text{outstanding wrapped supply}} = \underbrace{\text{locked BTH reserve}}_{\text{custodied on Botho}}. \tag{70}$$

Solana is comparatively cheap on the custody axis because it uses Ed25519—the same key type as Botho node keys—so the validator federation (§11.2) signs Solana authorizations natively; only the Ethereum side requires a secondary key type.

What wBTH is. wBTH is denominated in **picocredits** (12 decimals): one wBTH base unit equals one picocredit equals one unit of native BTH, so the peg carries no scaling factor. The token is transparent on its host chain; moving it is publicly linkable there. This is a deliberate privacy boundary, treated normatively in §11.4.

Why a bridge is the highest-value target. Whoever controls the mint authority on a destination chain or the reserve-release authority on Botho can print or drain funds. The bridge's entire security posture (§11.2, §11.6) is organized around denying any single party that control. The custody, peg, privacy, and import-tagging designs in this section are the four load-bearing decisions that make a bridge safe to operate against a privacy-preserving, anti-hoarding base layer.

11.2 Custody and Trust Model

Botho has no EVM or SVM light client, so neither host chain can natively verify Botho's state, and Botho cannot natively verify a host chain's state. Every privileged cross-chain action MUST therefore be authorized by *signatures from a trusted signer set*, and MUST NOT rely on on-chain proof of the counterparty chain (ADR 0002). The normative decision is:

> **The bridge federation is the SCP validator set, operating as a t-of-n threshold signer group.** The mechanism is threshold signing on both sides; the signer identity is the existing consensus validators.

11.2.1 Threshold authorization

No privileged action fires without a valid set of at least t distinct federation signatures, domain-separated and bound to the exact order: a mint or release MUST NOT be authorized by fewer than t distinct signatures. The threshold floor is normatively constrained:

$$t \geq t_{\mathrm{SCP}}, \tag{71}$$

where t_{SCP} is the SCP safety threshold: *it MUST never be easier to move the reserve than to move consensus.* The distinct-signer requirement is exact—a repeated or equivocating signer MUST count once toward t (verified in `release/bth.rs::validate_release_attestation`).

11.2.2 Per-chain signature schemes

- **BTH reserve release** MUST be authorized by a t-of-n threshold of the validators' **Ed25519** node keys, reusing the operator-signed action envelope machinery (domain-separated payloads, reserve-then-apply nonce replay protection, audit log).
- **Solana wBTH mint** MUST be authorized by the same validators' **Ed25519** keys—Solana is Ed25519-native, so no new key type is required.
- **Ethereum wBTH mint** requires **secp256k1**, which SCP node keys are not. Each validator therefore also operates a secp256k1 signer, and the Ethereum mint authority MUST be a **Gnosis Safe** whose owners are those secp256k1 keys and which holds `MINTER_ROLE` on the wBTH contract.

11.2.3 Attestation envelopes and exactly-once semantics

Every authorization MUST be carried in a *domain-separated, order-bound, single-use signed envelope.* The domain tags are pairwise-distinct and are proven distinct from the node's operator-action domain, so no signature made for one purpose can authorize another (`bridge/core/src/attestation.rs`)

Purpose	Domain tag
Envelope (Ethereum / Solana / BTH)	`botho-bridge-attest-{eth,sol,bth}-v1`
Release payload	`botho-bridge-release-v1`
Mint payload (Ethereum / Solana)	`botho-bridge-mint-{eth,sol}-v1`

The bridge holds two exactly-once invariants:

1. **Exactly-once mint** — one confirmed BTH deposit MUST mint wBTH at most once. Each order carries a deterministic 32-byte on-chain `orderId`; the destination contract records it and MUST revert on a duplicate.
2. **Exactly-once release** — one confirmed wBTH burn MUST release reserve BTH at most once. The engine records the release claim and the signed transaction before broadcast, so a crash or re-broadcast reuses the recorded transaction rather than signing a second one.

11.2.4 The three-Safe structure on Ethereum

On Ethereum the contract's privileged roles MUST be split across *three distinct Gnosis Safes*, so no single Safe both mints and configures the breaker:

Role	Held by	Capability
`MINTER_ROLE`	Validator Safe (t-of-n secp256k1)	the only mint authority
`DEFAULT_ADMIN_ROLE`	Governance Safe	limits / breaker config, role admin
`PAUSER_ROLE`	Guardian Safe (may be lower-t)	pause / unpause

The deployer MUST receive no roles. The t-of-n enforcement lives *in the validator Safe*, not in the token contract: the contract only checks that the caller holds `MINTER_ROLE`. A single relayer EOA submits `Safe.execTransaction` carrying the collected threshold owner signatures over the EIP-712 `SafeTx` digest wrapping `bridgeMint(to, amount, orderId)`. On Solana the analogous authority is an SPL/Squads multisig of the validators' keys, and a startup custody guard MUST refuse to operate if the on-chain mint authority equals the local relayer key (a lone-hot-key configuration).

Figure 17 depicts the federation custody and attestation topology across both chains.

11.3 The Peg

The peg MUST hold *over time*, not merely at the instant of locking. The complication is demurrage. Botho charges a holding cost on wealthy-cluster coins at spend time (§10):

$$\text{charge} \;=\; \text{value} \times \text{rate} \times \frac{\text{factor} - 1}{\text{max_factor} - 1} \times \frac{\text{elapsed}}{\text{blocks_per_year}}, \tag{72}$$

where factor $\in [1\times, 6\times]$ is derived from the coin's cluster wealth. Reserve BTH that demurred while wrapped would shrink the reserve below the outstanding supply and silently make the bridge fractionally reserved.

11.3.1 Factor-1-only wrapping (ADR 0003)

Because of the (factor $-$ 1) term, a **factor-1** (background / commerce) coin pays exactly *zero* demurrage, permanently and independently of elapsed. The normative rule is therefore:

> **Only factor-1 coins are wrappable.** The reserve holds only zero-demurrage coins, so (70) holds over time by construction, with no "reserve class" exemption and no rebasing.

Wrap eligibility is checkable at deposit: an output carries its cluster-tag vector, so the bridge reads its factor directly and MUST mint only for factor-1 deposits (a non-factor-1 deposit MUST be rejected with an audit event and MUST never mint). Reciprocally, releases MUST spend *only* factor-1 / background reserve outputs, so a release can never launder cluster provenance and the reserve change stays zero-demurrage. Figure 18 shows the wrap flow.

11.3.2 The demurrage-settlement on-ramp (target-state)

A holder of a wealthy-cluster coin cannot wrap it directly. A **demurrage-settlement operation** lets them pay a charge to reclassify the coin down to factor-1 / background, after which it is wrappable; the settlement fee is routed to the redistribution lottery pool—the same sink demurrage charges already feed. The sanctioned way to shed cluster provenance is thus to *pay* for it, and that payment funds redistribution rather than escaping it. This is the one consensus-level piece of the bridge (new transaction semantics plus a validation carve-out in the cluster-tag inheritance bound). It is **target-state**, tracked in the register (§11.7, row **IMP-3**); its charge formula and the shared lineage-reset horizon are pending ratification (§11.5).

11.3.3 Post-wrap demurrage escape (accepted)

Once wrapped, coins sit in transparent wBTH and pay no further demurrage. This is treated as "settled on the way in": a holder MAY settle once, wrap, and remain in wBTH demurrage-free. This is a deliberate economic stance, accepted so that wBTH is a clean 1:1 DeFi asset.

11.4 Privacy at the Boundary

Botho is privacy-by-default: senders are hidden in CLSAG rings of size 20, and recipients receive to one-time stealth addresses. Confidential amounts (amounts hidden in Pedersen commitments) are a *base-layer* target, not a bridge component: the live chain records *public* amounts, and the confidential-amounts transition is specified and tracked at the base layer (ADR 0006 / §5, §10; botho#902), out of scope for this section's register. wBTH is fully transparent regardless. The bridge is therefore a designed privacy boundary with explicit leakage semantics (ADR 0004).

1. **Lock reveals the amount.** To mint the correct wBTH, the bridge MUST learn the deposited amount. This is an unavoidable, deliberate privacy exit; it leaks only the amount, not the source ring.

2. **The wrapped side is public.** Anyone holding or moving wBTH is linkable on the host chain. Wallet UX SHOULD warn that *wrapping exits the anonymity set.*

3. **Unwrap re-shields the recipient.** A burn → release MUST pay out to a *fresh one-time stealth address*, never a reused address, breaking the on-Botho link to the destination-chain burn.

4. **Residual leakage is documented and accepted for v1.** Amount and timing correlation across the bridge remain; an observer correlating a distinctive wrap amount with a same-size unwrap shortly after can link the two ends. Denomination-bucketing and randomized-delay unlinkability is a later enhancement, not a v1 requirement.

Because only factor-1 / background coins are wrappable (§11.3), the amount revealed at lock is already of the least sensitive wealth class. This public-at-the-boundary property is exactly what makes the import-tagging factor (§11.5) computable with no zero-knowledge machinery.

11.5 Import Cluster Tagging

Botho's anti-hoarding mechanism prices *coin lineage*: the cluster-factor curve maps the wealth traceable to a coin's cluster origin(s) onto a demurrage / progressive-fee / lottery-tilt multiplier (1×–6×), and it is Sybil-resistant domestically because splitting a coin does not change the wealth traceable to its origin. The bridge introduces a vector the lineage mechanism cannot see (ADR 0007). If unwrapped BTH returned at factor-1, two leaks would follow:

1. **The entry leak.** External wealth of arbitrary size could buy wBTH on a DEX and unwrap into Botho at factor-1, having paid no lineage premium. This is unavoidable in the general case—you cannot tax external wealth entry without breaking the liquidity the bridge exists to provide.

2. **The round-trip laundromat.** A domestic holder who accumulated a high-factor lineage could round-trip (BTH → wBTH → unwrap → factor-1 BTH) and reset to background for the price of a wrap. This is a real defect: if any lineage reset door is cheap, the cluster-factor mechanism is trivially bypassable.

11.5.1 Epoch-keyed import cluster

The normative rule (ADR 0007), which *narrows* the entry leak and *closes* the round-trip laundromat, is:

> **Unwrapping MUST mint BTH into a bridge-import cluster keyed to the block-height epoch of the unwrap, at an elevated factor derived from that epoch cluster's aggregate unwrap wealth, subject to a floor. Imported wealth normalizes toward background only by circulating.**

Concretely, for an unwrap at block height h:

$$m = \lfloor h/K \rfloor\,, \tag{73}$$

$$c_{\text{import}}(m) = \mathcal{H}(\text{“bridge-import”} \,\|\, m)\,, \tag{74}$$

$$\text{import_factor}(m) = \max\Big(F,\ \text{ClusterFactorCurve}\big(W_m\big)\Big), \tag{75}$$

where W_m is the import cluster's cumulative wealth. It starts at the sum of the epoch's unwrap amounts and grows with every downstream output that carries the tag. On a spend, the node charges max(curve(output-tag wealth), ring centroid, F-blend on output tags). The epoch length and floor are:

Constant	Value	Meaning
K	17,280 blocks (1 day)	epoch length
F	1.5×	import-factor floor

The unwrapped output carries a 100%-weight tag to $c_{\text{import}}(m)$, exactly as a minting output carries 100% attribution to its new mint cluster—a bridge-import cluster is simply a third way to create a cluster origin, requiring no new machinery beyond the tag. The curve is the identical production log-sigmoid curve ($W_{\text{mid}} = 100$k, saturating 6×) that domestic clusters use.

11.5.2 Why the epoch key is load-bearing (Sybil-resistance)

A flat per-unwrap factor is Sybil-proof but over-taxes a small entrant identically to a whale. A size-based per-unwrap factor recovers size-sensitivity but is Sybil-able: a whale drip-splits into many dust unwraps, each a separate small origin at factor ≈ 1, then reassembles domestically. The epoch key defeats the split because all unwraps in a window *share one accumulating cluster*: intra-epoch splitting piles into the same pool and still hits the high factor. Diluting requires spreading across *epochs*, which costs wall-clock time (K blocks each)—**time-as-cost replaces provenance-as-cost**. The production-semantics calibration (#1697) finds a 2M-BTH whale needs 584 separate epochs to drip-dilute to the floor. Because K is counted in blocks, that is 584 days at 5 s blocks and 2,336–4,672 days at the 20–40 s blocks observed on testnet. The split-game wall holds only while the import tag cannot be shed by other means; see the caveat below.

11.5.3 Shared fate and decay by circulation

You cannot simultaneously have {Sybil-resistant, no-shared-fate, no-external-identity}. Forcing split pieces to aggregate into a shared pool is the only identity-free way to make splitting costly, which necessarily means an unwrapper's factor depends partly on strangers who unwrapped in the same epoch. This shared-fate coupling is judged a feature: bridge-inflow-*rate* becomes a concentration signal—a sudden capital flood is treated as maximally concentrated, an organic trickle enters benignly. The floor F plus a modest K bound the collateral to innocent small co-entrants, whose small coins blend down quickly. The design intent is that decay happens *only* by circulation: an imported coin's factor falls through value-weighted tag-blending on spends. A flood-epoch coin that mixes with equal domestic value each spend falls from 5.44× to 1.43× in 9 spends and its tag is pruned on the 10th. The import cluster's wealth is cumulative: every downstream output carrying the tag adds to it. So the shared-fate collateral keeps growing as the epoch's coins circulate.

Caveat: the floor is read from output tags (target-state). As built, the import floor is a blend over the spender's *output* tags, and consensus rejects only tag inflation. The stock wallet's 5% per-transfer decay prunes the import tag after 135 self-spends with no domestic

money at all. A spender who declares weight $\div 20$ per hop sheds a 5.44× flood coin to 1× in 3 spends for about 0.15% of value. Until consensus derives the floor from inputs (#1687, #1686), the guarantees of this subsection hold only for compliant wallets that do not churn. The recalibration (`docs/research/bridge-import-calibration.md`) leaves K and F unchanged.

Confidential-amounts-clean by construction. The epoch cluster's wealth is the sum of unwrap amounts, which are *public at the bridge boundary by necessity* (§11.4). The import factor is therefore computable from already-public data with no zero-knowledge gadget—it sidesteps, for imported coins, the confidential-amounts problem that domestic cluster-wealth accounting faces. This holds for the epoch's initial unwrap total. As built, W_m also accumulates the amounts of downstream outputs that carry the tag, and those amounts are not public under confidential amounts (#902, #1697).

Figure 19 shows the unwrap flow with import tagging.

11.6 Security

The bridge's security rests on five load-bearing invariants, each with a named adversarial-test surface (`bridge/service/src/adversarial_tests.rs`, `watchers/adversarial_tests.rs`, `chaos_tests.rs`, and the contract test suites). A conformant bridge MUST uphold every one of them:

1. **Exactly-once mint** — one confirmed deposit MUST mint at most once (order-id idempotency, on-chain replay guard, crash-safe transitions).
2. **Exactly-once release** — one confirmed burn MUST release at most once (record-before-broadcast, idempotent re-broadcast).
3. **Threshold authorization** — no mint or release fires without a t-of-n domain-separated, order-bound signature set; an equivocating signer MUST count once and is additionally flagged by a dedicated audit event.
4. **Peg solvency** — (70) MUST hold within tolerance (default 0), and any drift MUST trip a fail-closed circuit breaker.
5. **Finality safety** — an action MUST fire only against a block final on its chain (SCP finality on BTH; a confirmation-depth window plus canonical re-check on Ethereum; `Finalized` commitment on Solana).

Proof-of-reserves. A reconciler continuously checks (supply − locked) and (locked − supply) against in-flight tolerance. Because the reserve holds only zero-demurrage factor-1 coins (§11.3), the invariant is *exact* rather than approximate. Positive drift beyond tolerance (unbacked wrapped supply) or a reserve-balance shortfall (custody theft) MUST trip the breaker and emit a drift alert. A chain whose live supply / reserve-balance transport is unavailable MUST be reported *unverified* and excluded from drift math—never silently counted as healthy.

Fail-closed liveness trade. The bridge trades liveness for safety everywhere: peg drift, backlog and global caps, the on-chain auto-pause, and the operator kill-switch all *halt* rather than degrade. Every breaker trips on real, value-backed activity, so there is no free griefing vector; a halt leaves the invariants intact and is recovered by the documented operator procedure. The service-side global cap and the far looser on-chain auto-pause are deliberate two-layer defense (a tight cap that trips first, a last-resort on-chain breaker that halts even if the federation layer is compromised).

Coupling to consensus safety (accepted). Because the federation is the validator set, bridge safety is coupled to consensus safety: a validator-majority compromise also drains the reserve, and bridge-key operations MUST meet validator-grade opsec. Growing and hardening the validator set is a soft prerequisite to holding meaningful bridge value. This coupling is an accepted consequence of reusing a single trust root (ADR 0002).

Mainnet gate. No bridge value MUST move on mainnet until an external security audit of the Rust service, the Solidity token contract, and the Anchor program has cleared. This is a hard gate, tracked in the register (§11.7, row **IMP-6**).

11.7 Implementation Status

The bridge is partially shipped. Per the spec discipline, every mechanism this section describes in the present tense is live in the implementation unless it appears as a target-state row below, in which case the *live* behavior and the *target* behavior are named explicitly with a tracking reference. A claim that reads as "is" when the code says "will be" without a row here is a defect. Rows carry stable IDs (**IMP-1**–**IMP-6**) so that figure captions and cross-references resolve to an exact row.

ID	Component	Live behavior	Target (this spec)	Status	Tracking
IMP-1	Ethereum wrap / unwrap path	Deposit lock, t-of-n attestation, Safe `bridgeMint`, burn watcher, release; exactly-once at ledger + contract; factor-1 peg	(same)	live	PRs #832–#864
IMP-2	Import cluster tagging (ADR 0007)	Unwrap tags the released output 100% to $c_{\text{import}}(m)$ at import_factor$(m) \geq F$ ($F = 1.5\times$, $K = 1$ day); (73)–(75) (`bth_scan.rs` assigns the epoch import cluster)	(same)	live	botho#938 / #942
IMP-3	Demurrage-settlement operation	No consensus settlement op; a wealthy coin cannot be reclassified to factor-1	Paid reclassification to factor-1, fee routed to the lottery pool; shared $\approx$ 1-year lineage-reset horizon	target-state	botho#831 (horizon #833/#925)
IMP-4	Solana transports	Mint assembly / sign / submit / confirm and burn watcher are code-complete against a raw JSON-RPC client and mock-tested; live-node integration is `#[ignore]`d and unverified end-to-end	Fully live, externally-audited Solana mint + release + reserve-balance transport	target-state	botho#853/#856/#857/#858
IMP-5	BTH / Solana live supply + reserve-balance transport	Chain reported *unverified*, excluded from drift math (fail-safe)	Live supply + reserve-balance feeds the exact peg reconciliation	target-state	botho#853
IMP-6	External security audit	Internal adversarial + chaos + fuzz suites pass; no external audit	External audit of service + Solidity + Anchor cleared (mainnet gate)	target-state	botho#616/#830

Import-tagging landed (protocol 5.0.0). The import-factor machinery of §11.5 is live as of protocol 5.0.0 (#942, register row **IMP-2**). The production unwrap path (`bridge/service/src/bth_scan.rs`) now tags the released recipient output 100% to the block-epoch import cluster $c_{\text{import}}(\lfloor h/K \rfloor)$ via `import_cluster_id_for_height`, at import_factor$(m) = \max(F,$ ClusterFactorCurve$(\cdot))$ with $F = 1.5\times$ and $K = 17{,}280$ blocks; the pre-5.0.0 behavior (a factor-1 output carrying empty cluster tags) no longer applies. Every present-tense statement in §11.5 therefore holds of the implementation today.

12 Implementation

This section describes the reference implementation of Bothoand its performance characteristics.

Confidential-amount implementation status. The confidential-amount target remains under implementation. Production CLSAG transactions still publish amounts; the inactive single-input integer/R1CS experiments in issues #1284/#1288 do not supply a complete confidential transaction. They exercise exact staged floors, saturation and statement binding, but not the final multi-input commitment linkage, conservation, fee quantization, wire format or client migration. ADR0009 remains Proposed. The candidate contract in `docs/design/ct-transaction-contract.m` preserves ratified EpochOrigin while identifying remaining circulation-policy and fee-bucket decisions. Its arithmetic vectors and source inventory are specification evidence, not activation, an audited protocol or a mainnet-readiness claim. Timings below must not be interpreted as measurements of that unimplemented combined CT proof.

12.1 Reference Implementation

12.1.1 Architecture Overview

The reference implementation is written in Rust for memory safety and performance:

```
botho/
+-- account-keys/      # Key derivation (BIP39, SLIP-10)
+-- botho/             # Full node implementation
+-- botho-wallet/      # Command-line wallet
+-- cluster-tax/       # Progressive fee calculation
+-- consensus/scp/     # Stellar Consensus Protocol
+-- crypto/
|   +-- box/           # Encryption primitives
|   +-- keys/          # Key management
|   +-- lion/          # (Deprecated) Lattice signatures
|   +-- pq/            # Post-quantum (ML-KEM, ML-DSA)
|   +-- ring-signature/# CLSAG implementation
+-- transaction/
|   +-- core/          # Transaction types and validation
|   +-- signer/        # Transaction signing
|   +-- types/         # Shared types
+-- web/               # Desktop wallet (Tauri)
```

12.1.2 Dependencies

Key external dependencies:

- **curve25519-dalek** [12]: Ristretto255 group operations
- **bulletproofs**: Range proof implementation
- **pqcrypto**: ML-KEM and ML-DSA implementations
- **libp2p** [56]: Peer-to-peer networking
- **rocksdb** [18]: Key-value storage for blockchain data

12.1.3 Build Requirements

- Rust 1.75+ (stable)
- CMake 3.16+ (for native dependencies)
- 8 GB RAM minimum for compilation
- Linux, macOS, or Windows (with WSL2)

12.2 Performance Characteristics

12.2.1 Transaction Validation

Table 30: Validation timing (AMD Ryzen 9 5900X)

Operation	Time
CLSAG signature verify (ring=20)	2.1 ms
Bulletproof verify (2 outputs)	1.8 ms
ML-KEM decapsulate	0.05 ms
Full transaction validate	4.2 ms

12.2.2 Transaction Creation

Table 31: Transaction creation timing

Operation	Time
CLSAG signature create (ring=20)	3.8 ms
Bulletproof create (2 outputs)	45 ms
ML-KEM encapsulate	0.04 ms
Full transaction create	52 ms

12.2.3 Throughput

Table 32: System throughput

Metric	Value	Conditions
Max TPS (validation)	238 tx/s	Single core
Max TPS (validation)	1,428 tx/s	6 cores
Block validation	120 ms	100 tx block
Initial sync	4 hours	1M blocks

12.2.4 Resource Usage

12.3 Wallet Implementations

12.3.1 Command-Line Wallet

The `botho-wallet` CLI provides:

Table 33: Resource requirements

Node Type	Storage	Memory
Full node (unpruned)	50 GB	4 GB
Full node (pruned)	10 GB	2 GB
Light client	500 MB	256 MB
Minting node	50 GB	8 GB

```
botho-wallet init          # Create new wallet
botho-wallet balance       # Show balance
botho-wallet address       # Generate receiving address
botho-wallet send          # Create transaction
botho-wallet history       # Transaction history
botho-wallet sync          # Synchronize with network
botho-wallet export        # Export view key
```

12.3.2 Desktop Wallet

A cross-platform desktop wallet built with Tauri provides:

- Graphical interface for all wallet operations
- Secure mnemonic generation and storage
- Transaction history with privacy indicators
- Node connection management
- Subaddress management

12.3.3 Exchange Scanner

The `botho-exchange-scanner` supports exchange integration:

- View-key-only scanning for deposits
- Webhook notifications for incoming transactions
- Subaddress management for customer deposits
- No spend key required (security isolation)

12.4 Security Measures

12.4.1 Memory Protection

- Key material stored in `Zeroizing` wrappers
- Memory locking (`mlock`) for sensitive data
- Immediate zeroization on drop
- No key material in debug output

12.4.2 Constant-Time Operations

All cryptographic operations use constant-time algorithms:

- Scalar multiplication (curve25519-dalek)
- Signature verification
- Key comparison
- Decapsulation

12.4.3 Input Validation

Extensive input validation prevents:

- Buffer overflows (Rust memory safety)
- Invalid curve points (checked on deserialization)
- Integer overflows (checked arithmetic)
- Denial of service (resource limits)

12.5 Testing

12.5.1 Test Coverage

- Unit tests: 2,400+ tests
- Integration tests: 150+ tests
- Property-based tests: Key derivation, serialization
- Fuzzing: Transaction parsing, network messages

12.5.2 Test Networks

- **Testnet**: Public test network with accelerated timing
- **Stagenet**: Pre-production testing environment
- **Regtest**: Local regression testing mode

12.6 Scalability Analysis

This section analyzes Botho's scalability characteristics and long-term growth projections.

12.6.1 Blockchain Growth

Table 34: Projected blockchain size growth

Year	Blocks	Unpruned	Pruned
1	6.3M	30 GB	6 GB
5	31.5M	150 GB	24 GB
10	63M	300 GB	40 GB
20	126M	600 GB	70 GB

Assumptions: the 5 s monetary reference (6,307,200 blocks/year; the maximum block rate and thus the storage upper bound), matching the emission schedule of Section 7. Under the dynamic schedule the realized average is slower, so these are conservative ceilings. Also assumes 50% block utilization and a 4 KB average transaction size.

12.6.2 UTXO Set Growth

The UTXO set grows with network usage:

$$\text{UTXO}_{\text{count}}(t) = \text{UTXO}_0 + \int_0^t (\text{outputs}(\tau) - \text{spends}(\tau))\, d\tau \tag{76}$$

Growth projections:

- Year 1: ∼5 million UTXOs (∼800 MB)
- Year 5: ∼20 million UTXOs (∼3.2 GB)
- Year 10: ∼35 million UTXOs (∼5.6 GB)

Mitigation: UTXO set is significantly smaller than full blockchain and fits comfortably in RAM for most systems.

12.6.3 Key Image Database

The key image database grows monotonically (spent outputs are never removed):

$$\text{KeyImages}(t) = \sum_{\tau=0}^{t} \text{inputs}(\tau) \tag{77}$$

Growth projections (32 bytes per key image):

- Year 1: ∼10 million key images (∼320 MB)
- Year 5: ∼50 million key images (∼1.6 GB)
- Year 10: ∼100 million key images (∼3.2 GB)

Storage optimization: Key images are stored in a Bloom filter for fast negative lookups, with full database for confirmation.

12.6.4 Cluster Tag Database

Cluster tag vectors require additional storage:

- Per UTXO: Variable (average ∼100 bytes for tag vector)
- Total: Approximately 2× UTXO set storage

Pruning strategy: Old cluster data for spent outputs can be pruned after sufficient confirmations.

12.6.5 Throughput Limits

The block-time row uses the 5-second tier, the sustained high-load level and monetary reference. The consensus floor is 3 seconds (`ConsensusConfig::MIN_BLOCK_TIME_SECS`, `botho/src/consensus/service.rs`), reached at ≥20 tx/s; while load holds that tier, the block-time bound is ≈167 TPS and network propagation (200 TPS) is the next limit.

Table 35: Throughput analysis

Constraint	Limit	Bottleneck
Block size (2 MB)	∼500 tx/block	Storage/bandwidth
Block time (5s tier)	100 TPS peak	Consensus latency
Validation speed	1,400 TPS	CPU (6 cores)
Network propagation	200 TPS	Bandwidth (10 Mbps)
Effective limit	**∼100 TPS**	Block time

Table 36: Throughput comparison

System	TPS	Finality	Privacy
Bitcoin	∼7	60 min	None
Monero	∼30	20 min	Ring signatures
Zcash (shielded)	∼10	75 min	ZK proofs
Botho	∼100	5–10s	Ring + PQ stealth

12.6.6 Comparison to Existing Systems

12.6.7 Layer-2 Scaling

For higher throughput requirements, layer-2 solutions are planned:

Payment Channels:

- Open channel with on-chain transaction
- Unlimited off-chain transfers between parties
- Close channel to settle final balances
- Privacy preserved via channel encryption

Design considerations:

- Ring signatures complicate HTLC-style constructions
- Stealth addresses require novel routing approaches
- Research ongoing for privacy-preserving channel networks

State Channels:

- Generalized off-chain computation
- Requires scripting capability (future upgrade)
- Enables complex multi-party protocols

12.6.8 Pruning Effectiveness

Pruned nodes retain full validation capability while reducing storage:

Pruning ratio: Approximately 80% of historical data can be pruned after sufficient confirmations (default: 10,000 blocks).

Table 37: Pruning analysis

Data Type	Prunable?	Reason
Old block bodies	Yes	Only headers needed for chain
Spent TX outputs	Yes	Not needed for validation
Block headers	No	Required for chain verification
UTXO set	No	Required for spending
Key image set	No	Required for double-spend detection

Table 38: Recommended hardware by use case

Use Case	CPU	RAM	Storage
Light client	Any	512 MB	1 GB
Pruned full node	2 cores	4 GB	20 GB SSD
Archive full node	4 cores	8 GB	200 GB SSD
Minting node	8 cores	16 GB	200 GB NVMe

12.6.9 Node Hardware Requirements

12.7 Future Development

12.7.1 Planned Improvements

- **Mobile wallet**: iOS and Android applications
- **Hardware wallet support**: Ledger/Trezor integration
- **Atomic swaps**: Cross-chain exchange without intermediaries
- **Payment channels**: Layer-2 scaling for micropayments

12.7.2 Post-Quantum Ring Signatures

Future protocol upgrade may introduce post-quantum ring signatures once efficient constructions become practical:

- Lattice-based ring signatures (research ongoing)
- Hybrid classical + PQ approach
- Backward compatibility via soft fork

12.8 User Experience Design

Privacy-preserving systems often sacrifice usability. Bothoprioritizes user experience while maintaining strong privacy defaults.

12.8.1 Address Format

Addresses use a human-friendly format with error detection:

```
botho1[network][type][payload][checksum]

Example:
botho1mq7k9p8z4x5c3v2b1n0m9q8w7e6r5t4y3u2i1o0p9a8s7d6f
     ^  ^    ^--------------------------^ ^------^
```

```
|  |              payload            checksum
|  type (main=m, sub=s)
network (mainnet=m, testnet=t)
```

Design choices:

- **Prefix "botho1"**: Clear identification, avoids confusion with other cryptocurrencies
- **Bech32 encoding**: Error detection, case-insensitive, no ambiguous characters (0/O, 1/l)
- **Network identifier**: Prevents accidental cross-network sends
- **Checksum**: 6-character checksum catches typos

12.8.2 Transaction Confirmation UX

Users receive clear feedback on transaction status:

Table 39: Transaction status indicators

Status	Meaning	Wait Time
Pending	In mempool, not yet in block	0–5s
Confirming	In block, awaiting SCP	5–10s
Finalized	SCP externalized	Instant

Key UX improvement: Unlike Bitcoin (10+ confirmations) or Monero (10+ confirmations), Bothotransactions are final after single SCP externalization—typically 5–10 seconds total.

12.8.3 Wallet Backup and Recovery

Mnemonic-based backup:

- 24-word BIP39 mnemonic (256 bits entropy)
- Optional passphrase for additional security
- Derives all keys deterministically
- Compatible with hardware wallets

Recovery process:

1. Enter 24-word mnemonic (and passphrase if used)
2. Wallet derives all keys from seed
3. Wallet scans blockchain for owned outputs
4. Full balance and history restored

Scan time: Approximately 1 hour per year of blockchain history (can be parallelized with multiple full nodes).

12.8.4 Subaddress Management

Subaddresses provide unlimited receiving addresses from a single wallet:

```
// Generate labeled subaddress
wallet.create_subaddress("Alice payment")
// -> botho1s7k9p8z4x5c3v2b1n0m9q8w7e6r5t4y3u2i1o0p9a8s7d6g

// List subaddresses with balances
wallet.list_subaddresses()
// -> [(0, "Primary", 100.0 BTH),
//     (1, "Alice payment", 5.0 BTH),
//     (2, "Bob refund", 0.0 BTH)]
```

Use cases:

- Separate addresses for different payers
- Merchant payment tracking
- Exchange deposit addresses
- Privacy (fresh address per transaction)

12.8.5 Fee Estimation

Wallets provide clear fee information:

```
Transaction Summary:
  Amount:      50.00 BTH
  Consensus floor: 19,200 pico-BTH  (4 KB x 1 pico/byte x 1.2 factor
                                     x 2^2 output penalty)
  Cluster factor:  1.2x
  Relay minimum:   100,000,000 pico-BTH  (MIN_TX_FEE)
  Total fee:       100,000,000 pico-BTH  (= 1e-4 BTH)

  [Expected confirmation: ~5 seconds]
```

Fee transparency: Users see the consensus floor and their cluster factor multiplier. At today's constants the flat relay minimum (`MIN_TX_FEE`, `transaction/clsag/src/lib.rs`) exceeds the floor, so it sets the fee an ordinary wallet pays (pending #1688, #1684).

12.8.6 Error Handling

Clear, actionable error messages:

12.8.7 Privacy Indicators

Wallets display privacy-relevant information:

- **Ring size**: Number of decoys used (always 20)
- **Output age**: How long outputs have existed
- **Cluster factor**: Current fee multiplier
- **Network privacy**: Tor/I2P connection status

Privacy score: Optional aggregate privacy indicator (1–5 stars) based on output age distribution, cluster diversity, and network connection.

Table 40: Example error messages

Situation	Message
Insufficient funds	"Balance (45.00 BTH) insufficient for amount (50.00) plus fee (0.001)"
Invalid address	"Address checksum invalid. Please verify the address."
Network offline	"Cannot connect to network. Check internet connection."
Wallet locked	"Wallet is locked. Enter password to unlock."

12.9 Testnet Deployment

A public testnet is available for development and testing.

12.9.1 Testnet Parameters

Table 41: Testnet vs. mainnet parameters

Parameter	Testnet	Mainnet
Block time	3–40s (dynamic)	3–40s (dynamic)
Initial block reward	50 BTH	50 BTH
Halving interval	120,000 blocks	6,307,200 blocks
Ring size	20	20
Minimum fee	1 pico-BTH/byte	1 pico-BTH/byte
Network port	19732	9732
Magic bytes	`0x00075E57`	`0x000B07B0`

12.9.2 Faucet

Testnet coins are freely available:

```
# Web faucet
https://faucet.testnet.botho.org

# CLI request (1000 tBTH daily limit)
botho-wallet --testnet faucet request <address>

# Response
Transaction broadcast: 0x7f3a...2b1c
Expected confirmation: ~30 seconds
```

12.9.3 Genesis Block

```
GenesisBlock
    timestamp: 1704067200,  // 2024-01-01 00:00:00 UTC
    difficulty: 0x00000fff...fff,  // Low initial difficulty
    nonce: 0,

    // Genesis coinbase to testnet foundation
    minting_tx:
```

```
    outputs: [
         amount: 1_000_000, recipient: "testnet_treasury" ,
    ],
    minter: TestnetFoundationKey,
,

// Genesis quorum set
initial_validators: [
    "testnet1.botho.org",
    "testnet2.botho.org",
    "testnet3.botho.org",
],
```

Genesis hash (testnet): `0x1a2b3c...` (truncated for brevity)

12.9.4 Testnet Reset Policy

The testnet may be reset periodically:

- Scheduled resets announced 2 weeks in advance
- Protocol upgrades may require chain reset
- No guarantee of persistent state or balances
- Export wallet seeds before announced resets

12.10 Deployment

Node operation is documented separately in the operator guide shipped with the reference implementation, covering configuration (`config.toml`), hardware sizing by role (light / full / minting), process supervision, Prometheus metrics, and backup/restore. These are operational concerns rather than protocol specification and are out of scope here.

12.11 Code Availability

The reference implementation is open source:

- Repository: https://github.com/botho-project/botho
- License: MIT
- Contributing: See CONTRIBUTING.md
- Security issues: security@botho.org

13 Governance and Protocol Upgrades

Protocol evolution is essential for long-term viability. Bothoemploys a structured governance process that balances agility with stability, enabling necessary upgrades while protecting users from disruptive changes.

13.1 Design Principles

13.1.1 Conservative by Default

The protocol favors stability over rapid iteration:

- Changes require demonstrated necessity, not merely novelty
- Backward compatibility is preserved wherever feasible
- Breaking changes require extended notice and migration support
- Security fixes take precedence over feature additions

13.1.2 Transparency and Predictability

All governance processes are public and follow documented procedures:

- Proposal discussions occur in public forums
- Technical specifications are peer-reviewed before implementation
- Activation timelines are announced well in advance
- Post-activation monitoring reports are published

13.2 Protocol Versioning

13.2.1 Version Scheme

Bothouses semantic versioning with blockchain-specific semantics:

```
MAJOR.MINOR.PATCH

MAJOR: Consensus-breaking changes (hard fork required)
MINOR: New features, backward compatible
PATCH: Bug fixes, no new features
```

Examples:

- 1.0.0 → 1.0.1: Security patch, no action required
- 1.0.0 → 1.1.0: New transaction type added, old nodes still valid
- 1.0.0 → 2.0.0: Consensus rule change, upgrade mandatory

Nodes advertise their protocol version during handshake and disconnect gracefully from peers with an incompatible (different MAJOR) version.

13.3 Upgrade Mechanisms

13.3.1 Soft Forks

Soft forks tighten consensus rules—old nodes accept new blocks, but new nodes may reject some blocks old nodes would accept.

Use cases:

- Adding new transaction validation rules
- Restricting previously-allowed behaviors

- Deprecating obsolete features

Activation process:

1. **Proposal**: BIP-style document published with technical specification
2. **Implementation**: Reference client updated with activation logic
3. **Signaling**: Miners include version bits in block headers
4. **Lock-in**: When 95% of blocks in a ∼2-week window signal support (≈240,000 blocks at the 5 s reference; the window is block-denominated and scaled to block time, not copied from Bitcoin's 2016-block figure)
5. **Activation**: Rules enforced after an equal grace period

```
BlockHeader
    version: u32,  // Bits 0-28: version, bits 29-31: soft fork signals
    // ...


// Check signaling for proposal #3
fn signals_proposal(header: &BlockHeader, proposal: u8) -> bool
    (header.version >> (29 + proposal)) & 1 == 1
```

13.3.2 Hard Forks

Hard forks relax or modify consensus rules—old nodes reject new blocks.

Use cases:

- Fundamental protocol changes (new cryptographic primitives)
- Emergency security responses
- Removal of technical debt

Activation process:

1. **Proposal**: Extended discussion period (minimum 6 months)
2. **Specification**: Formal specification with test vectors
3. **Implementation**: All major clients implement changes
4. **Testnet**: Minimum 3-month testnet validation
5. **Announcement**: Activation height announced 3+ months ahead
6. **Activation**: At specified block height

Mandatory upgrade policy: Users have minimum 6 months warning before hard fork activation, except for emergency security fixes.

13.4 Emergency Procedures

Critical vulnerabilities follow coordinated private disclosure (reported to security@botho.org, assessed within 24 hours, patched and disclosed with an advisory). For catastrophic consensus events, the last-resort mechanism is signed checkpoints:

Checkpoint authority: In extreme emergencies, core developers may publish signed checkpoints that compliant nodes refuse to reorg past — a mechanism that trades decentralization for safety and is consensus-relevant because it bounds the reorg depth honest nodes will accept.

13.5 Decentralization and Fork Rights

Governance resists capture through rough consensus (not vote counting), multiple independent implementations, and a transparent public process. The ultimate check is the right to fork:

13.5.1 Fork Rights

Users retain the ultimate governance power—the right to fork:

- Protocol specification is open and unencumbered
- Reference implementation is permissively licensed (MIT)
- No trademark restrictions on protocol-compatible implementations
- Community can reject controversial changes by running alternatives

Philosophy: Formal governance exists to coordinate, not to rule. The community's ability to fork ensures that governance serves users rather than capturing them.

14 Conclusion

14.1 Summary

This paper has presented Botho, a privacy-preserving cryptocurrency that addresses fundamental challenges in existing systems through principled design tradeoffs.

14.1.1 Key Contributions

1. **Hybrid post-quantum architecture**: By applying post-quantum cryptography selectively based on data lifetime, Bothoachieves meaningful quantum resistance while maintaining practical transaction sizes. Recipient identities are protected by ML-KEM-768 against future quantum attacks; sender privacy uses efficient CLSAG ring signatures appropriate for ephemeral data.

2. **PoW + SCP consensus**: Combining proof-of-work block proposal with Stellar Consensus Protocol finalization achieves the best of both worlds: permissionless participation and fair distribution from PoW, with fast deterministic finality and Byzantine fault tolerance from SCP.

3. **Progressive fee mechanism**: Cluster-based fees and demurrage price coins by the wealth of their ancestry while preserving privacy. Unlike identity-based approaches, fees are determined by coin ancestry, resisting Sybil attacks without requiring user identification. The mechanism is aimed at wealth concentration, but a measurable reduction in inequality is not demonstrated at production parameters (Section 10.6.2).

4. **Sustainable economics**: Perpetual tail emission ensures long-term security funding. Fee burning creates deflationary pressure proportional to network usage. Dynamic block timing aligns emission with network utility.

14.1.2 Design Philosophy

Bothoembodies the principle of *botho*—that currency should serve community rather than concentrate power. This manifests in:

- Privacy as baseline, not premium feature

- Progressive economics aimed at discouraging hoarding
- Decentralized consensus with open participation
- Sustainable security through perpetual emission

14.2 Limitations and Future Work

14.2.1 Current Limitations

- **Classical ring signatures**: CLSAG provides sender privacy against classical adversaries only. A sufficiently powerful quantum computer could potentially identify signers retroactively.
- **Transaction size**: At approximately 4 KB per transaction, Bothotransactions are larger than non-private alternatives, though significantly smaller than fully post-quantum constructions.
- **Synchronization time**: Full nodes require several hours to synchronize, limiting immediate usability for new users.
- **Redistribution not demonstrated**: At production parameters the redistribution mechanism does not measurably reduce inequality (mechanism ΔG of -0.028 to $+0.003$ against a 0.05 bar), and lottery ticket-splitting is profitable. A redesign is under study (Section 10.6.2).

14.2.2 Future Research Directions

1. **Post-quantum ring signatures**: As lattice-based ring signature constructions mature, a future protocol upgrade could provide full post-quantum sender privacy while maintaining practical transaction sizes.
2. **Layer-2 scaling**: Payment channels and other off-chain constructions could enable micropayments and higher throughput while preserving privacy guarantees.
3. **Cross-chain interoperability**: Atomic swap protocols enabling trustless exchange with other cryptocurrencies would enhance utility without sacrificing privacy.
4. **Advanced privacy techniques**: Research into proof aggregation, recursive proofs, and other techniques may enable improved privacy-efficiency tradeoffs.

14.2.3 Interoperability Architecture

Cross-chain functionality requires careful design to preserve privacy. We outline preliminary approaches for future development.

Atomic Swaps. Hash Time-Locked Contracts (HTLCs) enable trustless cross-chain exchange:

1. Alice (BTH) and Bob (BTC) agree on exchange rate
2. Alice generates secret s, publishes $h = \mathcal{H}(s)$
3. Alice locks BTH in contract: "Redeemable by Bob with preimage of h before block T, else refundable to Alice after $T + \Delta$"
4. Bob locks BTC in mirrored contract on Bitcoin

5. Alice reveals s to claim BTC
6. Bob uses s to claim BTH

Privacy considerations:

- Hash h links both transactions—correlation attack possible
- Mitigation: Use adaptor signatures [1] instead of hash locks, eliminating on-chain correlation
- Timing correlation remains; recommend delays between claim operations

Protocol sketch (adaptor signature variant):

```
// Setup: Alice has BTH, wants BTC from Bob
Alice: Generate adaptor secret t, T = tG
       Create BTH tx with adaptor signature sig_A(T)
Bob:   Verify adaptor, create BTC tx with sig_B(T)
Alice: Complete sig_A by revealing t
Bob:   Extract t from completed sig_A
       Complete sig_B to claim BTC
// No hash appears on either chain
```

Bridge Design. Bridges enabling wrapped BTH on other chains face inherent tradeoffs:

Table 42: Bridge architecture comparison

Type	Trust	Privacy	Capital Efficiency
Custodial	High	Low	High
Federated	Medium	Medium	High
Optimistic	Low	Medium	Medium
ZK Light Client	Minimal	High	Medium

Recommended architecture: ZK light client bridge

- Target chain verifies BTH header proofs (SCP finality)
- Deposits create locked UTXOs with verifiable commitments
- ZK proof attests to valid lock without revealing details
- Withdrawals require ZK proof of burn on target chain

DEX Integration. Decentralized exchange on Bothofaces challenges:

- **Order book privacy**: Orders reveal trading intent
  - Solution: Encrypted order book with threshold decryption
  - Alternative: AMM-style pools (simpler but less efficient)
- **Settlement privacy**: Matched trades reveal amounts
  - Solution: Batch settlement with commitment aggregation
  - Users prove trade validity without revealing specific amounts
- **Front-running resistance**: SCP finality helps but mempool remains visible during PoW phase

- Solution: Encrypted mempool with threshold reveal
- Requires additional infrastructure investment

AMM Implementation Sketch:

```
Pool
    reserve_commitment: Commitment,  // C = r_A*H + r_B*G
    k_commitment: Commitment,        // x*y = k hidden


Swap
    input_commitment: Commitment,
    output_commitment: Commitment,
    // ZK proof: new reserves satisfy constant product
    // ZK proof: amounts are non-negative
    // ZK proof: fee correctly calculated
```

Research challenges:

1. Efficient ZK proofs for constant product verification
2. Privacy-preserving price oracle mechanisms
3. MEV resistance in decentralized setting
4. Liquidity provider privacy (shares reveal exposure)

These interoperability mechanisms remain active research areas. We expect community contributions to refine and implement practical solutions.

14.3 Broader Impact

14.3.1 Privacy as Human Right

Financial privacy protects fundamental human interests:

- Victims of abuse can escape financial control
- Dissidents can support causes without persecution
- Individuals can conduct legal business without surveillance
- Commercial entities can protect competitive information

Bothoprovides these protections by default, ensuring privacy is not relegated to those with technical sophistication or willingness to pay premium fees.

14.3.2 Economic Implications

The progressive fee mechanism represents an experiment in cryptocurrency economics:

- Can market mechanisms encourage circulation without coercion?
- Can a provenance-based lottery and demurrage measurably reduce inequality? At production parameters the current design does not (Section 10.6.2)
- Does cluster-based taxation effectively resist Sybil attacks?
- Will tail emission prove superior to fixed supply for security?

Only real-world deployment will answer these questions definitively.

14.4 Closing Remarks

Cryptocurrencies present a rare opportunity to redesign fundamental economic infrastructure. Rather than replicating the surveillance and inequality of traditional finance, we can build systems that respect privacy and encourage fair participation.

Motho ke motho ka batho—a person is a person through other people. Bothois designed with this principle at its core: technology that serves community, privacy that protects dignity, and economics designed to resist concentration.

We invite the community to examine, critique, and improve upon this design. The source code is open; the conversation continues.

Acknowledgments

We thank the cryptographic research community whose foundational work makes systems like Bothopossible, particularly:

- The CryptoNote developers for ring signature privacy
- The Monero Research Lab for CLSAG and related improvements
- The Bulletproofs authors for efficient range proofs
- The NIST PQC team for standardizing ML-KEM and ML-DSA
- The Stellar Development Foundation for SCP

We also thank the early reviewers and testers who provided invaluable feedback on protocol design and implementation.

References

[1] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Hostakova, Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. One-time verifiably encrypted signatures a.k.a. adaptor signatures. Cryptology ePrint Archive, 2021. URL https://eprint.iacr.org/2020/476. Scriptless atomic swaps without on-chain hash correlation.

[2] Aztec Labs. Aztec protocol: Zero-knowledge privacy for ethereum, 2023. URL https://aztec.network/. Private L2 zkRollup using PLONK proofs.

[3] Beam Development. Beam: Confidential cryptocurrency and DeFi platform. Open-source project, 2019. URL https://beam.mw.

[4] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. Succinct non-interactive zero knowledge for a von neumann architecture. In *USENIX Security Symposium*, 2014.

[5] Daniel J. Bernstein. Curve25519: New Diffie-Hellman speed records. In *Public Key Cryptography – PKC 2006*, pages 207–228, 2006. doi: 10.1007/11745853_14.

[6] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS – Kyber: A CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367, 2018. doi: 10.1109/EuroSP.2018.00032.

[7] Eric Budish. The economic limits of bitcoin and anonymous, decentralized trust on the blockchain. *Journal of Political Economy*, 2022. Economic analysis of PoW security.

[8] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *IEEE Symposium on Security and Privacy*, 2018.

[9] Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. *OSDI*, 99:173–186, 1999.

[10] Wouter Castryck and Thomas Decru. An efficient key recovery attack on SIDH. Cryptology ePrint Archive, 2022. URL https://eprint.iacr.org/2022/975. Polynomial-time attack breaking SIKE/SIDH isogeny assumptions.

[11] Nicolas T. Courtois and Rebekah Mercer. Stealth address and key management techniques in blockchain systems. *Proceedings of the 3rd International Conference on Information Systems Security and Privacy (ICISSP)*, pages 559–566, 2017. doi: 10.5220/0006270005590566. Dual-key stealth address protocol (DKSAP); primary reference for CryptoNote-style stealth addressing.

[12] dalek cryptography. curve25519-dalek: A pure-Rust implementation of group operations on Ristretto and Curve25519. Open-source library, 2019. URL https://github.com/dalek-cryptography/curve25519-dalek.

[13] Henry de Valence, Isis Lovecruft, Tony Arcieri, and Martin Kleppmann. The ristretto group, 2018. URL https://ristretto.group.

[14] Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: A lattice-based digital signature scheme. In *IACR Transactions on Cryptographic Hardware and Embedded Systems*, volume 2018, pages 238–268, 2018. doi: 10.13154/tches.v2018.i1.238-268.

[15] Muhammed F. Esgin, Ron Steinfeld, Joseph K. Liu, and Dongxi Liu. Efficient lattice-based ring signatures. In *International Conference on Post-Quantum Cryptography*, pages 176–194, 2019. doi: 10.1007/978-3-030-25510-7_10. Module-LWE based linkable ring signatures.

[16] Muhammed F. Esgin, Ron Steinfeld, Joseph K. Liu, and Dongxi Liu. Lattice-based ring signatures: A survey. Cryptology ePrint Archive, 2019. URL https://eprint.iacr.org/2019/1167. Survey of post-quantum ring signature constructions.

[17] Ittay Eyal and Emin Gün Sirer. Majority is not enough: Bitcoin mining is vulnerable. In *International Conference on Financial Cryptography and Data Security*, pages 436–454, 2014. doi: 10.1007/978-3-662-45472-5_28.

[18] Facebook. RocksDB: A persistent key-value store. Open-source project, 2012. URL https://rocksdb.org.

[19] Giulia Fanti, Shaileshh Bojja Venkatakrishnan, Surya Bakshi, Bradley Denby, Shruti Bhargava, Andrew Miller, and Pramod Viswanath. Dandelion++: Lightweight cryptocurrency networking with formal anonymity guarantees. In *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, volume 2, pages 1–35, 2018.

[20] Giulia Fanti et al. Breaking mimblewimble's privacy model. In *arXiv preprint*, 2019. arXiv:1907.10039.

[21] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology — CRYPTO' 86*, pages 186–194, 1986. doi: 10.1007/3-540-47721-7_12. Interactive to non-interactive proof transformation.

[22] Eiichiro Fujisaki and Koutarou Suzuki. Traceable ring signature. In *Public Key Cryptography – PKC 2007*, pages 181–200, 2007. doi: 10.1007/978-3-540-71677-8_13. Introduces linkable/traceable ring signatures.

[23] Qian Ge, Yuval Yarom, David Cock, and Gernot Heiser. A survey of microarchitectural side-channel vulnerabilities, attacks, and defenses in cryptographic software. In *ACM Computing Surveys*, volume 51, pages 1–40, 2019. doi: 10.1145/3212479. Comprehensive survey of side-channel attacks including SGX vulnerabilities.

[24] Silvio Gesell. *The Natural Economic Order*. Peter Owen Ltd, 1958. Historical demurrage currency theory (originally 1916).

[25] Brandon Goodell, Sarang Noether, and RandomRun. Concise linkable ring signatures and forgery against adversarial keys. In *Cryptology ePrint Archive*, 2019. Report 2019/654.

[26] Grin Developers. Grin: A lightweight implementation of MimbleWimble. Open-source project, 2019. URL https://grin.mw.

[27] Lov K Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996.

[28] Jona Harris and Aviv Zohar. Flood & loot: A systemic attack on the lightning network. In *Advances in Financial Technologies (AFT)*, 2020. Attack on payment channels.

[29] Ethan Heilman, Alison Kendler, Aviv Zohar, and Sharon Goldberg. Eclipse attacks on Bitcoin's peer-to-peer network. In *USENIX Security Symposium*, pages 129–144, 2015.

[30] Daira Hopwood, Sean Bowe, Taylor Hornby, and Nathan Wilcox. Zcash protocol specification. 2016. URL https://zips.z.cash/protocol/protocol.pdf.

[31] Daira Hopwood, Sean Bowe, et al. ZIP 224: Orchard shielded protocol. Zcash Improvement Proposal, 2021. URL https://zips.z.cash/zip-0224. Halo 2 based shielded transactions.

[32] Gur Huberman, Jacob D. Leshno, and Ciamac Moallemi. Monopoly without a monopolist: An economic analysis of the bitcoin payment system. *The Review of Economic Studies*, 88 (6):3011–3040, 2021. doi: 10.1093/restud/rdab014.

[33] Aram Jivanyan and Aaron Feickert. Lelantus spark: Secure and flexible private transactions. Cryptology ePrint Archive, 2021. URL https://eprint.iacr.org/2021/1173. Privacy protocol combining Spark addresses with one-out-of-many proofs.

[34] Hugo Krawczyk and Pasi Eronen. HMAC-based extract-and-expand key derivation function (HKDF). Technical Report RFC 5869, IETF, 2010. URL https://www.rfc-editor.org/rfc/rfc5869.

[35] Jae Kwon. Tendermint: Consensus without mining. In *Tendermint Whitepaper*, 2014. URL https://tendermint.com/static/docs/tendermint.pdf. BFT consensus for proof-of-stake.

[36] Leslie Lamport, Robert Shostak, and Marshall Pease. The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, 1982. doi: 10.1145/357172.357176.

[37] Petar Maymounkov and David Mazières. Kademlia: A peer-to-peer information system based on the xor metric. In *International Workshop on Peer-to-Peer Systems*, pages 53–65, 2002.

[38] David Mazières. The stellar consensus protocol: A federated model for internet-level consensus. In *Stellar Development Foundation*, 2015.

[39] Sarah Meiklejohn, Marjori Pomarole, Grant Jordan, Kirill Levchenko, Damon McCoy, Geoffrey M. Voelker, and Stefan Savage. A fistful of bitcoins: Characterizing payments among men with no names. In *Proceedings of the 2013 Internet Measurement Conference*, pages 127–140, 2013. doi: 10.1145/2504730.2504747. Bitcoin transaction graph analysis.

[40] MobileCoin Foundation. Mechanics of mobilecoin, 2021. URL https://mobilecoin.com. First Edition.

[41] Monero Project. Monero: A private digital currency, 2014. URL https://getmonero.org.

[42] Malte Möser, Kyle Soska, Ethan Heilman, Kevin Lee, Henry Heffan, Shashvat Srivastava, Kyle Hogan, Jason Hennessey, Andrew Miller, Arvind Narayanan, and Nicolas Christin. An empirical analysis of traceability in the Monero blockchain. In *Proceedings on Privacy Enhancing Technologies*, volume 2018, pages 143–163, 2018. doi: 10.1515/popets-2018-0025.

[43] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2008. URL https://bitcoin.org/bitcoin.pdf.

[44] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. Technical Report FIPS 202, NIST, 2015. URL https://csrc.nist.gov/pubs/fips/202/final.

[45] National Institute of Standards and Technology. Fips 203: Module-lattice-based key-encapsulation mechanism standard, 2024. URL https://csrc.nist.gov/pubs/fips/203/final.

[46] National Institute of Standards and Technology. Fips 204: Module-lattice-based digital signature standard, 2024. URL https://csrc.nist.gov/pubs/fips/204/final.

[47] National Institute of Standards and Technology. Post-quantum cryptography standardization, 2024. URL https://csrc.nist.gov/projects/post-quantum-cryptography.

[48] Shen Noether. Ring signature confidential transactions for monero. Cryptology ePrint Archive, 2015. URL https://eprint.iacr.org/2015/1098. MLSAG ring signatures for multiple inputs.

[49] Shen Noether, Adam Mackenzie, and Monero Research Lab. Ring confidential transactions. Monero Research Lab Report MRL-0005, 2016. URL https://lab.getmonero.org/pubs/MRL-0005.pdf. Introduces RingCT combining ring signatures with confidential transactions.

[50] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3: One function, fast everywhere. Specification, 2020. URL https://github.com/BLAKE3-team/BLAKE3-specs.

[51] Marek Palatinus, Pavol Rusnak, Aaron Voisine, and Sean Bowe. Bip-0039: Mnemonic code for generating deterministic keys, 2013. URL https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki.

[52] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Advances in Cryptology—CRYPTO'91*, pages 129–140, 1992.

[53] Trevor Perrin. The noise protocol framework, 2018. URL https://noiseprotocol.org.

[54] Andreas Pfitzmann and Marit Hansen. A terminology for talking about privacy by data minimization: Anonymity, unlinkability, undetectability, unobservability, pseudonymity, and identity management. Technical report, TU Dresden, 2010. URL http://dud.inf.tu-dresden.de/Anon_Terminology.shtml. Standard terminology for privacy properties, v0.34.

[55] Andrew Poelstra and Tom Elvis Jedusor. Mimblewimble, 2016. URL https://scalingbitcoin.org/papers/mimblewimble.pdf.

[56] Protocol Labs. libp2p: A modular network stack. Open-source project, 2019. URL https://libp2p.io. Peer-to-peer networking library.

[57] QRL Foundation. The quantum resistant ledger, 2018. URL https://theqrl.org.

[58] Ronald L. Rivest, Adi Shamir, and Yael Tauman. How to leak a secret. *Advances in Cryptology — ASIACRYPT 2001*, pages 552–565, 2001. doi: 10.1007/3-540-45682-1_32. Original ring signature construction.

[59] Pavol Rusnak and Marek Palatinus. Slip-0010: Universal private key derivation from master private key, 2016. URL https://github.com/satoshilabs/slips/blob/master/slip-0010.md.

[60] Secret Network. Secret network: Decentralized privacy-preserving computation, 2020. URL https://scrt.network/. TEE-based privacy using Intel SGX enclaves.

[61] Peter W Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

[62] Tevador et al. RandomX: A proof-of-work algorithm designed for general-purpose cpus. Specification, 2019. URL https://github.com/tevador/RandomX. CPU-friendly, ASIC-resistant PoW.

[63] Wilson Abel Alberto Torres, Ron Steinfeld, Amin Sakzad, Joseph K Liu, Veronika Kuchta, Nandita Bhattacharjee, Man Ho Au, and Jacob Cheng. Post-quantum lattice-based ring signatures, 2018. Cryptology ePrint Archive, Report 2018/762.

[64] United Nations General Assembly. Universal declaration of human rights, 1948. Article 12.

[65] Nicolas van Saberhagen. Cryptonote v2.0, 2013. URL https://cryptonote.org/whitepaper.pdf.

A Notation Reference

This appendix provides a comprehensive reference for notation used throughout the paper.

A.1 Mathematical Notation

Symbol	Description
Groups and Fields	
$\mathbb{G}$	Elliptic curve group (Ristretto255)
G	Generator of $\mathbb{G}$
H	Secondary generator for Pedersen commitments
q	Order of $\mathbb{G}$ (prime)
$\mathbb{Z}_q$	Integers modulo q
R_q	Polynomial ring for lattice operations
Keys	
a, b	View and spend private keys
A, B	View and spend public keys
(C_i, D_i)	Subaddress public key pair for index i
P	One-time public key
x	One-time private key
I	Key image
Hash Functions	
$H_s(\cdot)$	Hash function to scalar
$H_p(\cdot)$	Hash function to curve point
$\mathcal{H}(\cdot)$	General cryptographic hash
Ring Signatures	
$\{P_i\}_{i=0}^{n-1}$	Ring of public keys
π	Secret index of real key in ring
σ	Signature
n	Ring size (default: 20)
Commitments	
C	Pedersen commitment
C_v	Commitment to value v
r	Blinding factor
Operations	
$\xleftarrow{\$}$	Uniform random sampling
$\|$	Concatenation
$\oplus$	Bitwise XOR
$\bmod q$	Modular reduction
Complexity	
$\mathsf{negl}(\lambda)$	Negligible function in λ
$\mathsf{poly}(\lambda)$	Polynomial function in λ
λ	Security parameter

A.2 Protocol Identifiers

Identifier	Description
CLSAG	Concise Linkable Spontaneous Anonymous Group signatures
ML-KEM	Module-Lattice-based Key Encapsulation Mechanism
ML-DSA	Module-Lattice-based Digital Signature Algorithm
SCP	Stellar Consensus Protocol
PoW	Proof of Work
BTH	Native currency unit (Botho)

A.3 Network Constants

Parameter	Value	Description
Ring size	20	Decoys per input
Min block time	3s	Consensus floor (very high load, $\geq$20 tx/s)
Max block time	40s	At zero utilization
Block size limit	2 MB	Maximum block size
Transaction limit	100 KB	Maximum transaction size
Default port	9732	Main network port

A.4 Cryptographic Parameters

Algorithm	Parameter	Value
ML-KEM-768	Public key	1,184 bytes
	Ciphertext	1,088 bytes
CLSAG	Signature (ring=20)	704 bytes
	Key image	32 bytes
Bulletproofs	2-output proof	736 bytes
	Aggregation	Logarithmic

A.5 Transaction Sizes

Component	Size	Notes
Transaction header	32 bytes	Version, prefix data
Input (per)	680 bytes	Ring refs + key image + tag
Output (per)	1,152 bytes	Commitment + key + ciphertext + tag
CLSAG signature	704 bytes	Per input
Bulletproof	736 bytes	2 outputs, aggregated
1-in-2-out total	~4,552 bytes	Typical transaction

A.6 Monetary Parameters

Parameter	Value	Description
Initial block reward	50 BTH	Genesis reward
Halving period	6,307,200 blocks	$\approx$1 year (at 5s blocks)
Halving count	5	Time-to-tail $\approx$5 years
Phase-1 supply	$\approx$611M BTH	After 5 halvings
Tail emission	2.5% gross/yr	Supply-dependent ($\approx$2.42 BTH/block at 611M); realized net $\approx$2.1–2
Decimal places	12	Smallest: 1 picocredit
Base fee rate	1 pico-BTH/byte	Minimum fee
Max cluster factor	6.0$\times$	Top 1% multiplier

B Parameter Justification

This appendix provides detailed rationale for key protocol parameters. Parameters are grouped by subsystem and include analysis of alternatives considered.

B.1 Cryptographic Parameters

B.1.1 Ring Size: 20

The ring size determines the anonymity set for sender privacy.

Table 43: Ring size trade-off analysis

Ring Size	Signature Size	Nominal Anonymity	Effective Anonymity*
11 (Monero)	∼400 B	1/11	1/3–1/5
16	∼550 B	1/16	1/4–1/6
20 (Botho)	**∼700 B**	**1/20**	**1/5–1/8**
32	∼1,100 B	1/32	1/7–1/10
64	∼2,200 B	1/64	1/10–1/15

**Effective anonymity accounts for chain analysis techniques (timing, amount correlation, decoy age distribution biases).*

Why 20?

- **Diminishing returns**: Doubling ring size from 20 to 40 adds only ∼0.5 bits of anonymity while doubling signature size
- **Practical anonymity**: Ring size 20 provides meaningful protection against passive observers
- **Size budget**: At 700 bytes per input, multi-input transactions remain practical (<5 KB for 2-in-2-out)
- **Decoy availability**: Sufficient recent outputs exist in moderate-activity network

Why not Monero's 11?

- Research shows Monero's effective anonymity is lower than nominal
- Empirical traceability studies report success rates suggesting <5 effective ring members for Monero-style rings [42]
- Botho's 20 provides meaningful improvement with acceptable overhead

B.1.2 ML-KEM Security Level: 768

ML-KEM-768 provides NIST Security Level 3 (equivalent to AES-192).

Why not ML-KEM-512 (Level 1)?

- Recipient privacy is permanent—data persists indefinitely on-chain
- Conservative security margin appropriate for long-term protection
- Size difference minimal (800 vs 1088 bytes ciphertext)

Why not ML-KEM-1024 (Level 5)?

- Level 5 adds 50% size overhead (1568 byte ciphertext)
- No known attacks approach Level 3 security
- Can upgrade if Level 3 becomes threatened

B.2 Consensus Parameters

B.2.1 Block Time: 3–40 Seconds (Dynamic)

Block time is selected from five discrete load levels (Section 7.3); it is not fixed.

Table 44: Block time parameter analysis

Bound	Value	Constraint
Consensus floor	3 seconds	Minimum SCP externalization time
Idle ceiling	40 seconds	User experience, emission rate
Monetary reference	5 seconds	Sustained-high-load level

Why not fixed 10 seconds (like many chains)?

- Fixed time wastes capacity during high demand
- Fixed time over-emits during low utilization periods
- Dynamic adjustment aligns emission with network utility

Lower bound (3s): the consensus floor is the fastest dynamic-timing tier, 3 seconds at $\geq$20 tx/s (`ConsensusConfig::MIN_BLOCK_TIME_SECS`, `botho/src/consensus/service.rs`; `BLOCK_TIME_LEVELS`, `botho/src/block.rs`, with a compile-time check that the two agree). The budget is roughly:

- SCP nomination: $\sim$1 second
- Ballot prepare: $\sim$1 second
- Ballot commit and propagation: $\sim$1 second

This leaves no explicit safety margin, which is why 3 seconds is used only under sustained very high load; the 5-second tier is the monetary reference.

B.2.2 Quorum Thresholds: 3-of-4 Infrastructure, 2-of-3 Community

Infrastructure tier (3-of-4):

- Tolerates 1 Byzantine or offline node
- Requires 75% agreement (standard BFT threshold)
- Balances availability with safety

Community tier (2-of-3):

- Tolerates 1 Byzantine or offline node
- Lower threshold acknowledges higher variance in community uptime
- Inner set structure prevents single community node from blocking

Why tiered structure?

- Pure flat structure requires trusting all validators equally
- Infrastructure tier provides stability baseline
- Community tier ensures decentralization input
- Combined structure requires both tiers for consensus

B.2.3 Difficulty Adjustment: Per-Block, Time-Driven

The live difficulty controller is `block::difficulty::EmissionController`, whose `record_block` path retargets difficulty *every block* rather than on a fixed multi-block window. Each block, the controller compares the observed inter-block time against the target (`TARGET_BLOCK_TIME_SECS` = 5 s) and applies one integer, basis-point (`u128`) adjustment step. There is no accumulate-then-retarget epoch: the retarget cadence *is* the block cadence.

Why per-block time-driven?

- Responds to hashrate changes within a single block instead of waiting for a window to fill
- Driven purely by observed block time, not transaction count, so a producer cannot skew difficulty by stuffing or starving blocks (M5, #554)
- Consensus-critical: all math is integer basis-point arithmetic (no `f64`), so every node computes the identical difficulty (`RATIO_SCALE` = 10,000; audit cycle-6 C5, #552)

Per-block adjustment bounds (0.5× to 2×):

- Each step is clamped to [`RATIO_SCALE`/2, `MAX_ADJUSTMENT_FACTOR_BPS`] = [0.5×, 2.0×] per block
- A single (consensus-bounded but loose) timestamp gap is additionally capped at `MAX_OBSERVED_BLOCK_TIM` = 12 target intervals ($\leq$ 60 s at 5 s target) before it feeds the controller
- Prevents oscillation while still permitting recovery from a large hashrate drop over a few consecutive blocks

Note on the 17,280-block interval. The `MonetaryPolicy.difficulty_adjustment_interval` field in `mainnet_policy()` is set to `BLOCKS_PER_YEAR`/365 = 17,280 blocks (one day at 5 s). This is *not* the live retarget cadence: it is consumed only by the simulation-tooling `DifficultyController` in the `cluster-tax` crate, which has no callers on the node's block path (the node's live controller is the per-block `EmissionController` above). The 17,280 figure is a daily block count (it reappears as `blocks_per_day` in the halving derivation below), not a difficulty window. Prior revisions of this appendix cited a 144-block window (Bitcoin's 2016-block / ∼2-week retarget scaled to a ∼12-minute interval); no such constant exists in the Botho node, and it is dropped here as a stale port.

B.3 Economic Parameters

B.3.1 Initial Block Reward: 50 BTH

Rationale:

- Round number for simplicity
- Matches Bitcoin's initial reward (symbolic continuity)
- At 5-second blocks: ≈864,000 BTH/day initially
- Creates sufficient early liquidity for network bootstrap

B.3.2 Halving Period: 6,307,200 Blocks (∼1 Year)

Why ∼1-year halvings (vs. Bitcoin's 4-year)?

- Faster initial distribution to early participants

- Reaches tail emission sooner (∼5 years), stabilizing economics
- At 5-second blocks: 6,307,200 blocks ≈ 365 days = 1.0 year per halving epoch, with 5 halvings totaling ∼5 years

Calculation:

$$\text{blocks_per_day} = \frac{86400\text{s}}{5\text{s/block}} = 17{,}280$$

$$\text{blocks_per_year} = 17{,}280 \times 365 = 6{,}307{,}200$$

The five halving epochs ($R_0 = 50$ BTH, halving each year to 25, 12.5, 6.25, 3.125 BTH/block) emit a geometric sum that fixes the Phase-1 supply at $50 \times 6{,}307{,}200 \times \frac{31}{16} = 611{,}010{,}000$ BTH (≈611M).

B.3.3 Tail Emission: 2.5% Gross Annual Rate (Supply-Dependent)

The tail is not a fixed per-block constant but a fixed annual rate on the cumulative issuance, recomputed every block. Gross per-block emission is $R_{\text{tail}} = S \cdot (\beta_{\text{net}} + \beta_{\text{burn}})/N_{\text{year}}$ with $\beta_{\text{net}} = 2\%$, $\beta_{\text{burn}} = 0.5\%$ (assumed fee burn), and $N_{\text{year}} = 6{,}307{,}200$.

Security budget analysis (at the ≈611M Phase-1 supply):

- Gross emission = 611,010,000 × 0.025 = 15,275,250 BTH/year ⇒≈ 2.42 BTH/block
- Net emission = 611,010,000 × 0.020 = 12,220,200 BTH/year ⇒≈ 1.94 BTH/block net of the assumed fee burn
- Design target: 2% net, which holds only if burns reach 0.5%/year. Burning that much in base fees would take ≈4,800 tx/s at the relay minimum, so the realized net rate is ≈2.1–2.53% (Section 7, Realized net rate)
- Because $R_{\text{tail}} \propto S$, the per-block reward grows with supply and gross issuance compounds at ≈2.53% per N_{year} blocks

Why not 0?

- Zero tail emission requires fees to fund all security
- Fee-only security creates volatility and potential death spirals
- Perpetual emission ensures predictable security budget

Why a supply-proportional rate (rather than a fixed BTH constant)?

- A fixed per-block constant would yield ever-shrinking percentage inflation, eventually starving the security budget relative to supply
- Pinning net inflation at 2% keeps the security budget a stable fraction of the economy and acts as a mild, predictable wealth tax that rewards circulation over idle holding
- The rate can be adjusted via governance if the security budget proves insufficient

B.3.4 Fee Split: 80% Lottery / 20% Burn

Why redistribute 80%?

- Pure burn benefits all holders in proportion to their holdings
- The lottery was intended to favor small holders statistically
- 80% was chosen to give the lottery a meaningful pool

At production parameters the lottery does not measurably reduce inequality and ticket-splitting is profitable; a redesign is under study (Section 10.6.2). The split itself is unchanged.

Why burn 20%?

- Some deflation counters tail emission inflation
- Creates tangible "cost" to network usage
- Prevents infinite supply growth from fees alone

Alternative considered: 90/10

- Larger lottery pool but less deflationary pressure
- 80/20 was judged the better balance

B.3.5 Progressive Fee Multiplier: 1× to 6×

Log-domain sigmoid (see Section 5.4.5):

$$\text{factor} = 1 + 5 \cdot \sigma\left(\frac{\log_2 W_{\text{eff}} - \log_2 W_{\text{mid}}}{\omega}\right), \quad W_{\text{mid}} = 100{,}000 \text{ BTH}, \ \omega = 2.0.$$

The curve rises across *orders of magnitude* of effective cluster wealth rather than linearly, so a factor-1 floor covers the vast majority of holders and the midpoint (3.5×) sits at 100k BTH.

Why maximum 6×?

- High enough to make the fee schedule meaningfully progressive
- Low enough to not price out large legitimate transactions
- Comparable to progressive tax brackets in traditional systems

Why sigmoid shape?

- Smooth transition avoids cliff effects
- Asymptotic bound prevents infinite fees
- Most users (bottom 50%) experience near-base fees

B.3.6 Tag Decay: 0.95 per Transfer (Wallet Policy)

As built (Section 5.4.4):

- Stock wallets retain 0.95 of each tag weight per transfer (`DEFAULT_CLUSTER_DECAY_RATE` = 50,000 ppm, `botho/src/wallet.rs`, `botho-wallet/src/transaction.rs`), with no age gate
- Half-life: $\ln(0.5)/\ln(0.95) \approx 13.5$ transfers, with no time bound

- Consensus applies no decay (`CONSENSUS_TAG_DECAY_RATE` = 0, `botho/src/ledger/store.rs`) and rejects only tag inflation, so lower or empty output tags are valid (pending #1687)

Not implemented: 720-block minimum age. Earlier drafts gated decay on a 720-block minimum UTXO age, which would cap a wash trader at one decay event per 720 blocks. That rule exists only as a library model (`cluster-tax/src/age_decay.rs`) with no production caller, so the event rates and hour-based half-lives quoted for it do not apply.

B.4 Network Parameters

B.4.1 Maximum Transaction Size: 100 KB

Rationale:

- Allows 16-in-16-out transactions (∼80 KB)
- Prevents single transactions from dominating blocks
- Provides headroom for future feature expansion

B.4.2 Maximum Block Size: 2 MB

Why 2 MB?

- At 4 KB average transaction: ∼500 transactions per block
- At 5-second blocks: ∼100 TPS theoretical maximum
- Manageable for full node storage and bandwidth
- Can increase via soft fork if network capacity grows

B.4.3 Connection Limits: 8 Outbound, 117 Inbound

Outbound (8):

- Sufficient for network connectivity
- Limits resource consumption for light nodes
- Eclipse attack resistance (diverse peer selection)

Inbound (117):

- Allows serving many peers
- Total 125 matches common socket limits
- Per-IP limit (2) prevents single-source flooding

B.5 Summary Table

C Regulatory Considerations

Disclaimer: This appendix provides technical information about Botho's capabilities relevant to regulatory compliance. It does not constitute legal advice. Users and service providers should consult qualified legal counsel regarding their specific obligations.

Table 45: Key parameter summary

Parameter	Value	Category	Governance
Ring size	20	Crypto	Soft fork
ML-KEM level	768	Crypto	Hard fork
Block time	3–40s (dynamic)	Consensus	Soft fork
Quorum threshold	3/4, 2/3	Consensus	Hard fork
Initial reward	50 BTH	Economic	Genesis
Halving period	6,307,200 blocks	Economic	Hard fork
Tail emission	2.5% gross/yr (2% net design)	Economic	Hard fork
Fee split	80/20	Economic	Soft fork
Max fee multiplier	6×	Economic	Soft fork
Max block size	2 MB	Network	Soft fork

C.1 Privacy and Compliance Tension

Privacy-preserving cryptocurrencies exist in tension with regulatory frameworks designed for transparent financial systems. Bothois designed to provide strong privacy by default while enabling selective transparency where legally required.

C.1.1 Design Philosophy

- **Privacy as default**: All users receive equal privacy protection regardless of transaction size or frequency
- **Voluntary disclosure**: Users can prove transaction details to third parties without compromising other transactions
- **No backdoors**: The protocol contains no mechanism for mass surveillance or privileged access
- **Technical neutrality**: The protocol neither facilitates nor prevents compliance—that is a user/application layer concern

C.2 View Key Disclosure

Bothosupports selective transparency through view key mechanisms.

C.2.1 View Key Capabilities

A view key allows a third party to:

- Identify incoming transactions to an address
- Decrypt transaction amounts for identified transactions
- Verify transaction inclusion in the blockchain
- Cannot identify outgoing transactions (requires spend key)
- Cannot spend funds (view-only access)

Table 46: View key disclosure scenarios

Scenario	Key Disclosed	Information Revealed
Tax audit	View key	Incoming transactions, amounts
Court order	View key	Incoming transactions, amounts
AML compliance	View key	Deposit monitoring
Full disclosure	View + Spend	Complete transaction history

C.2.2 Audit Scenarios

C.2.3 View Key Limitations

View keys cannot reveal:

- Which ring members were actually spent (sender privacy preserved)
- Recipients of outgoing transactions
- Total balance without scanning full blockchain

Implication: Complete audit trails require cooperation from transaction counterparties or additional record-keeping.

C.3 Selective Transparency Options

C.3.1 Payment Proofs

Users can generate cryptographic proofs of specific payments:

```
PaymentProof
    tx_hash: Hash,              // Transaction identifier
    recipient_address: Address, // Claimed recipient
    amount: u64,                // Claimed amount
    proof: ZKProof,             // Cryptographic proof


// Verifier can confirm:
// 1. Transaction exists in blockchain
// 2. Output belongs to claimed recipient
// 3. Amount matches claim
```

Use case: Proving payment to tax authority or in dispute resolution.

C.3.2 Income Proofs

Users can prove receipt of funds without revealing sender:

```
IncomeProof
    outputs: Vec<OutputRef>,    // List of received outputs
    amounts: Vec<u64>,          // Corresponding amounts
    proof: ZKProof,             // Ownership proof
```

Use case: Demonstrating income for loan applications or tax filing.

C.3.3 Balance Proofs

Users can prove minimum balance without revealing exact amount:

```
BalanceProof
    minimum: u64,               // Claimed minimum balance
    proof: ZKProof,             // Proof that balance >= minimum
```

Use case: Proof of reserves, creditworthiness verification.

C.4 Exchange Integration

C.4.1 Know Your Customer (KYC)

Exchanges can implement KYC at the application layer:

- Identity verification before account creation
- Address association with verified identities
- Transaction monitoring for associated addresses
- View key access for audit purposes

Note: KYC is an exchange policy, not a protocol feature.

C.4.2 Travel Rule Compliance

The FATF Travel Rule requires transmission of originator/beneficiary information for transfers above thresholds.

Technical options:

1. **Off-chain**: Exchange-to-exchange communication of customer data (protocol-agnostic)
2. **Encrypted memo**: Optional encrypted field in transaction for compliance data (requires recipient cooperation)
3. **VASP protocols**: Integration with Travel Rule compliance networks (Sygna, Notabene, etc.)

Botho position: The protocol provides optional encrypted memo fields; compliance implementation is left to service providers.

C.4.3 Suspicious Activity Reporting

Exchanges can monitor for suspicious patterns:

- Unusual transaction volumes
- Structuring (splitting to avoid thresholds)
- Rapid deposit/withdrawal cycles
- Connections to flagged addresses (limited effectiveness)

Limitation: Ring signatures prevent definitive source tracing; exchanges can only monitor their own customer activity.

Table 47: Privacy coin compliance capabilities

Feature	Botho	Monero	Zcash	Bitcoin
View key audit	Yes	Yes	Yes*	N/A
Payment proofs	Yes	Yes	Yes	Yes
Selective disclosure	Yes	Limited	Yes	N/A
Transparent mode	No	No	Yes	Default
Chain analysis	Limited	Limited	Possible*	Yes

C.5 Comparison to Other Privacy Approaches

**Zcash shielded transactions; transparent transactions are fully auditable.*

C.6 Jurisdictional Considerations

Privacy coin regulations vary significantly by jurisdiction:

- **Permitted**: Most jurisdictions permit privacy coins for personal use
- **Exchange restrictions**: Some jurisdictions prohibit exchange listing (Japan, South Korea for certain coins)
- **Reporting requirements**: Many jurisdictions require reporting of cryptocurrency holdings/gains
- **AML obligations**: Service providers have varying obligations depending on jurisdiction

Recommendation: Users should understand their local regulatory environment before using any cryptocurrency.

C.7 Technical Measures for Service Providers

Service providers can implement compliance measures:

C.7.1 Deposit Monitoring

```
ExchangeDepositMonitor
    // Use view key to scan for deposits
    scan_for_deposits(view_key, start_height) -> Vec<Deposit>

    // Associate with customer account
    associate_deposit(deposit, customer_id)

    // Generate audit trail
    export_deposits(customer_id, date_range) -> AuditReport
```

C.7.2 Withdrawal Controls

```
WithdrawalRequest
    customer_id: CustomerId,
    amount: u64,
    destination: Address,


// Compliance checks:
```

```
// 1. Customer KYC status verified
// 2. Amount within limits
// 3. Destination not on sanctions list (limited effectiveness)
// 4. Travel rule data prepared (if applicable)
```

C.8 Limitations and Honest Assessment

C.8.1 What Botho Cannot Provide

- **Sender identification**: Ring signatures prevent identifying which input was spent
- **Address clustering**: Stealth addresses prevent linking outputs to addresses
- **Amount revelation**: Without view key, amounts are hidden
- **Global surveillance**: No mechanism for mass transaction monitoring

C.8.2 Regulatory Risk

Users should be aware:

- Privacy coins may face increasing regulatory scrutiny
- Exchange availability may be limited in some jurisdictions
- Tax obligations apply regardless of transaction privacy
- Using privacy features does not exempt from legal obligations

C.9 Conclusion

Bothoprovides strong privacy by default while enabling users to voluntarily disclose transaction information where required. The protocol is designed to be technically neutral—compliance is a user and service provider responsibility, not enforced at the protocol level.

This approach reflects the belief that privacy is a fundamental right, while acknowledging that legitimate compliance requirements exist. Users and service providers must navigate this balance according to their specific legal obligations.

D Formal Specifications

This appendix provides formal state machine specifications for the Bothoprotocol, suitable for verification and implementation reference.

D.1 State Definitions

D.1.1 Blockchain State

The blockchain state Σ is a tuple:

$$\Sigma = (\mathcal{U}, \mathcal{I}, \mathcal{C}, h, t) \tag{78}$$

where:

- $\mathcal{U}$: UTXO set (unspent transaction outputs)
- $\mathcal{I}$: Key image set (spent output identifiers)
- $\mathcal{C}$: Cluster tag database

- h: Current block height
- t: Latest block timestamp

UTXO Structure:

```
UTXO =
    outpoint: (block_height, tx_index, output_index),
    commitment: CompressedPoint,  // Pedersen commitment
    one_time_key: CompressedPoint,
    kem_ciphertext: [u8; 1088],
    cluster_tag: ClusterTag,
    created_height: u64,


ClusterTag =
    weights: Map<ClusterId, Rational>,  // Sum to 1
```

Key Image Set:

$$\mathcal{I} = \{I \in \mathbb{G} : I \text{ has been spent}\} \tag{79}$$

Key images are deterministic: $I = x \cdot H_p(P)$ where x is the private key and P is the one-time public key.

D.1.2 Consensus State

The SCP consensus state Ξ for slot s is:

$$\Xi_s = (\phi_s, \beta_s, M_s, V_s) \tag{80}$$

where:

- $\phi_s \in \{\text{NOMINATING}, \text{PREPARING}, \text{COMMITTING}, \text{EXTERNALIZED}\}$
- β_s: Current ballot (n, v) where n is counter, v is value
- M_s: Set of received messages
- V_s: Set of nominated values

D.2 State Transition Functions

D.2.1 Block Application

The state transition function $\delta : \Sigma \times B \to \Sigma$ applies block B to state Σ:

```
function apply_block(state: State, block: Block) -> Result<State>
    // 1. Verify block header
    require(block.prev_hash == hash(state.latest_block))
    require(block.height == state.height + 1)
    require(verify_pow(block.header))

    // 2. Verify minting transaction
    let reward = compute_block_reward(block.height)
    require(sum(block.minting_tx.outputs) <= reward)
    require(verify_mldsa(block.minting_tx.signature))

    // 3. Apply each transaction
    let new_state = state
```

```
    for tx in block.transactions
        new_state = apply_transaction(new_state, tx)?


    // 4. Add minting outputs to UTXO set
    for (i, output) in block.minting_tx.outputs.enumerate()
        let utxo = create_utxo(block, 0, i, output)
        new_state.utxo_set.insert(utxo)


    // 5. Process fee redistribution
    new_state = process_fees(new_state, block)?

    // 6. Update metadata
    new_state.height = block.height
    new_state.timestamp = block.timestamp
    new_state.latest_block = hash(block)

    Ok(new_state)
```

D.2.2 Transaction Application

The transaction application function:

```
function apply_transaction(
    state: State,
    tx: Transaction
) -> Result<State>
    // 1. Validate transaction structure
    require(1 <= tx.inputs.len() <= 16)
    require(1 <= tx.outputs.len() <= 16)

    // 2. Check key image uniqueness
    for input in tx.inputs
        require(!state.key_images.contains(input.key_image))


    // 3. Verify ring signatures
    for (input, signature) in zip(tx.inputs, tx.signatures)
        let ring = get_ring_members(state, input.ring)
        require(verify_clsag(signature, ring, tx.prefix_hash))


    // 4. Verify value conservation
    let input_sum = sum_commitments(get_input_commitments(tx))
    let output_sum = sum_commitments(tx.outputs.map(|o| o.commitment))
    let fee_point = tx.fee * H
    require(input_sum == output_sum + fee_point)

    // 5. Verify range proofs
    require(verify_bulletproof(tx.bulletproof, tx.outputs))

    // 6. Check minimum fee
    let cluster_factor = compute_cluster_factor(tx)
    let min_fee = base_fee * tx.size * cluster_factor
    require(tx.fee >= min_fee)
```

```
    // 7. Apply state changes
    let mut new_state = state

    // Remove spent outputs (we don't know which, but add key images)
    for input in tx.inputs
        new_state.key_images.insert(input.key_image)


    // Add new outputs
    for (i, output) in tx.outputs.enumerate()
        let utxo = create_utxo_from_output(tx, i, output)
        new_state.utxo_set.insert(utxo)


    Ok(new_state)
```

D.2.3 Fee Processing

Fee redistribution follows the 80/20 lottery/burn model with a cumulative, emission-fed pool and a per-block payout cap. The pseudocode below mirrors the protocol-7.0.0 consensus implementation (`botho/src/ledger/lottery_v2.rs drawing`, `botho/src/consensus/lottery.rs`, `botho/src/ledger/store/validation.rs lottery_candidates`, `cluster-tax/src/lottery.rs`); it is the authoritative specification of the draw referenced from Sections 7 and 10. Payout outputs are LotteryV2 records with independently keyed one-time targets; that binding is omitted here.

```
const WINNERS_PER_DRAW:   usize = 4;
const MIN_UTXO_AGE:       u64 = 720;               // blocks
const CIRCULATION_WINDOW: u64 = 10_000;            // blocks (max age)
const MIN_UTXO_VALUE:     u64 = 1_000_000;         // 1 microBTH, picocredits
const MAX_CANDIDATES:   usize = 10_000;            // candidate-set cap
const LOTTERY_BASE_FEE:   u64 = 250_000_000_000;   // 0.25 BTH cap unit (never charged)

function process_fees(state: State, block: Block) -> State
    let total_fees = sum(block.transactions.map(|tx| tx.fee))
    let (fee_pool, burn) = (80% * total_fees, 20% * total_fees) // burn destroyed

    // Seed: PoW fields of the parent (tip) header, never its header hash.
    let parent = state.tip_header
    let seed = SHA256("BOTHO_LOTTERY_SEED_V2" || parent.pow_hash()
                      || parent.prev_block_hash || block.height.le_bytes())

    // Candidates: read from the state AFTER the parent block (see #1689).
    let candidates = lottery_candidates(state.utxo_set, block.height, seed)
    let rho = candidates.len()

    // Cap R = min(rho * base_fee, block_reward). rho counts the window (#1682).
    let cap = min(rho * LOTTERY_BASE_FEE, block.reward)

    // Pool is cumulative (u128) and also fed by the block reward's
    // height-scheduled emission share (0, then +10%/epoch, cap 50%).
    let emission_share = lottery_emission_share(block.height, block.reward)
    let available: u128 = state.pool_carryover + emission_share + fee_pool
    let payout: u64 = min(available, cap)

    // No draw when payout == 0 or there are no candidates (winners = []).
```

```
    let winners = draw_uniform(candidates, payout, seed)
    for w in winners
        state = apply_lottery_payout(state, w.outpoint, w.payout)

    state.pool_carryover = available - sum(winners.map(|w| w.payout))
    state


// Seed-offset window over the UTXO set, eligibility by age band and dust floor.
function lottery_candidates(utxos: Set<UTXO>, height: u64, seed: Hash) -> Vec<UTXO>
    let start = offset_key(seed, height)               // 36-byte UTXO-id key
    let mut out = []
    for u in utxos.iter_from(start).wrap_around()     // key order, deterministic
        if out.len() >= MAX_CANDIDATES  break
        let age = height - u.created_height
        if age >= MIN_UTXO_AGE && age <= CIRCULATION_WINDOW
           && u.value >= MIN_UTXO_VALUE              // reads the amount
            out.push(u)                                 // coinbase/payouts included (#1690)


    out


// Verifiable uniform selection without replacement (SelectionMode::Uniform).
function draw_uniform(eligible: Vec<UTXO>, payout: u64, seed: Hash) -> Vec<Winner>
    if eligible.is_empty() || payout == 0  return []
    let n = min(WINNERS_PER_DRAW, eligible.len())
    let per_winner = payout / n
    let mut winners = []
    for i in 0..n
        let remaining = eligible.len() - winners.len()
        let roll = verifiable_random_u128(seed, i) % remaining   // deterministic
        winners.push(nth_unselected(eligible, roll, exclude=winners))

    assign_payouts(winners, per_winner, remainder_to_last=payout - per_winner*n)
```

D.3 Validity Predicates

D.3.1 Transaction Validity

A transaction T is valid with respect to state Σ iff:

$$\text{Valid}(T, \Sigma) \Leftrightarrow \bigwedge_{i=1}^{8} P_i(T, \Sigma) \tag{81}$$

where predicates P_i are:

1. P_1: **Structure**
$$1 \leq |T.\text{inputs}| \leq 16 \land 1 \leq |T.\text{outputs}| \leq 16$$

2. P_2: **Key Image Freshness**
$$\forall I \in T.\text{key_images} : I \notin \Sigma.\mathcal{I}$$

3. P_3: **Key Image Uniqueness (intra-tx)**
$$|T.\text{key_images}| = |\{I : I \in T.\text{key_images}\}|$$

4. P_4: **Ring Validity**
$$\forall r \in T.\text{rings} : |r| = 20 \wedge \forall o \in r : o \in \Sigma.\mathcal{U}$$

5. P_5: **Signature Validity**
$$\forall (r, \sigma, I) \in \text{zip}(T.\text{rings}, T.\text{sigs}, T.\text{key_images}) : \text{CLSAG.Verify}(r, \sigma, I, T.\text{hash})$$

6. P_6: **Value Conservation**
$$\sum_i C_{\text{in}}^{(i)} = \sum_j C_{\text{out}}^{(j)} + T.\text{fee} \cdot H$$

7. P_7: **Range Proofs**
$$\text{Bulletproof.Verify}(T.\text{proof}, \{C_{\text{out}}^{(j)}\})$$

8. P_8: **Fee Sufficiency**
$$T.\text{fee} \geq f_{\text{base}} \cdot |T|_{\text{bytes}} \cdot \phi(T)$$
where $\phi(T)$ is the cluster factor

D.3.2 Block Validity

A block B is valid with respect to state Σ iff:

$$\text{Valid}(B, \Sigma) \Leftrightarrow \bigwedge_{i=1}^{6} Q_i(B, \Sigma) \tag{82}$$

where predicates Q_i are:

1. Q_1: **Chain Linkage**
$$B.\text{prev_hash} = \mathcal{H}(\Sigma.\text{latest_block})$$

2. Q_2: **Height Increment**
$$B.\text{height} = \Sigma.h + 1$$

3. Q_3: **Proof of Work**
$$\mathcal{H}(B.\text{header}) < B.\text{target}$$

4. Q_4: **Timestamp Validity**
$$\Sigma.t < B.t \leq \text{now} + \epsilon$$

5. Q_5: **Minting Validity**
$$\sum B.\text{mint.outputs} \leq R(B.\text{height}) \wedge \mathcal{H}\big(B.\text{mint.nonce} \| B.\text{prev} \| B.\text{mint.}\mathsf{pk}_{\text{view}} \| B.\text{mint.}\mathsf{pk}_{\text{spend}}\big) < B.\text{target}$$

6. Q_6: **Transaction Validity**
$$\forall T \in B.\text{transactions} : \text{Valid}(T, \Sigma')$$
where Σ' is state after applying preceding transactions

D.4 SCP State Machine

D.4.1 Message Types

```
enum SCPMessage
    Nominate
        slot: u64,
        voted: Set<Value>,
        accepted: Set<Value>,
    ,
    Prepare
        slot: u64,
        ballot: Ballot,
        prepared: Option<Ballot>,
        prepared_prime: Option<Ballot>,
        quorum_set_hash: Hash,
    ,
    Commit
        slot: u64,
        ballot: Ballot,
        prepared_counter: u32,
        quorum_set_hash: Hash,
    ,
    Externalize
        slot: u64,
        commit: Ballot,
        quorum_set_hash: Hash,
    ,


struct Ballot
    counter: u32,
    value: Value,  // Block hash
```

D.4.2 State Transitions

```
function process_scp_message(
    state: SCPState,
    msg: SCPMessage,
    sender: NodeId
) -> SCPState
    match msg
        Nominate  slot, voted, accepted  =>
            // Update nomination state
            for v in voted
                if sender in state.quorum_slice
                    state.slots[slot].voted.insert(v)


            for v in accepted
                if has_blocking_set(state, sender, v)
                    state.slots[slot].accepted.insert(v)



            // Check if nomination complete
            if has_quorum(state, state.slots[slot].accepted)
                state.slots[slot].phase = PREPARING
```

```
            let value = pick_composite_value(state.slots[slot].accepted)
            state.slots[slot].ballot = Ballot  counter: 1, value

    ,

    Prepare  slot, ballot, ..  =>
        // Process prepare vote
        if state.slots[slot].phase >= PREPARING
            update_prepare_state(state, slot, ballot, sender)

            // Check for commit transition
            if can_commit(state, slot)
                state.slots[slot].phase = COMMITTING


    ,

    Commit  slot, ballot, ..  =>
        // Process commit vote
        if state.slots[slot].phase >= COMMITTING
            update_commit_state(state, slot, ballot, sender)

            // Check for externalize
            if has_quorum_commit(state, slot)
                state.slots[slot].phase = EXTERNALIZED
                emit_externalize(state, slot)


    ,

    Externalize  slot, commit, ..  =>
        // Accept externalize from quorum
        if has_quorum_externalize(state, slot, commit)
            state.slots[slot].phase = EXTERNALIZED
            state.slots[slot].externalized_value = Some(commit.value)

    ,


state
```

D.5 Invariants

The following invariants are maintained:

1. **Key Image Uniqueness**

$$\forall I \in \mathcal{I} : |\{T : I \in T.\text{key_images}\}| = 1$$

2. **UTXO Conservation**

$$\sum_{U \in \mathcal{U}} \text{value}(U) + \text{burned} = \text{total_minted}$$

3. **Cluster Tag Normalization**

$$\forall U \in \mathcal{U} : \sum_{c} U.\text{cluster_tag.weights}[c] = 1$$

4. **SCP Safety**

$$\forall s, v_1, v_2 : \text{externalized}(s, v_1) \wedge \text{externalized}(s, v_2) \Rightarrow v_1 = v_2$$

5. **Chain Integrity**

$$\forall h > 0 : B_h.\text{prev_hash} = \mathcal{H}(B_{h-1})$$

These invariants can be checked by a model checker or used as assertions in implementation testing.

E Security Audit Guide

This appendix provides guidance for security auditors reviewing the Bothoprotocol and implementation.

E.1 Security Claims

The following security properties are claimed by Botho. Auditors should verify each claim against the implementation.

E.1.1 Cryptographic Security

Table 48: Cryptographic security claims

Property	Primitive	Assumption
Recipient unlinkability	ML-KEM-768	IND-CCA2 security of ML-KEM
Sender anonymity	CLSAG	DLP hardness in Ristretto255
Amount confidentiality	Pedersen + BP	DLP hardness, random oracle model
Double-spend prevention	Key images	Collision resistance of H_p
Minting authenticity	PoW preimage binding	Preimage resistance of RandomX

E.1.2 Consensus Security

1. **Fork freedom**: No two honest nodes externalize different blocks at the same height (Theorem 6.5)
2. **Liveness**: The network eventually makes progress under partial synchrony
3. **Byzantine tolerance**: Safety holds with up to f Byzantine nodes per quorum slice

E.1.3 Economic Security

1. **Sybil resistance**: Splitting wealth does not reduce total fees (Theorem 5.1), for wallets that write inherited tags; consensus accepts dropped tags (pending #1687)
2. **Lottery fairness**: Winner selection is uniform over eligible outputs and verifiable, seeded from the parent block's PoW fields (`BOTHO_LOTTERY_SEED_V2`), with the per-block payout capped at $\min(\rho \cdot 0.25\ \text{BTH}, R(h))$ (Appendix D). Auditors should treat these as open: candidate-set re-rolls through the parent's transactions (#1689), ticket-splitting under the implemented cap (#1682), free coinbase and payout tickets (#1690), carry-forward drain (#1691), and a PoW-regrind forfeit of $(1 - f_{\text{em}})R(h)$ rather than $R(h)$
3. **Inflation bounds**: Supply growth follows the emission schedule exactly

E.2 Threat Model

Auditors should consider the following adversary capabilities:

Table 49: Threat model assumptions

Adversary Type	Capabilities
Network adversary	Can observe, delay, or reorder messages; cannot forge signatures or break encryption
Computational adversary	Polynomial-time bounded; cannot solve DLP or break ML-KEM
Byzantine nodes	Up to f nodes per quorum slice may behave arbitrarily
Quantum adversary	Future: can break DLP but not lattice problems

E.3 Audit Scope

E.3.1 Critical Components

The following components require thorough review:

1. **Key derivation** (`crypto/keys.rs`)
   - BIP39 mnemonic handling
   - SLIP-10 derivation paths
   - ML-KEM key generation from seed
2. **Stealth addresses** (`crypto/stealth.rs`)
   - ML-KEM encapsulation/decapsulation
   - One-time key derivation
   - Output scanning correctness
3. **Ring signatures** (`crypto/clsag.rs`)
   - Signature generation and verification
   - Key image computation
   - Ring member selection
4. **Confidential transactions** (`crypto/bulletproofs.rs`)
   - Pedersen commitment arithmetic
   - Range proof generation and verification
   - Value conservation checks
5. **Consensus** (`consensus/scp.rs`)
   - Quorum slice validation
   - Ballot protocol state machine
   - Externalization logic
6. **Cluster tags** (`economics/cluster.rs`)
   - Tag inheritance calculation
   - Fee multiplier computation
   - Decay mechanism

E.3.2 Test Vectors

Auditors should verify the following test vectors. The hex digests below are truncated and *illustrative only*; the full byte-exact vectors are published with the reference test suite.

```
Mnemonic: "abandon abandon ... about" (standard 24-word)
Expected view key (hex): 0x7b3a...   (illustrative)
Expected spend key (hex): 0x4c1f...  (illustrative)
Expected ML-KEM public key (first 32 bytes): 0x9d2e...

Message: 0x0000...0000 (32 zero bytes)
Ring size: 20
Real index: 7
Expected key image (hex): 0x5f8c...  (illustrative)
Signature must verify: true

Curve: log-domain sigmoid, range 1.0x..6.0x
Midpoint w_mid: 100,000 BTH  -> factor 3.5x  (exact anchor)
Log-width:      2.0 octaves per sigmoid unit
Effective wealth: 1,000,000 BTH (~3.32 octaves above midpoint)
Expected factor: ~5.2x
```

E.4 Known Limitations

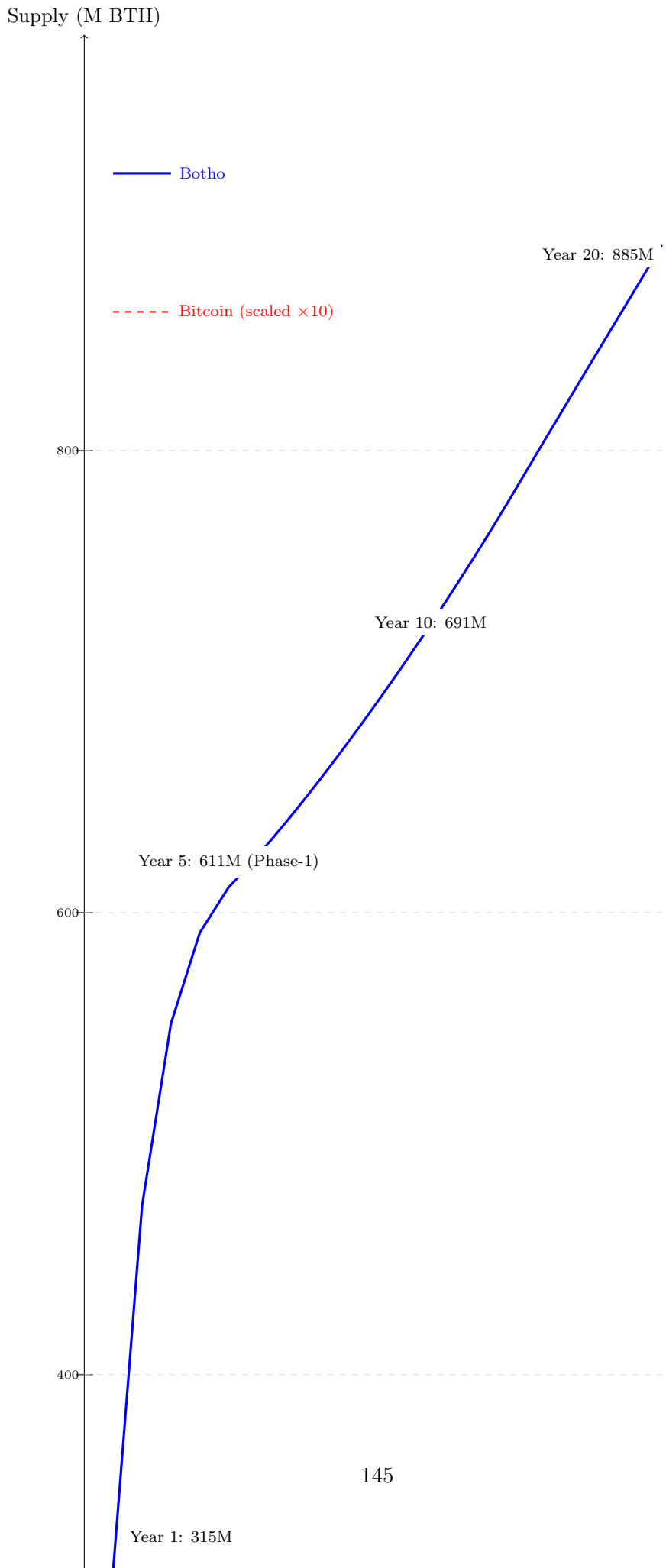
The following are acknowledged limitations, not vulnerabilities:

1. **Classical ring signatures**: CLSAG provides sender privacy against classical adversaries only. A future quantum computer could potentially identify signers retroactively.
2. **Metadata leakage**: Transaction size reveals approximate input/output count. Timing analysis may leak information despite Dandelion++.
3. **Ring selection bias**: The decoy selection algorithm uses a gamma distribution that may not perfectly match real spend patterns.
4. **Trusted setup**: Bulletproofs require no trusted setup, but the choice of generators G and H must be verifiably random.

E.5 Reporting

Security issues should be reported to:

- **Email**: security@botho.org (PGP key on website)
- **Severity levels**: Critical, High, Medium, Low, Informational
- **Bug bounty**: Available for critical and high severity issues


Supply (M BTH)
Botho
Bitcoin (scaled ×10)
Year 20: 885M
Year 10: 691M
Year 5: 611M (Phase-1)
Year 1: 315M
800
600
400

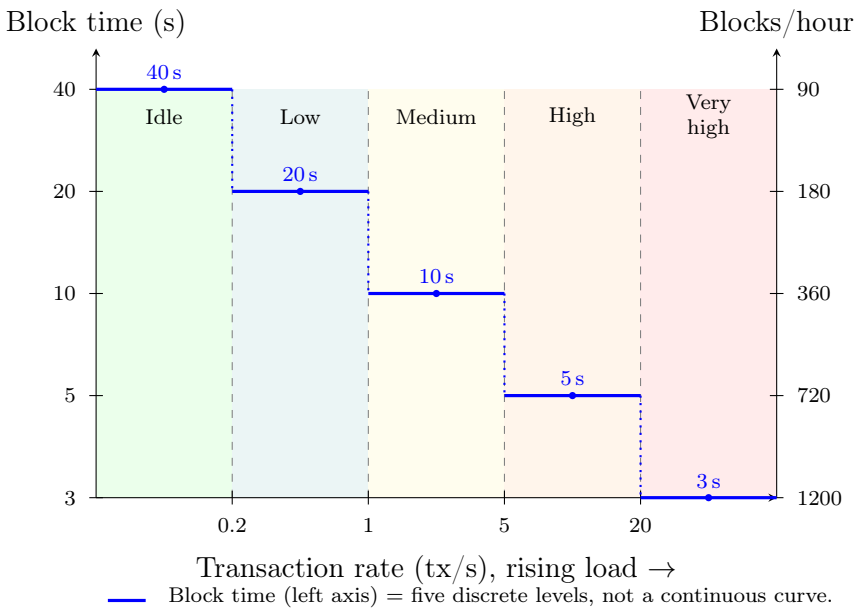


Figure 13: Dynamic block timing mechanism. The target block time is a deterministic *step function* of the recent transaction rate, selecting one of five discrete levels—40, 20, 10, 5, or 3 seconds—as network load rises. The interval steps down from the 40-second idle ceiling to the 3-second consensus floor (reached only under transient maximum demand), raising throughput from 90 to 1,200 blocks per hour. This creates natural inflation dampening: low usage holds slower blocks and less emission, while high usage steps to faster blocks to meet demand.

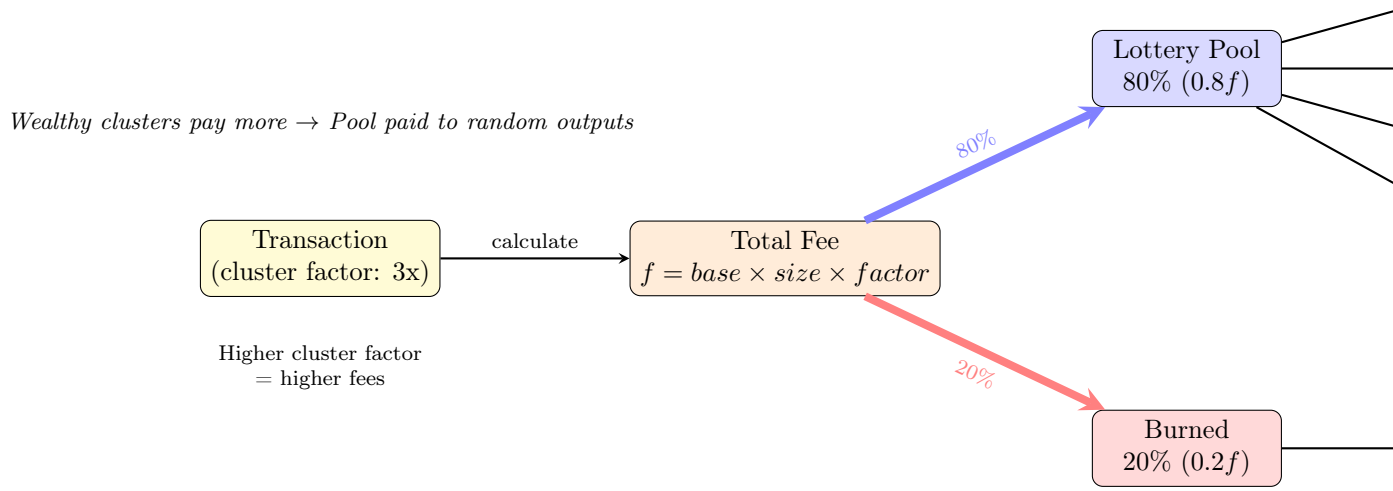


Figure 14: Fee flow mechanism. Transaction fees are calculated using a progressive cluster factor (1x–6x based on coin ancestry). 80% of fees are redistributed via verifiable lottery to 4 randomly selected UTXOs, while 20% is permanently burned, with deflationary pressure proportional to network usage. The flow is designed to move value from high-factor payers to random holders; a measurable reduction in inequality is not demonstrated at production parameters (Section 10.6.2).

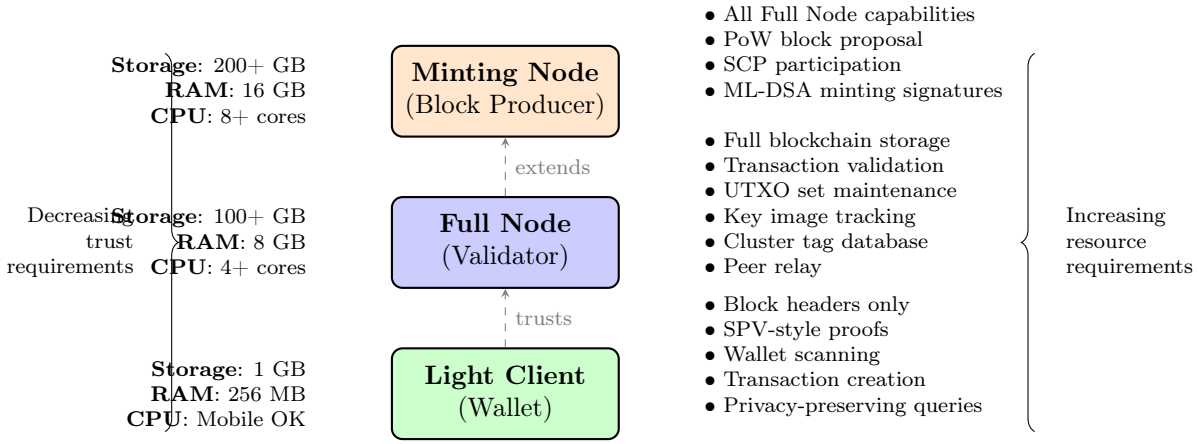


Figure 15: Node type hierarchy in Botho. Light clients trust full nodes for transaction validation but verify headers independently. Full nodes maintain complete state and validate all transactions. Minting nodes extend full nodes with block proposal capabilities. Resource requirements scale with trust assumptions—light clients trade some security for accessibility.

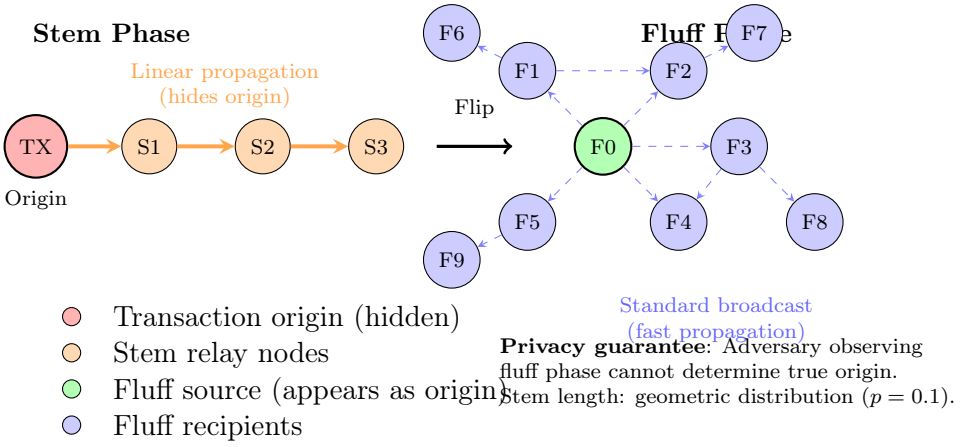


Figure 16: Dandelion++ transaction propagation [19]. In the *stem phase*, transactions propagate along a random linear path, obscuring the origin. After a probabilistic transition (average 10 hops), the *fluff phase* begins: standard broadcast propagation for fast network-wide dissemination. The fluff source appears to be the transaction origin, providing plausible deniability for the true sender.

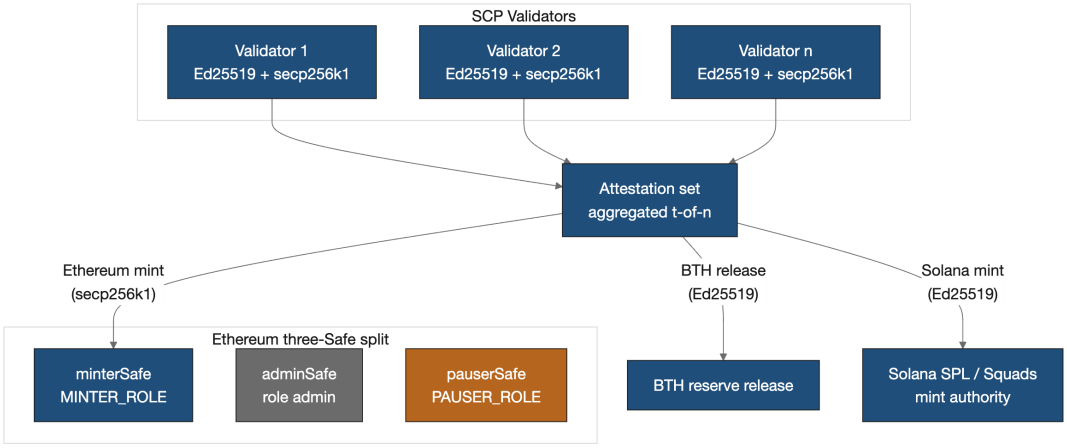


Figure 17: Federation custody and attestation topology. Each SCP validator operates an Ed25519 node key (BTH release + Solana mint) and a secp256k1 signer (Ethereum). Threshold envelopes are aggregated to t-of-n and consumed by the mint / release paths; the Ethereum authority is a three-Safe split.

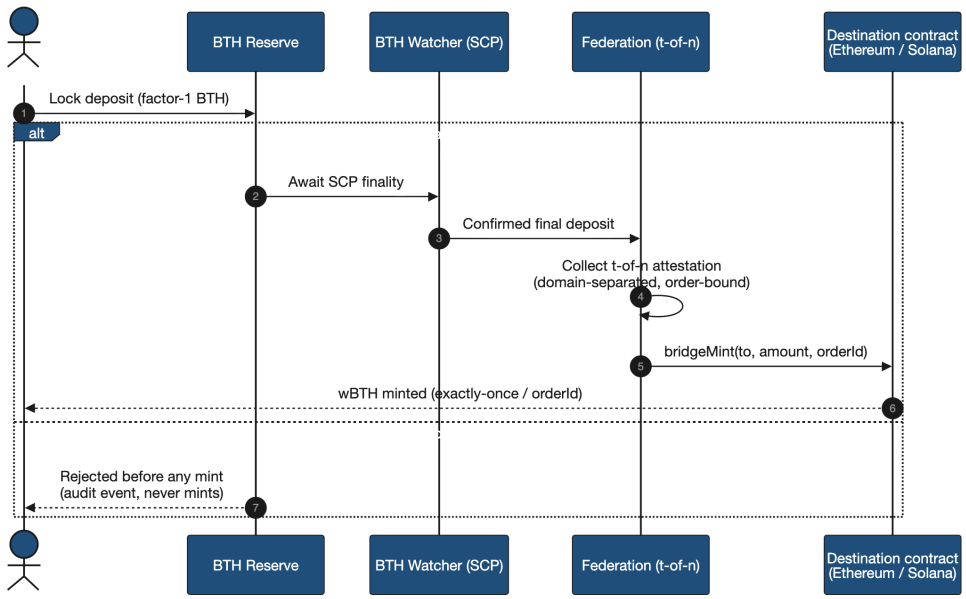


Figure 18: Wrap / lock → mint flow (BTH → wBTH). A factor-1 deposit is locked in the reserve, confirmed under SCP finality, attested by the federation to t-of-n, and minted on the destination chain against the deterministic `orderId`. A non-factor-1 deposit is rejected before any mint.

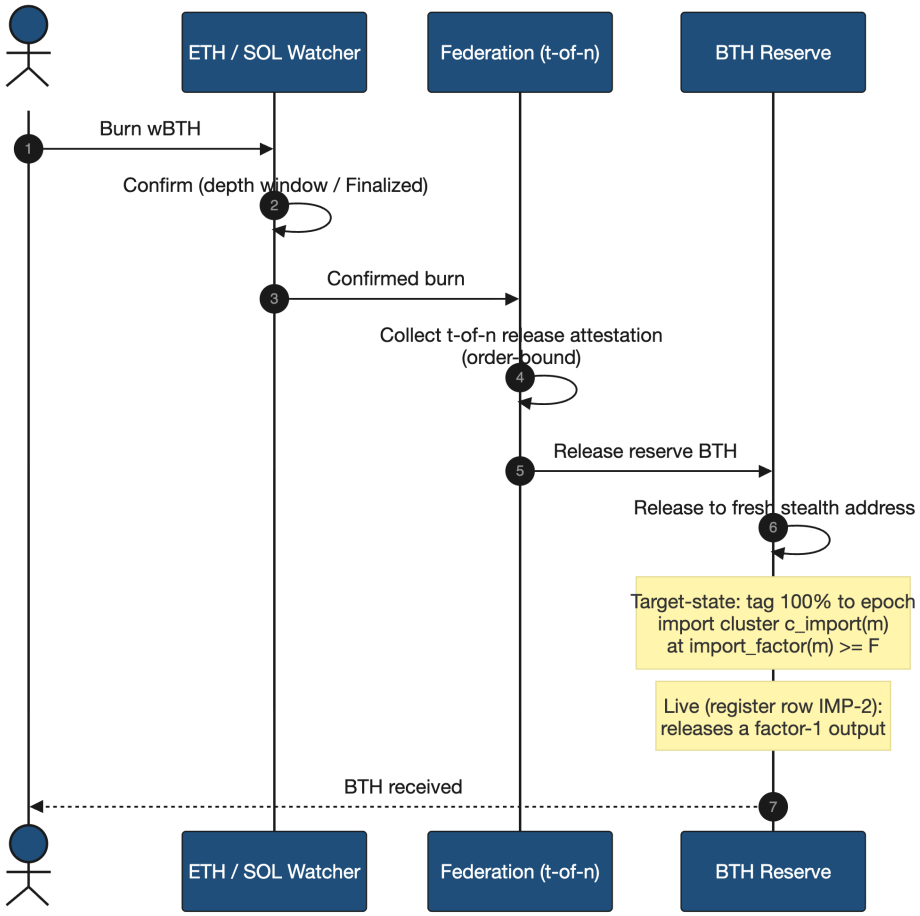


Figure 19: Unwrap → import-tagging flow (wBTH → BTH). A confirmed burn is attested and releases reserve BTH to a fresh stealth address, tagged 100% to the epoch import cluster $c_{\text{import}}(m)$ at import_factor$(m) \geq F$. *Live* as of protocol 5.0.0: the implementation tags the released output to the epoch import cluster at the elevated factor ($F = 1.5\times$ floor, $K = 1$ day epoch; register row IMP-2).